SOURCE CODE DEPOSIT COPY

Intellectual Property Office of the Philippines
Bureau of Copyright and Related Rights

Reference Number:	2026-06606
Title of Work:	MicroMeasure: A Web-Based Scientific Image Analysis and Measurement Tool
Applicant:	James Salveo L. Olarve
Affiliation:	i-Nano Research Facility, Department of Physics, De La Salle University Manila
Classification:	Class N (Computer Programs)
Programming Language(s):	HTML5 / CSS3 / JavaScript (client-side, single-file web application)
Software Version:	v1.3.0
Source File:	index.html

This deposit copy contains the complete, manually-authored source code of the above-titled computer program, submitted in fulfillment of the deposit requirement for works classified under Class N (Computer Programs). No redactions have been applied.

```
<!doctype html>
<html lang="en" class="h-full">
 <head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>MicroMeasure</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    function loadOptionalScript(src) {
      return new Promise((resolve) => {
        const script = document.createElement('script');
        script.src = src;
        script.async = true;
        script.onload = () => resolve(true);
        script.onerror = () => resolve(false);
        document.head.appendChild(script);
      });
    }

    const tiffDecoderReady = loadOptionalScript('https://cdn.jsdelivr.net/npm/utif@3.1.0/UTIF.min.js');

    Promise.all([
      tiffDecoderReady,
      loadOptionalScript('./_sdk/element_sdk.js'),
      loadOptionalScript('./_sdk/data_sdk.js')
    ]);
  </script>
  <style>
    body {
      box-sizing: border-box;
    }
    @import
      url('https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;500&family=Inter:wght@400;500;600;700&display=swa
      ;

    * { box-sizing: border-box; }

    :root {
      --bg-primary: #0f172a;
      --bg-secondary: #1e293b;
      --text-primary: #f1f5f9;
      --accent-primary: #3b82f6;
      --accent-secondary: #64748b;
      --muted: #94a3b8;
      --line: rgba(255,255,255,0.10);
    }

    body {
      font-family: 'Inter', sans-serif;
      background: var(--bg-primary);
      color: var(--text-primary);
    }

    .mono { font-family: 'JetBrains Mono', monospace; }

    .scrollbar-thin::-webkit-scrollbar { width: 6px; height: 6px; }
    .scrollbar-thin::-webkit-scrollbar-track { background: var(--bg-secondary); }
    .scrollbar-thin::-webkit-scrollbar-thumb { background: var(--accent-secondary); border-radius: 3px; }

    .tab-btn { transition: all 0.2s ease; }
    .tab-btn.active {
      background: var(--accent-primary);
      color: white;
      box-shadow: 0 4px 12px rgba(59, 130, 246, 0.3);
    }

    .tool-card {
      background: var(--bg-secondary);
      border-radius: 12px;
      border: 1px solid rgba(255,255,255,0.1);
    }

    .btn-primary {
      background: var(--accent-primary);
      transition: all 0.2s ease;
    }
    .btn-primary:hover {
      background: #2563eb;
      transform: translateY(-1px);
      box-shadow: 0 4px 12px rgba(59, 130, 246, 0.4);
    }
```

```
    .btn-primary:disabled {
      background: #475569;
      cursor: not-allowed;
      transform: none;
      box-shadow: none;
    }

    .btn-save-image {
      background: #16a34a;
      transition: all 0.2s ease;
    }
    .btn-save-image:hover {
      background: #15803d;
      transform: translateY(-1px);
      box-shadow: 0 4px 12px rgba(22, 163, 74, 0.4);
    }
    .btn-save-image:disabled {
      background: #475569;
      cursor: not-allowed;
      transform: none;
      box-shadow: none;
    }

    .btn-secondary {
      background: var(--accent-secondary);
      transition: all 0.2s ease;
    }
    .btn-secondary:hover { background: #475569; }

    .btn-danger {
      background: #dc2626;
      transition: all 0.2s ease;
    }
    .btn-danger:hover { background: #b91c1c; }

    input[type="range"] {
      -webkit-appearance: none;
      appearance: none;
      background: transparent;
    }
    input[type="range"]::-webkit-slider-track {
      height: 6px;
      background: linear-gradient(to bottom, #1e293b, #334155, #1e293b);
      border-radius: 3px;
      border: 1px solid #64748b;
      box-shadow: inset 0 1px 3px rgba(0,0,0,0.5), 0 1px 0 rgba(255,255,255,0.1);
    }
    input[type="range"]::-webkit-slider-thumb {
      -webkit-appearance: none;
      appearance: none;
      width: 18px;
      height: 18px;
      background: linear-gradient(to bottom, #60a5fa, #3b82f6);
      border-radius: 50%;
      cursor: pointer;
      margin-top: -6px;
      border: 2px solid #1e293b;
      box-shadow: 0 2px 6px rgba(59, 130, 246, 0.6), inset 0 1px 0 rgba(255,255,255,0.3);
    }
    input[type="range"]::-webkit-slider-thumb:hover {
      background: linear-gradient(to bottom, #93c5fd, #60a5fa);
      box-shadow: 0 3px 8px rgba(59, 130, 246, 0.8), inset 0 1px 0 rgba(255,255,255,0.4);
    }

    .canvas-container {
      position: relative;
      overflow: hidden;
      background: #000;
      border-radius: 8px;
    }

    /* Mobile performance & interaction hints for canvases */
    #mainCanvas, #overlayCanvas {
      will-change: transform;
      touch-action: none;
    }

    .canvas-container.drag-over {
      outline: 3px dashed #3b82f6;
      outline-offset: -8px;
      background: rgba(59, 130, 246, 0.1);
```

```
    }

    #mainCanvas, #overlayCanvas {
      position: absolute;
      top: 0;
      left: 0;
    }

    .measurement-item {
      background: rgba(0,0,0,0.3);
      border-radius: 8px;
      padding: 8px 12px;
      margin-bottom: 6px;
      display: flex;
      justify-content: space-between;
      align-items: center;
      font-size: 13px;
    }

    .histogram-canvas {
      background: var(--bg-primary);
      border-radius: 8px;
    }

    .status-badge {
      display: inline-flex;
      align-items: center;
      gap: 6px;
      padding: 4px 10px;
      border-radius: 20px;
      font-size: 12px;
      font-weight: 500;
    }
    .status-badge.warning { background: rgba(245, 158, 11, 0.2); color: #fbbf24; }
    .status-badge.success { background: rgba(34, 197, 94, 0.2); color: #4ade80; }

    .particle-table {
      font-size: 11px;
      width: 100%;
    }
    .particle-table th, .particle-table td {
      padding: 6px 8px;
      text-align: left;
      border-bottom: 1px solid rgba(255,255,255,0.1);
    }
    .particle-table th {
      background: rgba(0,0,0,0.3);
      font-weight: 600;
      position: sticky;
      top: 0;
    }

    .processing-modal {
      position: fixed;
      top: 0;
      left: 0;
      right: 0;
      bottom: 0;
      background: rgba(0, 0, 0, 0.8);
      backdrop-filter: blur(4px);
      display: flex;
      align-items: center;
      justify-content: center;
      z-index: 1000;
      animation: fadeIn 0.2s ease;
    }

    /* Documentation overlay + modal (match Roughness Visualizer) */
    .overlay{
      position:fixed;
      inset:0;
      background:rgba(0,0,0,0.55);
      display:flex;
      align-items:flex-start;
      justify-content:center;
      padding:18px;
      z-index:9999;
      overflow:auto;
    }
    .overlay.hidden{ display:none !important; }
    .modal{
```

```
      width:min(720px, 100%);
      max-height: calc(100dvh - 36px);
      background:var(--bg-secondary);
      border:1px solid rgba(255,255,255,0.12);
      border-radius:16px;
      box-shadow:0 24px 80px rgba(0,0,0,0.55);
      display:flex;
      flex-direction:column;
    }
    .modalHead{
      display:flex;
      align-items:center;
      justify-content:space-between;
      padding:12px 14px;
      border-bottom:1px solid rgba(255,255,255,0.10);
      gap:10px;
      flex: 0 0 auto;
    }
    .modalTitle{ font-weight:950; }
    .modalBody{
      padding:14px;
      color:var(--text-primary);
      line-height:1.45;
      overflow:auto;
      -webkit-overflow-scrolling: touch;
      flex: 1 1 auto;
      min-height: 0;
    }
    .iconBtn{
      width:32px;
      height:32px;
      border-radius:12px;
      border:1px solid rgba(255,255,255,0.14);
      background:rgba(255,255,255,0.06);
      color:var(--text-primary);
      cursor:pointer;
      padding:0;
      display:inline-flex;
      align-items:center;
      justify-content:center;
      flex:0 0 auto;
    }
    .iconBtn:hover{ border-color:rgba(255,255,255,0.22); }

    /* Neutralize Tailwind color blocks inside documentation so it matches Roughness docs */
    .rvDoc{ color: var(--text-primary); }
    .rvDoc .text-blue-400,
    .rvDoc .text-blue-300,
    .rvDoc .text-white,
    .rvDoc .text-slate-300{ color: var(--text-primary) !important; }
    .rvDoc .text-slate-400,
    .rvDoc .text-slate-500,
    .rvDoc .text-slate-600{ color: var(--muted) !important; }
    .rvDoc a{ color: var(--accent-primary) !important; }
    .rvDoc .bg-slate-900\/50{ background: transparent !important; }
    .rvDoc .border-slate-700{ border-color: var(--line) !important; }

    .quickstart-modal {
      position: fixed;
      top: 0;
      left: 0;
      right: 0;
      bottom: 0;
      background: rgba(0, 0, 0, 0.55);
      backdrop-filter: blur(2px);
      display: flex;
      align-items: center;
      justify-content: center;
      z-index: 900;
      animation: fadeIn 0.2s ease;
    }

    .quickstart-content {
      background: var(--bg-secondary);
      border-radius: 16px;
      padding: 20px;
      width: min(420px, 92vw);
      border: 1px solid rgba(255,255,255,0.1);
      box-shadow: 0 20px 60px rgba(0,0,0,0.5);
      animation: slideUp 0.3s ease;
    }
```

```
323
324      .quickstart-step {
325        display: flex;
326        align-items: center;
327        gap: 10px;
328        padding: 10px 12px;
329        border-radius: 12px;
330        border: 1px solid rgba(255,255,255,0.08);
331        background: rgba(0,0,0,0.18);
332      }
333
334      .quickstart-step.current {
335        border-color: rgba(59, 130, 246, 0.45);
336      }
337
338      .quickstart-step.done {
339        opacity: 0.9;
340      }
341
342      .quickstart-check {
343        width: 18px;
344        height: 18px;
345        border-radius: 999px;
346        border: 2px solid rgba(148, 163, 184, 0.8);
347        flex: 0 0 auto;
348      }
349
350      .quickstart-step.done .quickstart-check {
351        border-color: rgba(34, 197, 94, 0.9);
352        background: rgba(34, 197, 94, 0.25);
353      }
354
355      @keyframes fadeIn {
356        from { opacity: 0; }
357        to { opacity: 1; }
358      }
359
360      .processing-content {
361        background: var(--bg-secondary);
362        border-radius: 16px;
363        padding: 32px;
364        max-width: 400px;
365        width: 90%;
366        border: 1px solid rgba(255,255,255,0.1);
367        box-shadow: 0 20px 60px rgba(0,0,0,0.5);
368        animation: slideUp 0.3s ease;
369      }
370
371      @keyframes slideUp {
372        from { transform: translateY(20px); opacity: 0; }
373        to { transform: translateY(0); opacity: 1; }
374      }
375
376      .spinner {
377        width: 48px;
378        height: 48px;
379        border: 4px solid rgba(59, 130, 246, 0.2);
380        border-top-color: #3b82f6;
381        border-radius: 50%;
382        animation: spin 0.8s linear infinite;
383        margin: 0 auto 20px;
384      }
385
386      @keyframes spin {
387        to { transform: rotate(360deg); }
388      }
389
390      .progress-bar-container {
391        width: 100%;
392        height: 8px;
393        background: rgba(0,0,0,0.3);
394        border-radius: 4px;
395        overflow: hidden;
396        margin-top: 16px;
397      }
398
399      .progress-bar {
400        height: 100%;
401        background: linear-gradient(90deg, #3b82f6, #60a5fa);
402        border-radius: 4px;
403        transition: width 0.3s ease;
```

```
      box-shadow: 0 0 10px rgba(59, 130, 246, 0.5);
    }
  </style>
  <style>@view-transition { navigation: auto; }</style>
 </head>
 <body class="h-full overflow-auto">
  <div class="min-h-[100svh] flex flex-col"><!-- Header -->
   <header class="bg-slate-800/80 backdrop-blur border-b border-slate-700 px-4 py-3 flex items-center justify-between
      flex-shrink-0">
    <div class="flex items-center gap-3">
     <div class="w-10 h-10 rounded-xl bg-gradient-to-br from-blue-500 to-cyan-400 flex items-center justify-center">
      <svg class="w-6 h-6 text-white" fill="none" stroke="currentColor" viewbox="0 0 24 24"><path stroke-linecap="round"
      stroke-linejoin="round" stroke-width="2" d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0zM10 7v3m0 0v3m0-3h3m-3 0H7" />
      </svg>
     </div>
     <div>
      <h1 id="appTitle" class="text-lg font-bold text-white">MicroMeasure</h1>
      <p class="text-xs text-slate-400">Image Analysis</p>
     </div>
    </div>
    <div class="flex items-center gap-3">
     <button id="homeBtn" class="btn-secondary w-10 h-10 rounded-lg text-sm font-medium inline-flex items-center justify-cent
      title="Go to i-Nano Research Facility home page" aria-label="Go to i-Nano Research Facility home page">
      <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
       <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M3 10.5 12 3l9 7.5"/>
       <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M5.25 9.75V21h13.5V9.75"/>
       <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M9.75 21v-6h4.5v6"/>
      </svg>
     </button>
      <button id="loadDemoBtn" class="btn-secondary w-10 h-10 rounded-lg text-sm font-medium inline-flex items-center
      justify-center" title="Load a demo photo" aria-label="Load a demo photo">
       <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 16l4.586-4.586a2 2 0 0 1 2.828 0L16 16"/>
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M14 14l1.586-1.586a2 2 0 0 1 2.828 0L20 14"/
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 6h16v12H4z"/>
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M8.5 10.5h.01"/>
       </svg>
      </button>
     <button id="headerResetBtn" class="btn-danger w-10 h-10 rounded-lg text-sm font-medium inline-flex items-center
      justify-center" title="Reset MicroMeasure" aria-label="Reset MicroMeasure">
      <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
       <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4.5 12a7.5 7.5 0 0 1 12.78-5.303"/>
       <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19.5 12a7.5 7.5 0 0 1-12.78 5.303"/>
       <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M16.5 6.75V3.75h-3"/>
       <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M7.5 17.25v3h3"/>
      </svg>
     </button>
    <button id="helpBtn" class="btn-secondary w-10 h-10 rounded-lg text-sm font-medium inline-flex items-center justify-cente
      title="Documentation" aria-label="Documentation">
      <svg class="w-4 h-4" fill="none" stroke="currentColor" viewbox="0 0 24 24"><path stroke-linecap="round"
      stroke-linejoin="round" stroke-width="2" d="M8.228 9c.549-1.165 2.03-2 3.772-2 2.21 0 4 1.343 4 3 0 1.4-1.278 2.575-3.0
      2.907-.542.104-.994.54-.994 1.093m0 3h.01M21 12a9 9 0 11-18 0 9 9 0 0118 0z" />
     </svg></button> <button id="saveScreenshot" class="btn-save-image w-10 h-10 rounded-lg text-sm font-medium inline-flex
      items-center justify-center" title="Save Image" aria-label="Save Image" disabled>
      <svg class="w-4 h-4" fill="none" stroke="currentColor" viewbox="0 0 24 24"><path stroke-linecap="round"
      stroke-linejoin="round" stroke-width="2" d="M3 9a2 2 0 012-2h.93a2 2 0 001.664-.89l.812-1.22A2 2 0 0110.07 4h3.86a2 2 0
      011.664.89l.812 1.22A2 2 0 0018.07 7H19a2 2 0 012 2v9a2 2 0 01-2 2H5a2 2 0 01-2-2V9z" /> <path stroke-linecap="round"
      stroke-linejoin="round" stroke-width="2" d="M15 13a3 3 0 11-6 0 3 3 0 016 0z" />
      </svg></button>
    </div>
   </header><!-- Main Content -->
  <div class="flex-1 flex flex-col lg:flex-row gap-4 p-4 overflow-auto"><!-- Left Panel - Image Viewer -->
    <div class="lg:w-2/3 flex flex-col gap-3 min-h-0 flex-1"><!-- Image Controls -->
     <div class="tool-card p-3 flex flex-wrap items-center gap-4"><label class="btn-primary px-4 py-2 rounded-lg cursor-point
      text-sm font-medium flex items-center gap-2">
       <svg class="w-4 h-4" fill="none" stroke="currentColor" viewbox="0 0 24 24"><path stroke-linecap="round"
      stroke-linejoin="round" stroke-width="2" d="M4 16l4.586-4.586a2 2 0 012.828 0L16 16m-2-2l1.586-1.586a2 2 0 012.828 0L20
      14m-6-6h.01M6 20h12a2 2 0 002-2V6a2 2 0 00-2-2H6a2 2 0 00-2 2v12a2 2 0 002 2z" />
      </svg> Load Image <input type="file" id="imageInput" accept="image/*,.tif,.tiff" class="hidden"> </label>
      <div class="flex items-center gap-3 bg-slate-900/30 rounded-lg px-3 py-2 border border-slate-700/50"><span class="text-
      text-slate-400">Zoom:</span>
      <div class="bg-slate-800 rounded px-3 py-1.5 flex items-center gap-2"><input type="range" id="zoomSlider" min="50"
      max="500" value="100" class="w-32"> <span id="zoomValue" class="text-xs mono w-10">100%</span>
       </div>
      </div>
      <div id="imageDimensions" class="text-xs text-slate-400 mono hidden"><span id="imgWidth">0</span> × <span
      id="imgHeight">0</span> px
      </div><button id="fitToView" class="btn-secondary px-3 py-1.5 rounded-lg text-xs hidden">Fit to View</button> <!-- Scal
      Calibration Section -->
      <div class="flex items-center gap-2 bg-slate-900/30 rounded-lg px-3 py-2 border border-slate-700/50">
       <svg class="w-4 h-4 text-blue-400 flex-shrink-0" fill="none" stroke="currentColor" viewbox="0 0 24 24"><path
```

```
          stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19 11H5m14 0a2 2 0 012 2v6a2 2 0 01-2 2H5a2 2 0
          01-2-2v-6a2 2 0 012-2m14 0V9a2 2 0 00-2-2M5 11V9a2 2 0 012-2m0 0V5a2 2 0 012-2h6a2 2 0 012 2v2M7 7h10" />
466        </svg><span class="text-xs text-slate-400 whitespace-nowrap">Scale:</span> <button id="setScaleBtn" class="btn-primary
          px-3 py-1.5 rounded-lg text-xs font-medium disabled:opacity-50 whitespace-nowrap" disabled> Set Scale </button>
467        <div id="scaleInfo" class="hidden flex items-center gap-2"><span id="scaleValue" class="mono text-green-400 text-xs
          whitespace-nowrap">-</span> <button id="resetScaleBtn" class="btn-secondary px-2 py-1 rounded-lg text-xs" title="Reset
          Scale">Reset</button>
468        </div>
469       </div>
470       <div class="flex items-center gap-3 bg-slate-900/30 rounded-lg px-3 py-2 border border-slate-700/50 ml-auto hidden"
          id="panControls"><span class="text-xs text-slate-400">Pan:</span>
471        <div class="flex items-center gap-1"><button id="panLeft" class="btn-secondary px-2 py-1 rounded text-xs" title="Pan
          Left">←</button>
472          <div class="flex flex-col gap-1"><button id="panUp" class="btn-secondary px-2 py-0.5 rounded text-xs leading-none"
          title="Pan Up">↑</button> <button id="panDown" class="btn-secondary px-2 py-0.5 rounded text-xs leading-none" title="Pa
          Down">↓</button>
473          </div><button id="panRight" class="btn-secondary px-2 py-1 rounded text-xs" title="Pan Right">→</button>
474        </div><button id="resetPan" class="btn-secondary px-2 py-1 rounded text-xs" title="Reset Pan">■</button>
475       </div>
476      </div><!-- Calibration Inputs Panel -->
477      <div id="calibrationInputs" class="hidden tool-card p-3">
478       <div class="grid grid-cols-2 gap-3">
479        <div><label class="text-xs text-slate-400 block mb-1">Known Length</label> <input type="number" id="knownLength"
          step="any" min="0" class="w-full bg-slate-900 border border-slate-600 rounded-lg px-3 py-2 text-sm" placeholder="e.g.,
          100">
480        </div>
481        <div><label class="text-xs text-slate-400 block mb-1">Unit</label> <select id="unitSelect" class="w-full bg-slate-900
          border border-slate-600 rounded-lg px-3 py-2 text-sm"> <option value="nm">nm</option> <option value="µm"
          selected>µm</option> <option value="mm">mm</option> </select>
482        </div>
483        <div class="col-span-2 flex gap-2"><button id="saveScale" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1">Save
          Scale</button> <button id="cancelScale" class="btn-secondary px-4 py-2 rounded-lg text-sm">Cancel</button>
484        </div>
485       </div>
486      </div><!-- Canvas Container -->
487      <div class="flex-1 tool-card overflow-hidden min-h-[60vh] lg:min-h-0">
488       <div id="canvasWrapper" class="canvas-container w-full min-h-[60vh] lg:h-full flex items-center justify-center">
489        <div id="noImagePlaceholder" class="text-center p-8 pointer-events-none">
490         <svg class="w-16 h-16 mx-auto text-slate-600 mb-4" fill="none" stroke="currentColor" viewbox="0 0 24 24"><path
          stroke-linecap="round" stroke-linejoin="round" stroke-width="1.5" d="M4 16l4.586-4.586a2 2 0 012.828 0L16
          16m-2-2l1.586-1.586a2 2 0 012.828 0L20 14m-6-6h.01M6 20h12a2 2 0 002-2V6a2 2 0 00-2-2H6a2 2 0 00-2 2v12a2 2 0 002 2z" /
491         </svg>
492         <p class="text-slate-500 text-sm">Load an image to begin analysis</p>
493         <p class="text-slate-600 text-xs mt-1">Supports TIFF (first page), PNG, JPG, WebP, and other browser-supported image
          formats</p>
494         <p class="text-slate-500 text-xs mt-3">↓ Drag &amp;amp; drop an image here</p>
495        </div>
496        <canvas id="mainCanvas" class="hidden"></canvas>
497        <canvas id="overlayCanvas" class="hidden"></canvas>
498       </div>
499      </div>
500     </div><!-- Right Panel - Tools -->
501     <div class="lg:w-1/3 flex flex-col gap-3 min-h-0"><!-- Tool Tabs -->
502      <div class="tool-card p-2 flex gap-1 flex-wrap"><button class="tab-btn active px-3 py-2 rounded-lg text-xs font-medium
          flex-1 min-w-[60px]" data-tab="length">Length</button> <button class="tab-btn px-3 py-2 rounded-lg text-xs font-medium
          flex-1 min-w-[60px]" data-tab="angle">Angle</button> <button class="tab-btn px-3 py-2 rounded-lg text-xs font-medium
          flex-1 min-w-[60px]" data-tab="area">Area</button> <button class="tab-btn px-3 py-2 rounded-lg text-xs font-medium flex
          min-w-[60px]" data-tab="particles">Particles</button> <button class="tab-btn px-3 py-2 rounded-lg text-xs font-medium
          flex-1 min-w-[60px]" data-tab="holes">Holes</button>
503      </div><!-- Tool Content -->
504      <div class="tool-card flex-1 p-4 overflow-y-auto scrollbar-thin min-h-[200px]"><!-- Length Tab -->
505       <div id="lengthTab" class="tab-content">
506        <p class="text-xs text-slate-400 mb-3">Click two points on the image to measure distance.</p>
507        <div id="editLengthModeControls" class="hidden bg-yellow-500/20 border border-yellow-500/50 rounded-lg p-3 mb-3">
508         <p class="text-xs text-yellow-300 mb-2 font-medium">═■ Editing Length <span id="editingLengthLabel"></span></p>
509         <p class="text-xs text-slate-400 mb-3">Drag the endpoints to adjust the line.</p>
510         <div class="flex gap-2"><button id="saveLengthEdit" class="btn-primary px-3 py-1.5 rounded-lg text-xs flex-1">Save
          Changes</button> <button id="cancelLengthEdit" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1">Cancel</butt
511         </div>
512        </div>
513        <div id="lengthMeasurements" class="scrollbar-thin mb-3"></div>
514        <div class="flex gap-2">
515         <button id="lengthStatsBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Descriptive
          Statistics</button>
516          <button id="lengthHistBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Histogram</button>
517          <button id="clearLengths" class="btn-danger px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Clear All Lengths</button>
518        </div>
519        <div id="lengthExportRow" class="flex gap-2 mt-2 hidden">
520         <button id="copyLengthTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Copy TSV</button>
521         <button id="downloadLengthTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Download
          CSV</button>
```

```
          </div>
        </div><!-- Angle Tab -->
        <div id="angleTab" class="tab-content hidden">
         <p class="text-xs text-slate-400 mb-3">Click three points on the image to measure an angle. The second point is the
        vertex (middle point).</p>
         <div id="editAngleModeControls" class="hidden bg-yellow-500/20 border border-yellow-500/50 rounded-lg p-3 mb-3">
          <p class="text-xs text-yellow-300 mb-2 font-medium">═■ Editing Angle <span id="editingAngleLabel"></span></p>
          <p class="text-xs text-slate-400 mb-3">Drag any of the three points to adjust the angle.</p>
          <div class="flex gap-2"><button id="saveAngleEdit" class="btn-primary px-3 py-1.5 rounded-lg text-xs flex-1">Save
        Changes</button> <button id="cancelAngleEdit" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1">Cancel</butto
          </div>
         </div>
         <div id="angleMeasurements" class="scrollbar-thin mb-3"></div>
         <div class="flex gap-2">
          <button id="angleStatsBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Descriptive
        Statistics</button>
          <button id="angleHistBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Histogram</button>
          <button id="clearAngles" class="btn-danger px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Clear All Angles</button>
         </div>
         <div id="angleExportRow" class="flex gap-2 mt-2 hidden">
          <button id="copyAngleTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Copy TSV</button>
          <button id="downloadAngleTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Download
        CSV</button>
         </div>
        </div><!-- Area Tab -->
        <div id="areaTab" class="tab-content hidden">
          <div class="flex gap-2 mb-3">
           <button id="areaModePolygonBtn" class="btn-primary px-3 py-1.5 rounded-lg text-xs flex-1"
        type="button">Polygon</button>
           <button id="areaModeCircleBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1"
        type="button">Circle</button>
          </div>
          <p id="areaInstructions" class="text-xs text-slate-400 mb-3">Polygon: click points to create polygon.</p>
          <button id="closePolygonBtn" class="hidden btn-primary px-4 py-2 rounded-lg text-sm w-full mb-3">Close Polygon (3+
        points)</button>
         <div id="editModeControls" class="hidden bg-yellow-500/20 border border-yellow-500/50 rounded-lg p-3 mb-3">
          <p class="text-xs text-yellow-300 mb-2 font-medium">═■ Editing Area <span id="editingAreaLabel"></span></p>
          <p class="text-xs text-slate-400 mb-3">Drag points to adjust shape. Click edge to add point. Right-click point to
        delete.</p>
          <div class="flex gap-2"><button id="saveAreaEdit" class="btn-primary px-3 py-1.5 rounded-lg text-xs flex-1">Save
        Changes</button> <button id="cancelAreaEdit" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1">Cancel</button
          </div>
         </div>
        <div id="areaMeasurements" class="scrollbar-thin mb-3"></div>
        <div class="flex gap-2">
         <button id="areaStatsBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Descriptive
        Statistics</button>
          <button id="areaHistBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Histogram</button>
         <button id="clearAreas" class="btn-danger px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Clear All Areas</button>
        </div>
        <div id="areaExportRow" class="flex gap-2 mt-2 hidden">
         <button id="copyAreaTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Copy TSV</button>
        <button id="downloadAreaTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Download CSV</butt
        </div>
        </div><!-- Particles Tab -->
        <div id="particlesTab" class="tab-content hidden">
         <div class="space-y-3"><!-- ROI Selection -->
          <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
           <h4 class="text-xs font-semibold mb-2 text-blue-400">Region of Interest</h4>
           <div id="particleROIControls"><button id="selectParticleROI" class="btn-secondary px-3 py-1.5 rounded-lg text-xs w-f
        mb-2 disabled:opacity-50" disabled> ■ Select ROI (Draw Rectangle) </button>
            <div id="particleROIInfo" class="hidden">
             <div class="text-xs text-slate-400 mb-2"><span class="font-medium text-green-400">ROI Active:</span> <span
        id="particleROIDimensions" class="mono ml-1"></span>
             </div><button id="clearParticleROI" class="btn-danger px-2 py-1 rounded-lg text-xs w-full">Clear ROI (Use Full
        Image)</button>
            </div>
           </div>
          </div><!-- Preprocessing Section -->
          <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
           <h4 class="text-xs font-semibold mb-2 text-blue-400">Preprocessing</h4><label class="flex items-center gap-2 text-xs
        cursor-pointer mb-2"> <input type="checkbox" id="backgroundSubtraction" class="accent-blue-500"> Background subtraction
        </label>
           <div id="backgroundOptions" class="ml-5 mb-2 hidden"><label class="text-xs text-slate-400">Rolling ball radius
        (px)</label> <input type="number" id="rollingBallRadius" value="50" min="10" max="200" step="10" class="w-full
        bg-slate-900 border border-slate-600 rounded-lg px-2 py-1 text-xs mt-1">
           </div>
           <div class="mb-2"><label class="text-xs text-slate-400 block mb-1">Smoothing</label> <select id="smoothingMethod"
        class="w-full bg-slate-900 border border-slate-600 rounded-lg px-2 py-1.5 text-xs"> <option value="none">None</option>
        <option value="gaussian3">Gaussian 3×3</option> <option value="median3">Median 3×3</option> <option value="median5">Med
        5×5</option> </select>
```

```
583          </div><label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="checkbox" id="morphologicalOpening
         class="accent-blue-500"> Morphological opening (remove small objects) </label>
584        </div><!-- Thresholding Section -->
585        <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
586         <h4 class="text-xs font-semibold mb-2 text-blue-400">Thresholding</h4>
587          <div class="flex items-center gap-2 mb-2"><button id="autoThresholdBtn" class="btn-secondary px-3 py-1.5 rounded-lg
         text-xs flex-1">Auto (Otsu)</button> <label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="checkb
         id="adaptiveThreshold" class="accent-blue-500"> Adaptive </label>
588          </div>
589          <div id="manualThresholdControls"><label class="text-xs text-slate-400 flex justify-between"> <span>Manual
         Threshold</span> <span id="thresholdValue" class="mono">128</span> </label> <input type="range" id="particleThreshold"
         min="0" max="255" value="128" class="w-full mt-1">
590          </div>
591          <div id="adaptiveThresholdControls" class="hidden mt-2"><label class="text-xs text-slate-400">Block size (px)</label
         <input type="number" id="adaptiveBlockSize" value="32" min="8" max="128" step="8" class="w-full bg-slate-900 border
         border-slate-600 rounded-lg px-2 py-1 text-xs mt-1">
592          </div>
593        </div>
594        <div class="flex items-center gap-2"><label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="radi
         name="particlePolarity" value="bright" checked class="accent-blue-500"> Particles are bright </label> <label class="fle
         items-center gap-2 text-xs cursor-pointer"> <input type="radio" name="particlePolarity" value="dark"
         class="accent-blue-500"> Particles are dark </label>
595        </div>
596        <div><label class="text-xs text-slate-400" id="minParticleAreaLabel">Min Area (px²)</label> <input type="number"
         id="minParticleArea" value="10" min="0.001" step="any" class="w-full bg-slate-900 border border-slate-600 rounded-lg px
         py-2 text-sm mt-1" disabled>
597          <p class="text-xs text-slate-500 mt-1">Set scale first to enable</p>
598        </div><button id="analyzeParticles" class="btn-primary px-4 py-2 rounded-lg text-sm w-full disabled:opacity-50"
         disabled>Analyze Particles</button>
599        <div id="particleResults" class="hidden">
600         <div class="bg-slate-900/50 rounded-lg p-3 mt-3">
601          <h4 class="text-xs font-semibold mb-2">Summary Statistics</h4>
602          <div class="grid grid-cols-2 gap-2 text-xs">
603           <div>
604            <span class="text-slate-400">Count:</span> <span id="particleCount" class="mono">-</span>
605           </div>
606           <div>
607            <span class="text-slate-400">Mean Area:</span> <span id="particleMeanArea" class="mono">-</span>
608           </div>
609           <div>
610            <span class="text-slate-400">Mean ECD:</span> <span id="particleMeanECD" class="mono">-</span>
611           </div>
612           <div>
613            <span class="text-slate-400">Std Dev:</span> <span id="particleStdDev" class="mono">-</span>
614           </div>
615          </div>
616         </div>
617         <div class="mt-3">
618          <h4 class="text-xs font-semibold mb-2">ECD Histogram</h4>
619          <canvas id="particleHistogram" class="histogram-canvas w-full h-24"></canvas>
620         </div>
621        </div>
622        <div class="flex gap-2 flex-wrap">
623         <button id="particleStatsBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[120px]
         hidden">Descriptive Statistics</button>
624          <button id="particleHistBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[90px]
         hidden">Histogram</button>
625          <button id="clearParticles" class="btn-danger px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[110px] hidden">Clear
         Particles</button>
626        </div>
627        <div id="particlesExportRow" class="flex gap-2 flex-wrap hidden">
628         <button id="copyParticlesTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[90px]" disabled>C
         TSV</button>
629          <button id="downloadParticlesTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[110px]"
         disabled>Download CSV</button>
630        </div>
631       </div>
632      </div><!-- Holes Tab -->
633      <div id="holesTab" class="tab-content hidden">
634       <div class="space-y-3"><!-- ROI Selection -->
635        <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
636         <h4 class="text-xs font-semibold mb-2 text-blue-400">Region of Interest</h4>
637         <div id="holeROIControls"><button id="selectHoleROI" class="btn-secondary px-3 py-1.5 rounded-lg text-xs w-full mb-2
         disabled:opacity-50" disabled> ■ Select ROI (Draw Rectangle) </button>
638           <div id="holeROIInfo" class="hidden">
639            <div class="text-xs text-slate-400 mb-2"><span class="font-medium text-green-400">ROI Active:</span> <span
         id="holeROIDimensions" class="mono ml-1"></span>
640           </div><button id="clearHoleROI" class="btn-danger px-2 py-1 rounded-lg text-xs w-full">Clear ROI (Use Full
         Image)</button>
641          </div>
642         </div>
```

```
  </div><!-- Preprocessing Section -->
  <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
   <h4 class="text-xs font-semibold mb-2 text-blue-400">Preprocessing</h4><label class="flex items-center gap-2 text-xs cursor-pointer mb-2"> <input type="checkbox" id="holeBackgroundSubtraction" class="accent-blue-500"> Background subtraction </label>
   <div id="holeBackgroundOptions" class="ml-5 mb-2 hidden"><label class="text-xs text-slate-400">Rolling ball radius (px)</label> <input type="number" id="holeRollingBallRadius" value="50" min="10" max="200" step="10" class="w-full bg-slate-900 border border-slate-600 rounded-lg px-2 py-1 text-xs mt-1">
   </div>
   <div class="mb-2"><label class="text-xs text-slate-400 block mb-1">Smoothing</label> <select id="holeSmoothingMethod" class="w-full bg-slate-900 border border-slate-600 rounded-lg px-2 py-1.5 text-xs"> <option value="none">None</option> <option value="gaussian3">Gaussian 3×3</option> <option value="median3">Median 3×3</option> <option value="median5">Median 5×5</option> </select>
   </div><label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="checkbox" id="holeMorphologicalOpening" class="accent-blue-500"> Morphological opening (remove small objects) </label>
  </div><!-- Thresholding Section -->
  <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
   <h4 class="text-xs font-semibold mb-2 text-blue-400">Thresholding</h4>
   <div class="flex items-center gap-2 mb-2"><button id="autoThresholdHoleBtn" class="btn-secondary px-3 py-1.5 rounded text-xs flex-1">Auto (Otsu)</button> <label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="checkbox" id="adaptiveThresholdHole" class="accent-blue-500"> Adaptive </label>
   </div>
   <div id="manualThresholdHoleControls"><label class="text-xs text-slate-400 flex justify-between"> <span>Manual Threshold</span> <span id="holeThresholdValue" class="mono">128</span> </label> <input type="range" id="holeThreshold" min="0" max="255" value="128" class="w-full mt-1">
   </div>
   <div id="adaptiveThresholdHoleControls" class="hidden mt-2"><label class="text-xs text-slate-400">Block size (px)</label> <input type="number" id="holeAdaptiveBlockSize" value="32" min="8" max="128" step="8" class="w-full bg-slate-900 border border-slate-600 rounded-lg px-2 py-1 text-xs mt-1">
   </div>
  </div>
  <div class="flex items-center gap-2"><label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="radio" name="holePolarity" value="dark" checked class="accent-blue-500"> Holes are dark </label> <label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="radio" name="holePolarity" value="bright" class="accent-blue-500"> Holes are bright </label>
  </div>
  <div><label class="text-xs text-slate-400" id="minHoleAreaLabel">Min Area (px²)</label> <input type="number" id="minHoleArea" value="10" min="0.001" step="any" class="w-full bg-slate-900 border border-slate-600 rounded-lg px-3 py text-sm mt-1" disabled>
   <p class="text-xs text-slate-500 mt-1">Set scale first to enable</p>
  </div><button id="analyzeHoles" class="btn-primary px-4 py-2 rounded-lg text-sm w-full disabled:opacity-50" disabled>Analyze Holes</button>
  <div id="holeResults" class="hidden">
   <div class="bg-slate-900/50 rounded-lg p-3 mt-3">
    <h4 class="text-xs font-semibold mb-2">Summary Statistics</h4>
    <div class="grid grid-cols-2 gap-2 text-xs">
     <div>
      <span class="text-slate-400">Count:</span> <span id="holeCount" class="mono">-</span>
     </div>
     <div>
      <span class="text-slate-400">Mean Area:</span> <span id="holeMeanArea" class="mono">-</span>
     </div>
     <div>
      <span class="text-slate-400">Mean ECD:</span> <span id="holeMeanECD" class="mono">-</span>
     </div>
     <div>
      <span class="text-slate-400">Std Dev:</span> <span id="holeStdDev" class="mono">-</span>
     </div>
    </div>
   </div>
   <div class="mt-3">
    <h4 class="text-xs font-semibold mb-2">ECD Histogram</h4>
    <canvas id="holeHistogram" class="histogram-canvas w-full h-24"></canvas>
   </div>
  </div>
  <div class="flex gap-2 flex-wrap">
   <button id="holeStatsBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[120px] hidden">Descripti Statistics</button>
   <button id="holeHistBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[90px] hidden">Histogram</button>
   <button id="clearHoles" class="btn-danger px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[95px] hidden">Clear Holes</button>
  </div>
  <div id="holesExportRow" class="flex gap-2 flex-wrap hidden">
   <button id="copyHolesTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[90px]" disabled>Copy TSV</button>
   <button id="downloadHolesTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[110px]" disabled>Download CSV</button>
  </div>
 </div>
</div>
```

```
      </div>
     </div>
    </div>
     </div>
     <!-- Quick Start Overlay -->
     <div id="quickStartOverlay" class="quickstart-modal hidden" role="dialog" aria-modal="true" aria-label="Quick Start">
      <div class="quickstart-content">
       <h3 class="text-lg font-semibold text-white mb-3">Quick Start</h3>
       <ol class="space-y-2 text-sm text-slate-300">
        <li id="quickStartStep1" class="quickstart-step">
         <span class="quickstart-check" aria-hidden="true"></span>
         <span>1. Load Image</span>
        </li>
        <li id="quickStartStep2" class="quickstart-step">
         <span class="quickstart-check" aria-hidden="true"></span>
         <span>2. Set Scale</span>
        </li>
        <li id="quickStartStep3" class="quickstart-step">
         <span class="quickstart-check" aria-hidden="true"></span>
         <span>3. Start measurement</span>
        </li>
       </ol>
       <div class="mt-4">
        <button id="quickStartOk" class="btn-primary px-6 py-2 rounded-lg text-sm w-full">OK</button>
       </div>
      </div>
     </div><!-- Processing Modal -->
    <div id="processingModal" class="processing-modal hidden">
     <div class="processing-content">
      <div class="spinner"></div>
      <h3 id="processingTitle" class="text-lg font-semibold text-center mb-2">Processing Image</h3>
      <p id="processingStep" class="text-sm text-slate-400 text-center mb-1">Initializing...</p>
      <p id="processingTime" class="text-xs text-slate-500 text-center mono">Estimating time...</p>
      <div class="progress-bar-container">
       <div id="progressBar" class="progress-bar" style="width: 0%"></div>
      </div>
     </div>
    </div><!-- Documentation Modal -->
    <div id="docModal" class="overlay hidden">
      <div class="modal" role="dialog" aria-modal="true" aria-labelledby="docTitle">
        <div class="modalHead">
          <div id="docTitle" class="modalTitle">MicroMeasure Documentation</div>
          <button id="closeDocBtn" class="iconBtn" type="button" aria-label="Close documentation">X</button>
        </div>
        <div class="modalBody rvDoc"><!-- Project Information -->
       <section>
        <h3 class="text-xl font-bold text-blue-400 mb-3">Web-Based Image Analysis and Measurement Tool</h3>
        <div class="bg-slate-900/50 rounded-lg p-4 border border-slate-700 space-y-2">
            <p><strong>Version:</strong> 1.2.0</p>
         <p><strong>Website:</strong> <a href="https://www.inanolab.com/micromeasure.html" target="_blank" rel="noopener
        noreferrer" class="text-blue-400 hover:text-blue-300 underline">https://www.inanolab.com/micromeasure.html</a></p>
         <p><strong>Developer / Programmer:</strong> Dr. James Salveo Olarve</p>
         <p><strong>Affiliation:</strong> i-Nano Research Facility, De La Salle University Manila</p>
        </div>
          </section><!-- Citation Recommendation -->
          <section>
          <h3 class="text-lg font-semibold text-white mb-2">Citation Recommendation</h3>
          <p class="text-slate-300 mb-2">If you use MicroMeasure in academic work:</p>
          <div class="bg-slate-900/50 rounded-lg p-4 border border-slate-700">
             <p class="text-slate-300 text-sm font-mono leading-relaxed">Olarve, J. S. (2026). MicroMeasure: A Web-Based Image
        Measurement Tool (v1.2.0). Zenodo. https://doi.org/10.5281/zenodo.19418861</p>
          </div>
       </section><!-- Overview -->
       <section>
          <h3 class="text-lg font-semibold text-white mb-2">Overview</h3>
        <p class="text-slate-300 mb-2">MicroMeasure is a browser-based image analysis application designed for quantitative
        measurements of scientific images, particularly microscopy images such as scanning electron microscopy (SEM) micrograph
        The application provides intuitive tools for measuring length, angle, area, particle size, and hole size, without
        requiring software installation.</p>
        <p class="text-slate-300">MicroMeasure is inspired by widely used scientific image analysis platforms (e.g., ImageJ) bu
        optimized for fast, web-based workflows and educational use.</p>
       </section><!-- Intended Users -->
       <section>
          <h3 class="text-lg font-semibold text-white mb-2">Intended Users</h3>
        <p class="text-slate-300 mb-2">MicroMeasure is intended for:</p>
        <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
         <li>Researchers and laboratory personnel</li>
         <li>Graduate and undergraduate students</li>
         <li>Educators and instructors</li>
         <li>Materials science, physics, chemistry, biology, and engineering users</li>
        </ul>
```

```
      </section><!-- Quick Start -->
      <section>
        <h3 class="text-lg font-semibold text-white mb-2">Quick Start</h3>
        <ol class="list-decimal list-inside space-y-2 text-slate-300">
          <li><strong>Load an image</strong> - Click "Load Image" or drag &amp;amp; drop an image file</li>
          <li><strong>Set the scale</strong> - Click "Set Scale", draw a line of known length, then enter the real-world measurement</li>
          <li><strong>Make measurements</strong> - Use the tabs to measure lengths, angles, areas, particles, or holes</li>
          <li><strong>Export results</strong> - Use the <strong>Copy TSV</strong> / <strong>Download CSV</strong> buttons at the bottom of each measurement tab</li>
        </ol>
      </section><!-- Scale Calibration -->
      <section>
         <h3 class="text-lg font-semibold text-white mb-2">Scale Calibration</h3>
        <p class="text-slate-300 mb-2">Before making real-world measurements, you must calibrate the scale:</p>
        <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
          <li>Click the <strong>Set Scale</strong> button in the image controls</li>
          <li>Click two points on a feature of known length (e.g., a scale bar)</li>
          <li>Enter the known length and select the unit (nm, µm, mm)</li>
          <li>Click <strong>Save Scale</strong> to apply calibration</li>
          <li>All measurements will now show both pixel and real-world values</li>
          <li>Use <strong>Reset</strong> to clear calibration if needed</li>
        </ul>
      </section><!-- Length Measurements -->
      <section>
        <h3 class="text-lg font-semibold text-white mb-2">Length Measurements</h3>
        <p class="text-slate-300 mb-2">Measure distances between two points:</p>
        <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
          <li>Switch to the <strong>Length</strong> tab</li>
          <li>Click two points on the image to create a measurement</li>
          <li>Each measurement is labeled (L1, L2, etc.)</li>
          <li>Use the <strong>Edit (═■)</strong> button to adjust endpoints by dragging</li>
        <li>Use the <strong>Eye (■)</strong> button to show/hide measurements</li>
        <li>Click the <strong>×</strong> button to delete a measurement</li>
        </ul>
      </section><!-- Area Measurements -->
        <section>
          <h3 class="text-lg font-semibold text-white mb-2">Angle Measurements</h3>
        <p class="text-slate-300 mb-2">Measure the angle formed by two rays meeting at a vertex:</p>
        <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
          <li>Switch to the <strong>Angle</strong> tab</li>
          <li>Click <strong>three</strong> points on the image</li>
          <li>The <strong>second</strong> point is the <strong>vertex</strong> (middle point)</li>
          <li>Each angle is labeled (Ang1, Ang2, etc.) and reported in <strong>degrees (deg)</strong></li>
          <li>Use the <strong>Edit (═■)</strong> button to adjust by dragging any of the three points, then <strong>Save Changes</strong> or <strong>Cancel</strong></li>
          <li>Use the <strong>Eye (■)</strong> button to show/hide, and <strong>×</strong> to delete</li>
        </ul>
        </section><!-- Area Measurements -->
      <section>
        <h3 class="text-lg font-semibold text-white mb-2">Area Measurements</h3>
        <p class="text-slate-300 mb-2">Measure polygon areas by defining vertices:</p>
        <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
          <li>Switch to the <strong>Area</strong> tab</li>
          <li>Click points to create a polygon outline</li>
          <li>Click <strong>Close Polygon</strong> when you have 3+ points, or double-click to close</li>
          <li>Each area is labeled (A1, A2, etc.)</li>
          <li>Use <strong>Edit</strong> to adjust vertices by dragging points</li>
          <li>While editing, right-click a point to delete it, or click an edge to add a point</li>
          <li>Use <strong>Eye</strong> to show/hide, <strong>×</strong> to delete</li>
        </ul>
      </section><!-- Particle Analysis -->
      <section>
        <h3 class="text-lg font-semibold text-white mb-2">Particle Analysis</h3>
        <p class="text-slate-300 mb-2">Automatically detect and measure particles:</p>
        <h4 class="font-semibold text-blue-300 mt-3 mb-1">Region of Interest (ROI)</h4>
        <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
          <li>Click <strong>Select ROI</strong> to restrict analysis to a specific region</li>
          <li>Click and drag to draw a rectangular region</li>
          <li>Click <strong>Clear ROI</strong> to analyze the full image</li>
        </ul>
        <h4 class="font-semibold text-blue-300 mt-3 mb-1">Preprocessing Options</h4>
        <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
          <li><strong>Background subtraction</strong> - Removes uneven illumination (rolling ball method)</li>
          <li><strong>Smoothing</strong> - Reduces noise using Gaussian or Median filters</li>
          <li><strong>Morphological opening</strong> - Removes small noise objects</li>
        </ul>
        <h4 class="font-semibold text-blue-300 mt-3 mb-1">Thresholding</h4>
        <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
          <li><strong>Auto (Otsu)</strong> - Automatically calculates optimal threshold</li>
          <li><strong>Manual</strong> - Adjust threshold slider (0-255)</li>
```

```
        <li><strong>Adaptive</strong> - Local thresholding for uneven backgrounds</li>
        <li>Select whether particles are <strong>bright</strong> or <strong>dark</strong></li>
       </ul>
       <h4 class="font-semibold text-blue-300 mt-3 mb-1">Analysis</h4>
       <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
        <li>Set <strong>Min Area</strong> to filter out small noise (in real units if calibrated)</li>
        <li>Click <strong>Analyze Particles</strong></li>
        <li>View count, mean area, mean ECD (Equivalent Circular Diameter), and size distribution</li>
        <li>Each detected particle is outlined in orange on the image</li>
       </ul>
      </section><!-- Hole Analysis -->
      <section>
       <h3 class="text-lg font-semibold text-white mb-2">■■ Hole Analysis</h3>
       <p class="text-slate-300 mb-2">Automatically detect and measure holes/voids in materials:</p>
       <h4 class="font-semibold text-blue-300 mt-3 mb-1">Region of Interest (ROI)</h4>
       <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
        <li>Click <strong>Select ROI</strong> to restrict analysis to a specific region</li>
        <li>Click and drag to draw a rectangular region</li>
        <li>Click <strong>Clear ROI</strong> to analyze the full image</li>
       </ul>
       <h4 class="font-semibold text-blue-300 mt-3 mb-1">Preprocessing Options</h4>
       <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
        <li><strong>Background subtraction</strong> - Removes uneven illumination (rolling ball method)</li>
        <li><strong>Smoothing</strong> - Reduces noise using Gaussian or Median filters</li>
        <li><strong>Morphological opening</strong> - Removes small noise objects</li>
       </ul>
       <h4 class="font-semibold text-blue-300 mt-3 mb-1">Thresholding</h4>
       <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
        <li><strong>Auto (Otsu)</strong> - Automatically calculates optimal threshold</li>
        <li><strong>Manual</strong> - Adjust threshold slider (0-255)</li>
        <li><strong>Adaptive</strong> - Local thresholding for uneven backgrounds</li>
        <li>Select whether holes are <strong>dark</strong> or <strong>bright</strong></li>
       </ul>
       <h4 class="font-semibold text-blue-300 mt-3 mb-1">Analysis</h4>
       <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
        <li>Set <strong>Min Area</strong> to filter out small noise (in real units if calibrated)</li>
        <li>Click <strong>Analyze Holes</strong></li>
        <li>View count, mean area, mean ECD (Equivalent Circular Diameter), and size distribution</li>
        <li>Each detected hole is outlined in purple on the image</li>
       </ul>
      </section><!-- Navigation & Controls -->
       <section>
        <h3 class="text-lg font-semibold text-white mb-2">■ Descriptive Statistics &amp;amp; Histogram</h3>
        <p class="text-slate-300 mb-2">Summarize and visualize your measurements:</p>
        <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
         <li><strong>Descriptive Statistics</strong> shows mean, median, standard deviation, quartiles, etc.</li>
         <li><strong>Histogram</strong> shows the distribution of values (adjust bin count in the histogram window).</li>
         <li>Buttons appear when there is enough data (shown when <strong>n ≥ 4</strong> for the selected feature).</li>
         <li>For <strong>Length/Area/Particles/Holes</strong>, values are shown in pixels until scale is calibrated (then real
       units are shown).</li>
         <li><strong>Angle</strong> is always in <strong>degrees</strong> and does not require calibration.</li>
         <li>In the histogram window, <strong>Particles/Holes</strong> can be plotted by <strong>ECD</strong> or
       <strong>Area</strong> (metric selector).</li>
        </ul>
       </section><!-- Results & Export -->
     <section>
      <h3 class="text-lg font-semibold text-white mb-2">■ Navigation &amp;amp; Controls</h3>
       <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
        <li><strong>Zoom</strong> - Use the slider, mouse wheel, or pinch gesture</li>
        <li><strong>Pan</strong> - Middle-click drag, Alt+drag, two-finger drag, or use arrow buttons/keys</li>
        <li><strong>Fit to View</strong> - Automatically adjusts zoom to fit the image</li>
        <li><strong>Drag &amp;amp; Drop</strong> - Drag image files directly onto the canvas</li>
       </ul>
      </section><!-- Results & Export -->
      <section>
       <h3 class="text-lg font-semibold text-white mb-2">■ Results &amp;amp; Export</h3>
         <p class="text-slate-300 mb-2">Export data from within each measurement tab:</p>
       <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
        <li><strong>Save Image</strong> - Export image with all measurements overlaid (PNG)</li>
          <li><strong>Copy TSV</strong> - Copy the current tab’s data as tab-separated values (paste into spreadsheets)</li>
          <li><strong>Download CSV</strong> - Download the current tab’s data as a .csv file</li>
            <li><strong>Note:</strong> Export buttons appear only when the current tab has at least one measurement/object.</l
          <li><strong>Reset</strong> - Use the reset control to clear measurements and restart</li>
       </ul>
      </section><!-- Tips & Tricks -->
      <section>
       <h3 class="text-lg font-semibold text-white mb-2">■ Tips &amp;amp; Tricks</h3>
       <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
        <li>Always calibrate scale first for accurate real-world measurements</li>
        <li>Use ROI selection to speed up particle/hole analysis on large images</li>
        <li>Try Auto (Otsu) threshold first, then adjust manually if needed</li>
```

```
        <li>Adaptive thresholding works best for images with uneven lighting</li>
        <li>Background subtraction is useful for removing gradients in microscopy images</li>
        <li>Increase smoothing to reduce noise, but be careful not to blur features</li>
        <li>Morphological opening removes small speckles without affecting larger particles</li>
        <li>The Save Image button is only enabled when you have measurements on the image</li>
       </ul>
      </section><!-- Keyboard Shortcuts -->
      <section>
       <h3 class="text-lg font-semibold text-white mb-2">Keyboard Shortcuts</h3>
       <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
        <li><strong>Arrow Keys</strong> - Pan the image</li>
        <li><strong>Mouse Wheel</strong> - Zoom in/out</li>
        <li><strong>Alt + Left Click</strong> - Pan the image</li>
       </ul>
      </section><!-- Disclaimer -->
      <section>
       <h3 class="text-lg font-semibold text-white mb-2">Disclaimer</h3>
       <div class="bg-slate-900/50 rounded-lg p-4 border border-slate-700 space-y-2">
        <p class="text-slate-300 text-sm">MicroMeasure is provided for research and educational purposes only.</p>
        <p class="text-slate-300 text-sm">While every effort has been made to ensure accuracy, the developer makes no guarante
       regarding absolute measurement correctness for all imaging conditions. Users are solely responsible for verifying
       calibration accuracy and validating results for their specific applications.</p>
        <p class="text-slate-300 text-sm">MicroMeasure does not replace certified metrology software or instrument calibration
       standards.</p>
       </div>
      </section>
       </div>
     </div>
    </div>
    <!-- Descriptive Statistics Modal (publish-ready, light) -->
    <div id="descriptiveStatsModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Descriptive
        Statistics">
     <div class="max-w-3xl w-[92vw] max-h-[90vh] overflow-hidden rounded-2xl bg-white text-slate-900 border border-slate-200
        shadow-2xl">
      <div class="px-6 py-4 border-b border-slate-200 flex items-start justify-between gap-4">
       <div>
        <h3 id="statsModalTitle" class="text-lg font-semibold">Descriptive Statistics</h3>
        <p id="statsModalSubtitle" class="text-xs text-slate-500 mt-1"></p>
       </div>
       <button id="closeStatsModalBtn" class="text-slate-500 hover:text-slate-900 text-2xl leading-none"
        aria-label="Close">×</button>
      </div>
      <div class="px-6 py-4 overflow-y-auto" style="max-height: calc(90vh - 140px)">
      <div class="flex gap-2 flex-wrap mb-4">
       <button class="stats-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
        hover:bg-slate-100" data-stats-type="length">Length</button>
       <button class="stats-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
        hover:bg-slate-100" data-stats-type="angle">Angle</button>
       <button class="stats-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
        hover:bg-slate-100" data-stats-type="area">Area</button>
       <button class="stats-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
        hover:bg-slate-100" data-stats-type="particles">Particles</button>
       <button class="stats-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
        hover:bg-slate-100" data-stats-type="holes">Holes</button>
      </div>
       <div id="statsModalContent" class="space-y-4"></div>
      </div>
      <div class="px-6 py-4 border-t border-slate-200 flex gap-2">
      <button id="copyStatsTSVBtn" class="btn-secondary px-4 py-2 rounded-lg text-sm flex-1 text-white">Copy TSV</button>
      <button id="closeStatsModalBtnBottom" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1 text-white">Close</button>
      </div>
     </div>
    </div>
    <!-- Histogram Modal (publish-ready, light) -->
    <div id="histogramModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Histogram">
     <div class="max-w-4xl w-[94vw] max-h-[92vh] overflow-hidden rounded-2xl bg-white text-slate-900 border border-slate-200
        shadow-2xl">
      <div class="px-6 py-4 border-b border-slate-200 flex items-start justify-between gap-4">
       <div>
        <h3 id="histModalTitle" class="text-lg font-semibold">Histogram</h3>
        <p id="histModalSubtitle" class="text-xs text-slate-500 mt-1"></p>
       </div>
       <button id="closeHistModalBtn" class="text-slate-500 hover:text-slate-900 text-2xl leading-none"
        aria-label="Close">×</button>
      </div>
      <div class="px-6 py-4 overflow-y-auto" style="max-height: calc(92vh - 160px)">
      <div class="flex gap-2 flex-wrap mb-4">
       <button class="hist-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
        hover:bg-slate-100" data-hist-type="length">Length</button>
       <button class="hist-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
        hover:bg-slate-100" data-hist-type="angle">Angle</button>
```

```
      <button class="hist-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50 hover:bg-slate-100" data-hist-type="area">Area</button>
      <button class="hist-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50 hover:bg-slate-100" data-hist-type="particles">Particles</button>
      <button class="hist-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50 hover:bg-slate-100" data-hist-type="holes">Holes</button>
     </div>
     <div class="grid grid-cols-1 lg:grid-cols-3 gap-4 items-start mb-4">
      <div class="rounded-xl border border-slate-200 bg-slate-50 p-4">
       <div class="flex items-center justify-between gap-3">
        <label for="histBinsNumber" class="text-xs font-medium text-slate-700">Bins</label>
        <input id="histBinsNumber" type="number" min="3" max="80" step="1" value="15" class="w-24 bg-white border border-slate-300 rounded-lg px-2 py-1 text-xs mono text-slate-900" />
       </div>
       <div class="flex items-center justify-between gap-3 mt-3">
        <label for="histXTicksNumber" class="text-xs font-medium text-slate-700">X-axis divisions</label>
        <input id="histXTicksNumber" type="number" min="2" max="12" step="1" value="6" class="w-24 bg-white border border-slate-300 rounded-lg px-2 py-1 text-xs mono text-slate-900" />
       </div>
       <div class="flex items-center justify-between gap-3 mt-3">
        <label for="histYTicksNumber" class="text-xs font-medium text-slate-700">Y-axis divisions</label>
        <input id="histYTicksNumber" type="number" min="2" max="12" step="1" value="6" class="w-24 bg-white border border-slate-300 rounded-lg px-2 py-1 text-xs mono text-slate-900" />
       </div>
       <div class="mt-3">
        <label for="histMetricSelect" class="text-xs font-medium text-slate-700">Metric</label>
        <select id="histMetricSelect" class="w-full mt-1 bg-white border border-slate-300 rounded-lg px-2 py-1.5 text-xs text-slate-900">
         <option value="auto" selected>Auto</option>
         <option value="area">Area</option>
         <option value="ecd">ECD</option>
        </select>
        <p class="text-[11px] text-slate-500 mt-1">Metric applies to Particles/Holes (Length/Area ignore it).</p>
       </div>
      </div>
      <div class="lg:col-span-2 rounded-xl border border-slate-200 bg-white p-4">
       <div class="flex items-center justify-between flex-wrap gap-2 mb-2">
        <div class="text-xs text-slate-600">
         <span class="font-medium">n:</span> <span id="histN" class="mono">-</span>
         <span class="ml-3 font-medium">Range:</span> <span id="histRange" class="mono">-</span>
        </div>
        <button id="downloadHistSvgBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs text-white">Download SVG</button>
       </div>
       <div class="overflow-x-auto">
        <svg id="histSvg" class="w-full" viewBox="0 0 900 520" preserveAspectRatio="xMidYMid meet" role="img" aria-label="Histogram chart"></svg>
       </div>
       <div id="histEmptyNote" class="hidden mt-3 rounded-lg border border-slate-200 bg-slate-50 p-3">
        <p class="text-xs text-slate-700 font-medium">Not enough data</p>
        <p class="text-xs text-slate-600 mt-1">Add measurements/objects to generate a histogram.</p>
       </div>
      </div>
     </div>
    </div>
    <div class="px-6 py-4 border-t border-slate-200 flex gap-2">
     <button id="closeHistModalBtnBottom" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1 text-white">Close</button>
    </div>
   </div>
  </div>
  <!-- Reset Confirmation Modal -->
  <div id="resetConfirmModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Reset confirmation">
   <div class="processing-content max-w-lg">
    <h3 class="text-lg font-semibold text-center mb-2">Reset MicroMeasure?</h3>
    <p class="text-sm text-slate-300 text-center mb-4">
    This will clear the loaded image, scale calibration, ROI selection, and all measurements/results (Length, Angle, Area, Particles, Holes). This cannot be undone.
    </p>
    <div class="flex gap-2">
     <button id="confirmResetYes" class="btn-danger px-4 py-2 rounded-lg text-sm flex-1">Yes</button>
     <button id="confirmResetNo" class="btn-secondary px-4 py-2 rounded-lg text-sm flex-1">No</button>
    </div>
   </div>
  </div>
  <!-- Save Image Confirmation Modal -->
  <div id="saveImageConfirmModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Save image confirmation">
   <div class="processing-content max-w-lg">
    <h3 class="text-lg font-semibold text-center mb-2">Save Image?</h3>
    <p class="text-sm text-slate-300 text-center mb-4">
    This will export the current image as a PNG with all visible measurements and overlays included. Continue to download the image.
```

```
    </p>
    <div class="flex gap-2">
     <button id="confirmSaveImageYes" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1">Yes</button>
     <button id="confirmSaveImageNo" class="btn-secondary px-4 py-2 rounded-lg text-sm flex-1">No</button>
    </div>
   </div>
  </div>
  <!-- Home Redirect Confirmation Modal -->
  <div id="homeConfirmModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Home page confirmation">
   <div class="processing-content max-w-lg">
    <h3 class="text-lg font-semibold text-center mb-2">Go to Home Page?</h3>
    <p class="text-sm text-slate-300 text-center mb-4">
    This will redirect you to the i-Nano Research Facility home page at DLSU Manila. Continue to open https://www.inanolab.com/#extras.
    </p>
    <div class="flex gap-2">
     <button id="confirmHomeYes" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1">Yes</button>
     <button id="confirmHomeNo" class="btn-secondary px-4 py-2 rounded-lg text-sm flex-1">No</button>
    </div>
   </div>
  </div>
  <!-- Demo Image Confirmation Modal -->
  <div id="demoImageConfirmModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Demo image confirmation">
   <div class="processing-content max-w-lg">
    <h3 class="text-lg font-semibold text-center mb-2">Load Demo Photo?</h3>
    <p class="text-sm text-slate-300 text-center mb-4">
    This will load the bundled sample SEM image so you can try the app without uploading your own file. If you already have a loaded image or measurements, they will be replaced.
    </p>
    <div class="flex gap-2">
     <button id="confirmDemoImageYes" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1">Yes</button>
     <button id="confirmDemoImageNo" class="btn-secondary px-4 py-2 rounded-lg text-sm flex-1">No</button>
    </div>
   </div>
  </div>
  <script>
    // ========== STATE ==========
    const state = {
      image: null,
      imageData: null,
      zoom: 100,
      panX: 0,
      panY: 0,
      isPanning: false,
      lastPanX: 0,
      lastPanY: 0,

      // Guidance
      quickStartDismissed: true,

      // Calibration
      isCalibrated: false,
      unitsPerPixel: 0,
      unit: 'µm',
      calibrationLine: null,
      isSettingScale: false,
      scalePoints: [],

      // Current tool
      currentTool: 'length',

      // Length measurements
      lengthPoints: [],
      lengths: [],
      editingLengthIndex: null,
      draggedLengthPoint: null, // 'p1' or 'p2'

      // Angle measurements
      anglePoints: [],
      angles: [], // { p1, p2, p3, angleDeg, visible? }
      editingAngleIndex: null,
      draggedAnglePointIndex: null, // 0|1|2
      angleEditBackup: null,

      // Area measurements
      areaMode: 'polygon', // 'polygon' | 'circle'
      areaPoints: [],
      isDrawingCircle: false,
      circleDraft: null, // { center: {x,y}, radiusPx }
```

```
    areas: [],
    editingAreaIndex: null,
    draggedPointIndex: null,

    // Particles
    particles: [],

    // Holes
    holes: [],

    // ROI (Region of Interest)
    roi: null, // {x, y, width, height}
    isDrawingROI: false,
    roiStart: null,
    roiMode: null // 'particles' or 'holes'
  };

  // ========== DOM ELEMENTS ==========
  const elements = {
    imageInput: document.getElementById('imageInput'),
    mainCanvas: document.getElementById('mainCanvas'),
    overlayCanvas: document.getElementById('overlayCanvas'),
    canvasWrapper: document.getElementById('canvasWrapper'),
    noImagePlaceholder: document.getElementById('noImagePlaceholder'),
    zoomSlider: document.getElementById('zoomSlider'),
    zoomValue: document.getElementById('zoomValue'),
    imageDimensions: document.getElementById('imageDimensions'),
    imgWidth: document.getElementById('imgWidth'),
    imgHeight: document.getElementById('imgHeight'),
    fitToView: document.getElementById('fitToView'),
    panLeft: document.getElementById('panLeft'),
    panRight: document.getElementById('panRight'),
    panUp: document.getElementById('panUp'),
    panDown: document.getElementById('panDown'),
    resetPan: document.getElementById('resetPan'),
    setScaleBtn: document.getElementById('setScaleBtn'),
    calibrationInputs: document.getElementById('calibrationInputs'),
    knownLength: document.getElementById('knownLength'),
    unitSelect: document.getElementById('unitSelect'),
    saveScale: document.getElementById('saveScale'),
    cancelScale: document.getElementById('cancelScale'),
    scaleInfo: document.getElementById('scaleInfo'),
    scaleValue: document.getElementById('scaleValue'),
    calLineLength: document.getElementById('calLineLength'),
    resetScaleBtn: document.getElementById('resetScaleBtn'),
    quickStartOverlay: document.getElementById('quickStartOverlay'),
    quickStartOk: document.getElementById('quickStartOk'),
    quickStartStep1: document.getElementById('quickStartStep1'),
    quickStartStep2: document.getElementById('quickStartStep2'),
    quickStartStep3: document.getElementById('quickStartStep3'),
    appTitle: document.getElementById('appTitle'),
    loadDemoBtn: document.getElementById('loadDemoBtn'),
    headerResetBtn: document.getElementById('headerResetBtn'),

    // Tabs
    tabBtns: document.querySelectorAll('.tab-btn'),
    tabContents: document.querySelectorAll('.tab-content'),

    // Length
    lengthMeasurements: document.getElementById('lengthMeasurements'),
    clearLengths: document.getElementById('clearLengths'),
    lengthStatsBtn: document.getElementById('lengthStatsBtn'),
    lengthHistBtn: document.getElementById('lengthHistBtn'),
    lengthExportRow: document.getElementById('lengthExportRow'),
    copyLengthTSVBtn: document.getElementById('copyLengthTSVBtn'),
    downloadLengthTSVBtn: document.getElementById('downloadLengthTSVBtn'),
    editLengthModeControls: document.getElementById('editLengthModeControls'),
    editingLengthLabel: document.getElementById('editingLengthLabel'),
    saveLengthEdit: document.getElementById('saveLengthEdit'),
    cancelLengthEdit: document.getElementById('cancelLengthEdit'),

    // Angle
    angleMeasurements: document.getElementById('angleMeasurements'),
    clearAngles: document.getElementById('clearAngles'),
    angleStatsBtn: document.getElementById('angleStatsBtn'),
    angleHistBtn: document.getElementById('angleHistBtn'),
    angleExportRow: document.getElementById('angleExportRow'),
    copyAngleTSVBtn: document.getElementById('copyAngleTSVBtn'),
    downloadAngleTSVBtn: document.getElementById('downloadAngleTSVBtn'),
    editAngleModeControls: document.getElementById('editAngleModeControls'),
    editingAngleLabel: document.getElementById('editingAngleLabel'),
```

```
        saveAngleEdit: document.getElementById('saveAngleEdit'),
        cancelAngleEdit: document.getElementById('cancelAngleEdit'),

        // Area
        areaMeasurements: document.getElementById('areaMeasurements'),
        clearAreas: document.getElementById('clearAreas'),
        areaStatsBtn: document.getElementById('areaStatsBtn'),
        areaHistBtn: document.getElementById('areaHistBtn'),
        areaExportRow: document.getElementById('areaExportRow'),
        copyAreaTSVBtn: document.getElementById('copyAreaTSVBtn'),
        downloadAreaTSVBtn: document.getElementById('downloadAreaTSVBtn'),
        areaModePolygonBtn: document.getElementById('areaModePolygonBtn'),
        areaModeCircleBtn: document.getElementById('areaModeCircleBtn'),
        areaInstructions: document.getElementById('areaInstructions'),
        closePolygonBtn: document.getElementById('closePolygonBtn'),
        editModeControls: document.getElementById('editModeControls'),
        editingAreaLabel: document.getElementById('editingAreaLabel'),
        saveAreaEdit: document.getElementById('saveAreaEdit'),
        cancelAreaEdit: document.getElementById('cancelAreaEdit'),

        // Particles
        backgroundSubtraction: document.getElementById('backgroundSubtraction'),
        backgroundOptions: document.getElementById('backgroundOptions'),
        rollingBallRadius: document.getElementById('rollingBallRadius'),
        smoothingMethod: document.getElementById('smoothingMethod'),
        morphologicalOpening: document.getElementById('morphologicalOpening'),
        autoThresholdBtn: document.getElementById('autoThresholdBtn'),
        adaptiveThreshold: document.getElementById('adaptiveThreshold'),
        manualThresholdControls: document.getElementById('manualThresholdControls'),
        adaptiveThresholdControls: document.getElementById('adaptiveThresholdControls'),
        adaptiveBlockSize: document.getElementById('adaptiveBlockSize'),
        particleThreshold: document.getElementById('particleThreshold'),
        thresholdValue: document.getElementById('thresholdValue'),
        minParticleArea: document.getElementById('minParticleArea'),
        minParticleAreaLabel: document.getElementById('minParticleAreaLabel'),
        analyzeParticles: document.getElementById('analyzeParticles'),
        particleResults: document.getElementById('particleResults'),
        particleCount: document.getElementById('particleCount'),
        particleMeanArea: document.getElementById('particleMeanArea'),
        particleMeanECD: document.getElementById('particleMeanECD'),
        particleStdDev: document.getElementById('particleStdDev'),
        particleHistogram: document.getElementById('particleHistogram'),
        particleStatsBtn: document.getElementById('particleStatsBtn'),
        particleHistBtn: document.getElementById('particleHistBtn'),
        clearParticles: document.getElementById('clearParticles'),
        particlesExportRow: document.getElementById('particlesExportRow'),
        copyParticlesTSVBtn: document.getElementById('copyParticlesTSVBtn'),
        downloadParticlesTSVBtn: document.getElementById('downloadParticlesTSVBtn'),

        // ROI controls
        selectParticleROI: document.getElementById('selectParticleROI'),
        clearParticleROI: document.getElementById('clearParticleROI'),
        particleROIInfo: document.getElementById('particleROIInfo'),
        particleROIDimensions: document.getElementById('particleROIDimensions'),
        selectHoleROI: document.getElementById('selectHoleROI'),
        clearHoleROI: document.getElementById('clearHoleROI'),
        holeROIInfo: document.getElementById('holeROIInfo'),
        holeROIDimensions: document.getElementById('holeROIDimensions'),

        // Holes
        holeThreshold: document.getElementById('holeThreshold'),
        holeThresholdValue: document.getElementById('holeThresholdValue'),
        minHoleArea: document.getElementById('minHoleArea'),
        minHoleAreaLabel: document.getElementById('minHoleAreaLabel'),
        smoothHoles: document.getElementById('smoothHoles'),
        analyzeHoles: document.getElementById('analyzeHoles'),
        holeResults: document.getElementById('holeResults'),
        holeCount: document.getElementById('holeCount'),
        holeMeanArea: document.getElementById('holeMeanArea'),
        holeMeanECD: document.getElementById('holeMeanECD'),
        holeStdDev: document.getElementById('holeStdDev'),
        holeHistogram: document.getElementById('holeHistogram'),
        holeStatsBtn: document.getElementById('holeStatsBtn'),
        holeHistBtn: document.getElementById('holeHistBtn'),
        clearHoles: document.getElementById('clearHoles'),
        holesExportRow: document.getElementById('holesExportRow'),
        copyHolesTSVBtn: document.getElementById('copyHolesTSVBtn'),
        downloadHolesTSVBtn: document.getElementById('downloadHolesTSVBtn'),

        // Save Image
        saveScreenshot: document.getElementById('saveScreenshot'),
```

```
    homeBtn: document.getElementById('homeBtn'),

    // Processing modal
    processingModal: document.getElementById('processingModal'),
    processingTitle: document.getElementById('processingTitle'),
    processingStep: document.getElementById('processingStep'),
    processingTime: document.getElementById('processingTime'),
    progressBar: document.getElementById('progressBar'),

    // Descriptive statistics modal
    descriptiveStatsModal: document.getElementById('descriptiveStatsModal'),
    statsModalTitle: document.getElementById('statsModalTitle'),
    statsModalSubtitle: document.getElementById('statsModalSubtitle'),
    statsModalContent: document.getElementById('statsModalContent'),
    statsTypeBtns: document.querySelectorAll('.stats-type-btn'),
    closeStatsModalBtn: document.getElementById('closeStatsModalBtn'),
    closeStatsModalBtnBottom: document.getElementById('closeStatsModalBtnBottom'),
    copyStatsTSVBtn: document.getElementById('copyStatsTSVBtn'),

    // Histogram modal
    histogramModal: document.getElementById('histogramModal'),
    histModalTitle: document.getElementById('histModalTitle'),
    histModalSubtitle: document.getElementById('histModalSubtitle'),
    histTypeBtns: document.querySelectorAll('.hist-type-btn'),
    closeHistModalBtn: document.getElementById('closeHistModalBtn'),
    closeHistModalBtnBottom: document.getElementById('closeHistModalBtnBottom'),
    histBinsNumber: document.getElementById('histBinsNumber'),
    histXTicksNumber: document.getElementById('histXTicksNumber'),
    histYTicksNumber: document.getElementById('histYTicksNumber'),
    histMetricSelect: document.getElementById('histMetricSelect'),
    histSvg: document.getElementById('histSvg'),
    histN: document.getElementById('histN'),
    histRange: document.getElementById('histRange'),
    histEmptyNote: document.getElementById('histEmptyNote'),
    downloadHistSvgBtn: document.getElementById('downloadHistSvgBtn'),
    resetConfirmModal: document.getElementById('resetConfirmModal'),
    confirmResetYes: document.getElementById('confirmResetYes'),
    confirmResetNo: document.getElementById('confirmResetNo'),
    saveImageConfirmModal: document.getElementById('saveImageConfirmModal'),
    confirmSaveImageYes: document.getElementById('confirmSaveImageYes'),
    confirmSaveImageNo: document.getElementById('confirmSaveImageNo'),
    homeConfirmModal: document.getElementById('homeConfirmModal'),
    confirmHomeYes: document.getElementById('confirmHomeYes'),
    confirmHomeNo: document.getElementById('confirmHomeNo'),
    demoImageConfirmModal: document.getElementById('demoImageConfirmModal'),
    confirmDemoImageYes: document.getElementById('confirmDemoImageYes'),
    confirmDemoImageNo: document.getElementById('confirmDemoImageNo')
  };

  const mainCtx = elements.mainCanvas.getContext('2d');
  const overlayCtx = elements.overlayCanvas.getContext('2d');

  // ========== CONFIG ==========
  const defaultConfig = {
    app_title: 'MicroMeasure'
  };

  let config = { ...defaultConfig };

  // ========== ELEMENT SDK ==========
  if (window.elementSdk) {
    window.elementSdk.init({
      defaultConfig,
      onConfigChange: async (newConfig) => {
        config = { ...defaultConfig, ...newConfig };
        elements.appTitle.textContent = config.app_title || defaultConfig.app_title;
      },
      mapToCapabilities: () => ({
        recolorables: [],
        borderables: [],
        fontEditable: undefined,
        fontSizeable: undefined
      }),
      mapToEditPanelValues: (cfg) => new Map([
        ['app_title', cfg.app_title || defaultConfig.app_title]
      ])
    });
  }

  // ========== IMAGE LOADING ==========
  const TIFF_MIME_TYPES = new Set(['image/tiff', 'image/tif', 'image/x-tiff', 'application/tiff']);
```

```
1387
1388      function isTiffFile(file) {
1389        if (!file) return false;
1390        const fileType = (file.type || '').toLowerCase();
1391        return TIFF_MIME_TYPES.has(fileType) || /\.(tif|tiff)$/i.test(file.name || '');
1392      }
1393
1394      function isSupportedImageFile(file) {
1395        if (!file) return false;
1396        const fileType = (file.type || '').toLowerCase();
1397        return fileType.startsWith('image/') || isTiffFile(file);
1398      }
1399
1400      function showImageLoadError(message) {
1401        hideProcessingModal();
1402        window.alert(message);
1403      }
1404
1405      function applyLoadedImage(img) {
1406        state.image = img;
1407        state.imageData = null;
1408
1409        elements.mainCanvas.width = img.width;
1410        elements.mainCanvas.height = img.height;
1411        elements.overlayCanvas.width = img.width;
1412        elements.overlayCanvas.height = img.height;
1413
1414        elements.imgWidth.textContent = img.width;
1415        elements.imgHeight.textContent = img.height;
1416        elements.imageDimensions.classList.remove('hidden');
1417        elements.fitToView.classList.remove('hidden');
1418        document.getElementById('panControls').classList.remove('hidden');
1419
1420        elements.noImagePlaceholder.classList.add('hidden');
1421        elements.mainCanvas.classList.remove('hidden');
1422        elements.overlayCanvas.classList.remove('hidden');
1423
1424        elements.setScaleBtn.disabled = false;
1425        elements.analyzeParticles.disabled = false;
1426        elements.analyzeHoles.disabled = false;
1427        elements.selectParticleROI.disabled = false;
1428        elements.selectHoleROI.disabled = false;
1429
1430        // If the user was mid-calibration and loads a new image, reset the
1431        // calibration drawing state so auto-start can run reliably.
1432        state.isSettingScale = false;
1433        state.scalePoints = [];
1434
1435        // Reset pan and defer initial fit-to-view for placement
1436        state.panX = 0;
1437        state.panY = 0;
1438        requestAnimationFrame(() => {
1439          fitToView();
1440          // Auto-start scale setting as soon as an image is loaded
1441          // so the user is guided into the calibration step immediately.
1442          setScale();
1443          drawMainCanvas();
1444          drawOverlay();
1445          updateGuidance();
1446        });
1447      }
1448
1449      function loadBrowserImage(file) {
1450        return new Promise((resolve, reject) => {
1451          const objectUrl = URL.createObjectURL(file);
1452          const img = new Image();
1453
1454          img.onload = () => {
1455            URL.revokeObjectURL(objectUrl);
1456            resolve(img);
1457          };
1458
1459          img.onerror = () => {
1460            URL.revokeObjectURL(objectUrl);
1461            reject(new Error('The selected image could not be decoded by this browser.'));
1462          };
1463
1464          img.src = objectUrl;
1465        });
1466      }
1467
```

```
    async function loadTiffImage(file) {
      showProcessingModal('Loading TIFF Image');
      updateProgress('Loading TIFF decoder...', 15);

      try {
        const decoderLoaded = await tiffDecoderReady;
        if (!decoderLoaded || !window.UTIF) {
          throw new Error('TIFF support is unavailable because the TIFF decoder could not be loaded.');
        }

        updateProgress('Decoding TIFF pixels...', 45);
        const buffer = await file.arrayBuffer();
        const ifds = window.UTIF.decode(buffer);
        if (!ifds.length) {
          throw new Error('This TIFF file does not contain a readable image.');
        }

        const ifd = ifds[0];
        window.UTIF.decodeImage(buffer, ifd);
        const rgba = window.UTIF.toRGBA8(ifd);
        const width = ifd.width;
        const height = ifd.height;

        if (!width || !height || !rgba || !rgba.length) {
          throw new Error('The TIFF image was decoded, but no pixel data was produced.');
        }

        updateProgress('Rendering TIFF image...', 80);
        const canvas = document.createElement('canvas');
        canvas.width = width;
        canvas.height = height;

        const ctx = canvas.getContext('2d');
        if (!ctx) {
          throw new Error('Canvas rendering is unavailable in this browser.');
        }

        const imageData = new ImageData(new Uint8ClampedArray(rgba), width, height);
        ctx.putImageData(imageData, 0, 0);

        updateProgress('Finalizing TIFF import...', 100, 0);
        return canvas;
      } finally {
        hideProcessingModal();
      }
    }

    async function loadImage(file) {
      if (!isSupportedImageFile(file)) {
        showImageLoadError('Unsupported file type. Please choose a TIFF, PNG, JPG, WebP, or another browser-supported image
      format.');
        return;
      }

      try {
        const imageSource = isTiffFile(file)
          ? await loadTiffImage(file)
          : await loadBrowserImage(file);

        applyLoadedImage(imageSource);
      } catch (error) {
        console.error('Image load failed:', error);
        const fallbackMessage = isTiffFile(file)
          ? 'The TIFF file could not be loaded. If it is multi-page, only the first page is supported.'
          : 'The selected image could not be loaded.';
        showImageLoadError(error && error.message ? error.message : fallbackMessage);
      }
    }

    function loadDemoImage() {
      const img = new Image();
      img.onload = () => {
        resetToStart();
        applyLoadedImage(img);
      };
      img.src = 'data/sample-sem.jpg';
    }

    function fitToView() {
      const wrapper = elements.canvasWrapper;
      const wrapperW = wrapper.clientWidth;
```

```
    const wrapperH = wrapper.clientHeight;

    // Guard against zero-size wrapper (mobile layout settling)
    if (!wrapperW || !wrapperH) {
      requestAnimationFrame(fitToView);
      return;
    }

    const scaleX = wrapperW / state.image.width;
    const scaleY = wrapperH / state.image.height;
    const scale = Math.min(scaleX, scaleY) * 0.95;

    state.zoom = Math.max(50, Math.min(500, Math.round(scale * 100)));
    elements.zoomSlider.value = state.zoom;
    elements.zoomValue.textContent = state.zoom + '%';

    state.panX = 0;
    state.panY = 0;

    updateCanvasTransform();
  }

  function updateCanvasTransform() {
    const scale = state.zoom / 100;
    // Center in wrapper using translate, then apply pan in screen pixels
    const wrapper = elements.canvasWrapper;
    const wrapperW = wrapper.clientWidth;
    const wrapperH = wrapper.clientHeight;
    const canvasW = state.image.width * scale;
    const canvasH = state.image.height * scale;

    const centerX = (wrapperW - canvasW) / 2;
    const centerY = (wrapperH - canvasH) / 2;

    // Ensure absolute position baseline is 0,0 and use a single transform
    elements.mainCanvas.style.left = '0px';
    elements.mainCanvas.style.top = '0px';
    elements.overlayCanvas.style.left = '0px';
    elements.overlayCanvas.style.top = '0px';

    const transform = `translate(${centerX + state.panX}px, ${centerY + state.panY}px) scale(${scale})`;
    elements.mainCanvas.style.transformOrigin = 'top left';
    elements.overlayCanvas.style.transformOrigin = 'top left';
    elements.mainCanvas.style.transform = transform;
    elements.overlayCanvas.style.transform = transform;
  }

  function drawMainCanvas() {
    mainCtx.clearRect(0, 0, elements.mainCanvas.width, elements.mainCanvas.height);
    mainCtx.drawImage(state.image, 0, 0);
  }

  // ========== OVERLAY DRAWING ==========
  function drawOverlay() {
    overlayCtx.clearRect(0, 0, elements.overlayCanvas.width, elements.overlayCanvas.height);

    // Draw calibration line
    if (state.calibrationLine) {
      drawLine(state.calibrationLine.p1, state.calibrationLine.p2, '#fbbf24', 2, true);
    }

    // Draw scale setting points
    if (state.isSettingScale && state.scalePoints.length > 0) {
      drawPoint(state.scalePoints[0], '#fbbf24');
      if (state.scalePoints.length === 2) {
        drawLine(state.scalePoints[0], state.scalePoints[1], '#fbbf24', 2);
      }
    }

    // Draw length measurements
    state.lengths.forEach((len, i) => {
      if (len.visible !== false) {
        const isEditing = state.editingLengthIndex === i;
        drawLine(len.p1, len.p2, isEditing ? '#fbbf24' : '#3b82f6', 2);

        // Draw edit handles for editing length
        if (isEditing) {
          overlayCtx.fillStyle = '#fbbf24';
          overlayCtx.beginPath();
          overlayCtx.arc(len.p1.x, len.p1.y, 8, 0, Math.PI * 2);
          overlayCtx.fill();
```

```
            overlayCtx.strokeStyle = '#fff';
            overlayCtx.lineWidth = 2;
            overlayCtx.stroke();

            overlayCtx.beginPath();
            overlayCtx.arc(len.p2.x, len.p2.y, 8, 0, Math.PI * 2);
            overlayCtx.fill();
            overlayCtx.stroke();
          } else {
            drawPoint(len.p1, '#3b82f6');
            drawPoint(len.p2, '#3b82f6');
          }

          // Create measurement text only for non-editing lengths
          if (!isEditing) {
            const pixelText = `${formatNumber(len.pixelLength)} px`;
            const realText = state.isCalibrated ? `${formatNumber(len.realLength)} ${state.unit}` : '';
            const labelText = realText ? `L${i+1}: ${realText}` : `L${i+1}: ${pixelText}`;

            drawMeasurementLabel(midpoint(len.p1, len.p2), labelText, '#3b82f6');
          }
        }
      });

      // Draw current length points
      if (state.currentTool === 'length' && state.lengthPoints.length > 0) {
        drawPoint(state.lengthPoints[0], '#3b82f6');
      }

      // Draw angle measurements
      state.angles.forEach((ang, i) => {
        if (ang.visible === false) return;

        const isEditing = state.editingAngleIndex === i;
        const color = isEditing ? '#fbbf24' : '#06b6d4';

        // Rays from vertex (p2)
        drawLine(ang.p2, ang.p1, color, 2);
        drawLine(ang.p2, ang.p3, color, 2);

        // Arc inside the angle (minor or reflex)
        const arcInfo = drawAngleArc(ang.p1, ang.p2, ang.p3, !!ang.reflex, color);

        if (isEditing) {
          [ang.p1, ang.p2, ang.p3].forEach(p => {
            overlayCtx.fillStyle = '#fbbf24';
            overlayCtx.beginPath();
            overlayCtx.arc(p.x, p.y, 8, 0, Math.PI * 2);
            overlayCtx.fill();
            overlayCtx.strokeStyle = '#fff';
            overlayCtx.lineWidth = 2;
            overlayCtx.stroke();
          });
        } else {
          drawPoint(ang.p1, color);
          drawPoint(ang.p2, color);
          drawPoint(ang.p3, color);

          // Put label near the arc mid-angle (looks better than always above the vertex)
          let labelPos = { x: ang.p2.x, y: ang.p2.y - 18 };
          if (arcInfo && Number.isFinite(arcInfo.sweep)) {
            const mid = arcInfo.start + arcInfo.sweep / 2;
            const rr = (arcInfo.r || 24) + 16;
            labelPos = {
              x: ang.p2.x + rr * Math.cos(mid),
              y: ang.p2.y + rr * Math.sin(mid)
            };
          }
          drawMeasurementLabel(labelPos, `Ang${i + 1}: ${formatNumber(ang.angleDeg)}°`, color);
        }
      });

      // Draw current angle points (in-progress)
      if (state.currentTool === 'angle' && state.anglePoints.length > 0) {
        const color = '#06b6d4';
        state.anglePoints.forEach(p => drawPoint(p, color));
        if (state.anglePoints.length >= 2) drawLine(state.anglePoints[1], state.anglePoints[0], color, 2, true);
        if (state.anglePoints.length === 3) drawLine(state.anglePoints[1], state.anglePoints[2], color, 2, true);
      }

      // Draw area measurements (polygons + circles)
```

```
      state.areas.forEach((area, i) => {
        if (area.visible === false) return;

        const isEditing = state.editingAreaIndex === i;

        if (area.type === 'circle') {
          drawCircle(area.center, area.radiusPx, isEditing ? '#fbbf24' : '#22c55e');

          if (!isEditing && area.center) {
            const pixelText = `${formatNumber(area.pixelArea)} px²`;
            const realText = state.isCalibrated ? `${formatNumber(area.realArea)} ${state.unit}²` : '';
            const labelText = realText ? `A${i+1}: ${realText}` : `A${i+1}: ${pixelText}`;
            drawMeasurementLabel(area.center, labelText, '#22c55e');
          }

          return;
        }

        // Polygon (default)
        drawPolygon(area.points, isEditing ? '#fbbf24' : '#22c55e');

        // Draw edit handles for editing area
        if (isEditing) {
          area.points.forEach((p) => {
            overlayCtx.fillStyle = '#fbbf24';
            overlayCtx.beginPath();
            overlayCtx.arc(p.x, p.y, 8, 0, Math.PI * 2);
            overlayCtx.fill();
            overlayCtx.strokeStyle = '#fff';
            overlayCtx.lineWidth = 2;
            overlayCtx.stroke();
          });
        } else {
          // Create measurement text only for non-editing areas
          const pixelText = `${formatNumber(area.pixelArea)} px²`;
          const realText = state.isCalibrated ? `${formatNumber(area.realArea)} ${state.unit}²` : '';
          const labelText = realText ? `A${i+1}: ${realText}` : `A${i+1}: ${pixelText}`;

          drawMeasurementLabel(centroid(area.points), labelText, '#22c55e');
        }
      });

      // Draw current polygon area points (in-progress)
      if (state.currentTool === 'area' && state.areaMode === 'polygon' && state.areaPoints.length > 0) {
        overlayCtx.strokeStyle = '#22c55e';
        overlayCtx.lineWidth = 2;
        overlayCtx.setLineDash([5, 5]);
        overlayCtx.beginPath();
        overlayCtx.moveTo(state.areaPoints[0].x, state.areaPoints[0].y);
        state.areaPoints.forEach(p => overlayCtx.lineTo(p.x, p.y));
        overlayCtx.stroke();
        overlayCtx.setLineDash([]);
        state.areaPoints.forEach(p => drawPoint(p, '#22c55e'));
      }

      // Draw in-progress circle (Area tool)
      if (state.currentTool === 'area' && state.areaMode === 'circle' && state.circleDraft) {
        drawCircle(state.circleDraft.center, state.circleDraft.radiusPx, '#22c55e', true);
        if (state.circleDraft.center) drawPoint(state.circleDraft.center, '#22c55e');
      }

      // Draw particles
      state.particles.forEach((p, i) => {
        drawBoundingBox(p.bbox, '#f97316');
      });

      // Draw holes
      state.holes.forEach((h, i) => {
        drawBoundingBox(h.bbox, '#a855f7');
      });

      // Draw ROI rectangle
      if (state.roi) {
        overlayCtx.strokeStyle = '#fbbf24';
        overlayCtx.lineWidth = 3;
        overlayCtx.setLineDash([10, 5]);
        overlayCtx.strokeRect(state.roi.x, state.roi.y, state.roi.width, state.roi.height);
        overlayCtx.setLineDash([]);

        // Draw ROI label
        const roiLabel = `ROI: ${Math.round(state.roi.width)}×${Math.round(state.roi.height)} px`;
```

```
        overlayCtx.font = 'bold 13px Inter, sans-serif';
        overlayCtx.fillStyle = '#fbbf24';
        overlayCtx.textAlign = 'left';
        overlayCtx.textBaseline = 'bottom';

        const padding = 4;
        const metrics = overlayCtx.measureText(roiLabel);
        const labelX = state.roi.x;
        const labelY = state.roi.y - 5;

        overlayCtx.fillStyle = 'rgba(0,0,0,0.85)';
        overlayCtx.fillRect(labelX, labelY - 16, metrics.width + padding * 2, 18);

        overlayCtx.fillStyle = '#fbbf24';
        overlayCtx.fillText(roiLabel, labelX + padding, labelY);
      }
    }

    function drawLine(p1, p2, color, width, dashed = false) {
      overlayCtx.strokeStyle = color;
      overlayCtx.lineWidth = width;
      overlayCtx.setLineDash(dashed ? [8, 4] : []);
      overlayCtx.beginPath();
      overlayCtx.moveTo(p1.x, p1.y);
      overlayCtx.lineTo(p2.x, p2.y);
      overlayCtx.stroke();
      overlayCtx.setLineDash([]);
    }

    function drawPoint(p, color) {
      overlayCtx.fillStyle = color;
      overlayCtx.beginPath();
      overlayCtx.arc(p.x, p.y, 5, 0, Math.PI * 2);
      overlayCtx.fill();
      overlayCtx.strokeStyle = '#fff';
      overlayCtx.lineWidth = 1;
      overlayCtx.stroke();
    }

    function drawCircle(center, radiusPx, color, dashed = false) {
      if (!center) return;
      if (!Number.isFinite(center.x) || !Number.isFinite(center.y)) return;
      if (!Number.isFinite(radiusPx) || radiusPx <= 0) return;

      overlayCtx.fillStyle = color + '40';
      overlayCtx.strokeStyle = color;
      overlayCtx.lineWidth = 2;
      overlayCtx.setLineDash(dashed ? [8, 4] : []);
      overlayCtx.beginPath();
      overlayCtx.arc(center.x, center.y, radiusPx, 0, Math.PI * 2);
      overlayCtx.closePath();
      overlayCtx.fill();
      overlayCtx.stroke();
      overlayCtx.setLineDash([]);
    }

    function drawPolygon(points, color) {
      if (points.length < 3) return;
      overlayCtx.fillStyle = color + '40';
      overlayCtx.strokeStyle = color;
      overlayCtx.lineWidth = 2;
      overlayCtx.beginPath();
      overlayCtx.moveTo(points[0].x, points[0].y);
      points.forEach(p => overlayCtx.lineTo(p.x, p.y));
      overlayCtx.closePath();
      overlayCtx.fill();
      overlayCtx.stroke();
    }

    function drawBoundingBox(bbox, color) {
      overlayCtx.strokeStyle = color;
      overlayCtx.lineWidth = 1;
      overlayCtx.strokeRect(bbox.x, bbox.y, bbox.w, bbox.h);
    }

    function drawLabel(p, text, color) {
      overlayCtx.font = '12px Inter, sans-serif';
      overlayCtx.fillStyle = color;
      overlayCtx.textAlign = 'center';
      overlayCtx.textBaseline = 'middle';

```

```
      const padding = 4;
      const metrics = overlayCtx.measureText(text);
      const w = metrics.width + padding * 2;
      const h = 16;

      overlayCtx.fillStyle = 'rgba(0,0,0,0.7)';
      overlayCtx.fillRect(p.x - w/2, p.y - h/2, w, h);

      overlayCtx.fillStyle = color;
      overlayCtx.fillText(text, p.x, p.y);
    }

    function formatNumber(num) {
      return num.toLocaleString('en-US', { maximumFractionDigits: 2 });
    }

    function drawMeasurementLabel(p, text, color) {
      overlayCtx.font = 'bold 13px Inter, sans-serif';
      overlayCtx.textAlign = 'center';
      overlayCtx.textBaseline = 'middle';

      const padding = 6;
      const metrics = overlayCtx.measureText(text);
      const w = metrics.width + padding * 2;
      const h = 20;

      // Draw background with border
      overlayCtx.fillStyle = 'rgba(0,0,0,0.85)';
      overlayCtx.fillRect(p.x - w/2, p.y - h/2, w, h);

      overlayCtx.strokeStyle = color;
      overlayCtx.lineWidth = 2;
      overlayCtx.strokeRect(p.x - w/2, p.y - h/2, w, h);

      // Draw text
      overlayCtx.fillStyle = '#ffffff';
      overlayCtx.fillText(text, p.x, p.y);
    }

    function drawAngleArc(p1, p2, p3, reflex, color) {
      const v1x = p1.x - p2.x;
      const v1y = p1.y - p2.y;
      const v2x = p3.x - p2.x;
      const v2y = p3.y - p2.y;
      const mag1 = Math.hypot(v1x, v1y);
      const mag2 = Math.hypot(v2x, v2y);
      if (mag1 === 0 || mag2 === 0) return null;

      const a1 = Math.atan2(v1y, v1x);
      const a2 = Math.atan2(v2y, v2x);
      const TWO_PI = Math.PI * 2;
      const diff = (a2 - a1 + TWO_PI) % TWO_PI;

      const minorIsA1toA2 = diff <= Math.PI;
      const useA1toA2 = reflex ? !minorIsA1toA2 : minorIsA1toA2;

      const start = useA1toA2 ? a1 : a2;
      const end = useA1toA2 ? a2 : a1;
      const sweep = (end - start + TWO_PI) % TWO_PI;

      // Radius scales with ray lengths but is clamped for consistency
      const r = Math.min(44, Math.max(16, Math.min(mag1, mag2) * 0.28));

      overlayCtx.save();

      // Subtle filled wedge to make the interior obvious
      overlayCtx.globalAlpha = 0.10;
      overlayCtx.fillStyle = color;
      overlayCtx.beginPath();
      overlayCtx.moveTo(p2.x, p2.y);
      overlayCtx.arc(p2.x, p2.y, r, start, end, false);
      overlayCtx.closePath();
      overlayCtx.fill();

      // Dashed arc stroke
      overlayCtx.globalAlpha = 1;
      overlayCtx.strokeStyle = color;
      overlayCtx.lineWidth = 2;
      overlayCtx.setLineDash([6, 4]);
      overlayCtx.beginPath();
      overlayCtx.arc(p2.x, p2.y, r, start, end, false);
```

```
      overlayCtx.stroke();
      overlayCtx.setLineDash([]);

      overlayCtx.restore();

      return { r, start, sweep };
    }

    function midpoint(p1, p2) {
      return { x: (p1.x + p2.x) / 2, y: (p1.y + p2.y) / 2 };
    }

    function centroid(points) {
      const n = points.length;
      const sum = points.reduce((acc, p) => ({ x: acc.x + p.x, y: acc.y + p.y }), { x: 0, y: 0 });
      return { x: sum.x / n, y: sum.y / n };
    }

    // ========== COORDINATE CONVERSION ==========
    function canvasCoords(e) {
      const rect = elements.overlayCanvas.getBoundingClientRect();
      const scale = state.zoom / 100;
      return {
        x: (e.clientX - rect.left) / scale,
        y: (e.clientY - rect.top) / scale
      };
    }

    // ========== GUIDANCE SYSTEM ==========
    function hasAnyMeasurement() {
      return state.lengths.length > 0 || state.angles.length > 0 || state.areas.length > 0 || state.particles.length > 0 ||
      state.holes.length > 0;
    }

    function updateGuidance() {
      if (!elements.quickStartOverlay) return;

      const step1 = !!state.image;
      const step2 = !!state.isCalibrated;
      const step3 = hasAnyMeasurement();
      const allDone = step1 && step2 && step3;

      if (elements.quickStartStep1) elements.quickStartStep1.classList.toggle('done', step1);
      if (elements.quickStartStep2) elements.quickStartStep2.classList.toggle('done', step2);
      if (elements.quickStartStep3) elements.quickStartStep3.classList.toggle('done', step3);

      const steps = [elements.quickStartStep1, elements.quickStartStep2, elements.quickStartStep3].filter(Boolean);
      steps.forEach(el => el.classList.remove('current'));

      const current = !step1
        ? elements.quickStartStep1
        : (!step2 ? elements.quickStartStep2 : (!step3 ? elements.quickStartStep3 : null));
      if (current) current.classList.add('current');

      const shouldShow = !state.quickStartDismissed && !allDone;
      elements.quickStartOverlay.classList.toggle('hidden', !shouldShow);
    }

    // Update the Save Image button state
    function updateSaveButtonState() {
      const hasMeasurements = state.lengths.length > 0 || state.angles.length > 0 || state.areas.length > 0 ||
      state.particles.length > 0 || state.holes.length > 0;
      const canSave = state.image && hasMeasurements;
      elements.saveScreenshot.disabled = !canSave;
    }

    // ========== CALIBRATION ==========
    function setScale() {
      if (!state.image || state.isSettingScale) return;
      state.isSettingScale = true;
      state.scalePoints = [];
      elements.calibrationInputs.classList.remove('hidden');
      elements.setScaleBtn.textContent = 'Drawing...';
      elements.setScaleBtn.disabled = true;
    }

    function saveScale() {
      const knownLength = parseFloat(elements.knownLength.value);
      if (!knownLength || knownLength <= 0 || state.scalePoints.length !== 2) return;

      const p1 = state.scalePoints[0];
```

```
        const p2 = state.scalePoints[1];
        const pixelLength = Math.sqrt(Math.pow(p2.x - p1.x, 2) + Math.pow(p2.y - p1.y, 2));

        state.unitsPerPixel = knownLength / pixelLength;
        state.unit = elements.unitSelect.value;
        state.isCalibrated = true;
        state.calibrationLine = { p1, p2 };
        state.isSettingScale = false;
        state.scalePoints = [];

        elements.calibrationInputs.classList.add('hidden');
        elements.scaleInfo.classList.remove('hidden');
        elements.scaleValue.textContent = state.unitsPerPixel.toFixed(4) + ' ' + state.unit + '/px';
        elements.setScaleBtn.textContent = 'Set Scale';
        elements.setScaleBtn.disabled = false;

        // Enable min area inputs and update labels to use real units
        elements.minParticleArea.disabled = false;
        elements.minHoleArea.disabled = false;
        elements.minParticleAreaLabel.textContent = `Min Area (${state.unit}²)`;
        elements.minHoleAreaLabel.textContent = `Min Area (${state.unit}²)`;

        // Convert current pixel values to real units
        const currentParticleAreaPx = parseFloat(elements.minParticleArea.value) || 10;
        const currentHoleAreaPx = parseFloat(elements.minHoleArea.value) || 10;
        const areaConversion = Math.pow(state.unitsPerPixel, 2);

        elements.minParticleArea.value = (currentParticleAreaPx * areaConversion).toFixed(4);
        elements.minHoleArea.value = (currentHoleAreaPx * areaConversion).toFixed(4);

        // Update placeholder text
        elements.minParticleArea.nextElementSibling.textContent = `Value in ${state.unit}²`;
        elements.minHoleArea.nextElementSibling.textContent = `Value in ${state.unit}²`;

        drawOverlay();
        updateMeasurementDisplays();
        updateGuidance();
      }

      function cancelScale() {
        state.isSettingScale = false;
        state.scalePoints = [];
        elements.calibrationInputs.classList.add('hidden');
        elements.setScaleBtn.textContent = 'Set Scale';
        elements.setScaleBtn.disabled = false;
        drawOverlay();
      }

      function resetScale() {
        state.isCalibrated = false;
        state.calibrationLine = null;
        state.unitsPerPixel = 0;

        elements.scaleInfo.classList.add('hidden');
        elements.setScaleBtn.disabled = false;
        elements.setScaleBtn.textContent = 'Set Scale';

        // Reset min area inputs back to pixels
        elements.minParticleArea.disabled = true;
        elements.minHoleArea.disabled = true;
        elements.minParticleAreaLabel.textContent = 'Min Area (px²)';
        elements.minHoleAreaLabel.textContent = 'Min Area (px²)';

        // Convert current real values back to pixels (or reset to default)
        const currentParticleAreaReal = parseFloat(elements.minParticleArea.value);
        const currentHoleAreaReal = parseFloat(elements.minHoleArea.value);

        // If we had a previous calibration, convert back; otherwise use default
        if (state.unitsPerPixel > 0) {
          const areaConversion = Math.pow(state.unitsPerPixel, 2);
          elements.minParticleArea.value = Math.round(currentParticleAreaReal / areaConversion);
          elements.minHoleArea.value = Math.round(currentHoleAreaReal / areaConversion);
        } else {
          elements.minParticleArea.value = 10;
          elements.minHoleArea.value = 10;
        }

        // Update placeholder text
        elements.minParticleArea.nextElementSibling.textContent = 'Set scale first to enable';
        elements.minHoleArea.nextElementSibling.textContent = 'Set scale first to enable';

```

```
        // Recalculate measurements to remove real values
        state.lengths.forEach(len => {
          len.realLength = null;
        });
        state.areas.forEach(area => {
          area.realArea = null;
        });
        state.particles.forEach(p => {
          p.areaReal = null;
          p.ecdReal = null;
        });
        state.holes.forEach(h => {
          h.areaReal = null;
          h.ecdReal = null;
        });

        updateLengthDisplay();
        updateAreaDisplay();
        if (state.particles.length) updateParticleResults();
        if (state.holes.length) updateHoleResults();
        drawOverlay();
        updateGuidance();
      }

      // ========== LENGTH MEASUREMENT ==========
      function measureLength(p1, p2) {
        const pixelLength = Math.sqrt(Math.pow(p2.x - p1.x, 2) + Math.pow(p2.y - p1.y, 2));
        const realLength = state.isCalibrated ? pixelLength * state.unitsPerPixel : null;
        return { p1, p2, pixelLength, realLength };
      }

      function addLengthMeasurement(len) {
        state.lengths.push(len);
        updateLengthDisplay();
        drawOverlay();
        updateGuidance();
        updateSaveButtonState();
      }

      function updateLengthDisplay() {
        elements.lengthMeasurements.innerHTML = state.lengths.map((len, i) => `
          <div class="measurement-item">
            <div>
              <span class="font-medium">L${i+1}:</span>
              <span class="mono">${formatNumber(len.pixelLength)} px</span>
              ${state.isCalibrated ? `<span class="mono text-blue-400 ml-2">${formatNumber(len.realLength)} ${state.unit}</span
          : ''}
            </div>
            <div class="flex items-center gap-2">
              <button onclick="toggleLengthVisibility(${i})" class="text-slate-400 hover:text-blue-400 text-sm"
          title="${len.visible !== false ? 'Hide' : 'Show'}">
                ${len.visible !== false ? '■' : '■'}
              </button>
              <button onclick="editLengthMeasurement(${i})" class="text-slate-400 hover:text-yellow-400 text-sm"
          title="Edit">═■</button>
              <button onclick="deleteLengthMeasurement(${i})" class="text-red-400 hover:text-red-300 text-lg
          leading-none">&times;</button>
            </div>
          </div>
        `).join('');
        elements.clearLengths.classList.toggle('hidden', state.lengths.length === 0);
        updateStatsButtonsVisibility();
      }

      window.deleteLengthMeasurement = function(index) {
        state.lengths.splice(index, 1);
        updateLengthDisplay();
        drawOverlay();
        updateSaveButtonState();
      };

      window.toggleLengthVisibility = function(index) {
        state.lengths[index].visible = state.lengths[index].visible === false ? true : false;
        updateLengthDisplay();
        drawOverlay();
      };

      window.editLengthMeasurement = function(index) {
        state.editingLengthIndex = index;
        state.currentTool = 'length';

```

```
        // Switch to length tab UI
        elements.tabBtns.forEach(b => b.classList.remove('active'));
        document.querySelector('[data-tab="length"]').classList.add('active');
        document.querySelectorAll('.tab-content').forEach(content => content.classList.add('hidden'));
        document.getElementById('lengthTab').classList.remove('hidden');

        // Show edit mode controls
        elements.editLengthModeControls.classList.remove('hidden');
        elements.editingLengthLabel.textContent = `L${index + 1}`;

        drawOverlay();
      };

      function saveLengthEdit() {
        if (state.editingLengthIndex !== null) {
          const len = state.lengths[state.editingLengthIndex];
          const pixelLength = Math.sqrt(Math.pow(len.p2.x - len.p1.x, 2) + Math.pow(len.p2.y - len.p1.y, 2));
          len.pixelLength = pixelLength;
          len.realLength = state.isCalibrated ? pixelLength * state.unitsPerPixel : null;

          exitLengthEditMode();
          updateLengthDisplay();
        }
      }

      function cancelLengthEdit() {
        exitLengthEditMode();
      }

      function exitLengthEditMode() {
        state.editingLengthIndex = null;
        state.draggedLengthPoint = null;
        elements.editLengthModeControls.classList.add('hidden');
        drawOverlay();
      }

      function clearAllLengths() {
        state.lengths = [];
        state.lengthPoints = [];
        updateLengthDisplay();
        drawOverlay();
        updateSaveButtonState();
      }

      // ========== ANGLE MEASUREMENT ==========
      function computeAngleMetrics(p1, p2, p3) {
        const v1x = p1.x - p2.x;
        const v1y = p1.y - p2.y;
        const v2x = p3.x - p2.x;
        const v2y = p3.y - p2.y;
        const mag1 = Math.hypot(v1x, v1y);
        const mag2 = Math.hypot(v2x, v2y);
        if (mag1 === 0 || mag2 === 0) {
          return { minorDeg: 0, majorDeg: 0 };
        }

        // Use atan2(cross, dot) so we can derive both minor + reflex angles reliably.
        const dot = v1x * v2x + v1y * v2y;
        const cross = v1x * v2y - v1y * v2x;
        let angleRad = Math.atan2(Math.abs(cross), dot); // [0, π]
        if (!Number.isFinite(angleRad)) angleRad = 0;

        const minorDeg = angleRad * (180 / Math.PI);
        // Reflex is 360 - minor; suppress degenerate 360 when minor is 0.
        const majorDeg = minorDeg < 1e-9 ? 0 : (360 - minorDeg);
        return { minorDeg, majorDeg };
      }

      function computeAngleDeg(p1, p2, p3) {
        // Backwards-compatible: returns minor angle (0..180)
        return computeAngleMetrics(p1, p2, p3).minorDeg;
      }

      function computeSelectedAngleDeg(p1, p2, p3, reflex) {
        const m = computeAngleMetrics(p1, p2, p3);
        return reflex ? m.majorDeg : m.minorDeg;
      }

      function addAngleMeasurement(p1, p2, p3) {
        const reflex = false;
        const angleDeg = computeSelectedAngleDeg(p1, p2, p3, reflex);
```

```
        state.angles.push({ p1, p2, p3, angleDeg, reflex });
        updateAngleDisplay();
        drawOverlay();
        updateGuidance();
        updateSaveButtonState();
      }

      function updateAngleDisplay() {
        if (!elements.angleMeasurements) return;

        elements.angleMeasurements.innerHTML = state.angles.map((ang, i) => `
          <div class="measurement-item">
            <div>
              <span class="font-medium">Ang${i + 1}:</span>
              <span class="mono">${formatNumber(ang.angleDeg)}°</span>
              ${ang.reflex ? `<span class="text-xs text-slate-400 ml-2">(reflex)</span>` : ``}
            </div>
            <div class="flex items-center gap-2">
              <button onclick="toggleAngleVisibility(${i})" class="text-slate-400 hover:text-cyan-400 text-sm"
        title="${ang.visible !== false ? 'Hide' : 'Show'}">
                ${ang.visible !== false ? '■' : '■'}
              </button>
              <button onclick="toggleAngleReflex(${i})" class="text-slate-400 hover:text-blue-300 text-sm" title="Toggle
        minor/reflex angle">■</button>
              <button onclick="editAngleMeasurement(${i})" class="text-slate-400 hover:text-yellow-400 text-sm"
        title="Edit">═■</button>
              <button onclick="deleteAngleMeasurement(${i})" class="text-red-400 hover:text-red-300 text-lg
        leading-none">&times;</button>
            </div>
          </div>
        `).join('');

        if (elements.clearAngles) elements.clearAngles.classList.toggle('hidden', state.angles.length === 0);
        updateStatsButtonsVisibility();
      }

      window.deleteAngleMeasurement = function(index) {
        state.angles.splice(index, 1);
        updateAngleDisplay();
        drawOverlay();
        updateSaveButtonState();
      };

      window.toggleAngleVisibility = function(index) {
        state.angles[index].visible = state.angles[index].visible === false ? true : false;
        updateAngleDisplay();
        drawOverlay();
      };

      window.toggleAngleReflex = function(index) {
        const ang = state.angles[index];
        if (!ang) return;
        ang.reflex = !ang.reflex;
        ang.angleDeg = computeSelectedAngleDeg(ang.p1, ang.p2, ang.p3, !!ang.reflex);
        updateAngleDisplay();
        drawOverlay();
      };

      window.editAngleMeasurement = function(index) {
        state.editingAngleIndex = index;
        state.draggedAnglePointIndex = null;
        state.currentTool = 'angle';

        const a = state.angles[index];
        state.angleEditBackup = {
          p1: { ...a.p1 },
          p2: { ...a.p2 },
          p3: { ...a.p3 },
          angleDeg: a.angleDeg,
          reflex: !!a.reflex
        };

        // Switch to angle tab UI
        elements.tabBtns.forEach(b => b.classList.remove('active'));
        const angleBtn = document.querySelector('[data-tab="angle"]');
        if (angleBtn) angleBtn.classList.add('active');
        document.querySelectorAll('.tab-content').forEach(content => content.classList.add('hidden'));
        const tab = document.getElementById('angleTab');
        if (tab) tab.classList.remove('hidden');

        if (elements.editAngleModeControls) elements.editAngleModeControls.classList.remove('hidden');
```

```
    if (elements.editingAngleLabel) elements.editingAngleLabel.textContent = `Ang${index + 1}`;

    drawOverlay();
  };

  function saveAngleEdit() {
    if (state.editingAngleIndex === null) return;
    const a = state.angles[state.editingAngleIndex];
    a.angleDeg = computeSelectedAngleDeg(a.p1, a.p2, a.p3, !!a.reflex);
    exitAngleEditMode();
    updateAngleDisplay();
  }

  function cancelAngleEdit() {
    if (state.editingAngleIndex === null) return;
    if (state.angleEditBackup) {
      const a = state.angles[state.editingAngleIndex];
      a.p1 = { ...state.angleEditBackup.p1 };
      a.p2 = { ...state.angleEditBackup.p2 };
      a.p3 = { ...state.angleEditBackup.p3 };
      a.angleDeg = state.angleEditBackup.angleDeg;
      a.reflex = !!state.angleEditBackup.reflex;
    }
    exitAngleEditMode();
    updateAngleDisplay();
  }

  function exitAngleEditMode() {
    state.editingAngleIndex = null;
    state.draggedAnglePointIndex = null;
    state.angleEditBackup = null;
    if (elements.editAngleModeControls) elements.editAngleModeControls.classList.add('hidden');
    drawOverlay();
  }

  function clearAllAngles() {
    state.angles = [];
    state.anglePoints = [];
    exitAngleEditMode();
    updateAngleDisplay();
    drawOverlay();
    updateSaveButtonState();
  }

  // ========== AREA MEASUREMENT ==========
  function measurePolygonArea(points) {
    if (points.length < 3) return 0;
    let area = 0;
    const n = points.length;
    for (let i = 0; i < n; i++) {
      const j = (i + 1) % n;
      area += points[i].x * points[j].y;
      area -= points[j].x * points[i].y;
    }
    return Math.abs(area / 2);
  }

  function measureCircleArea(radiusPx) {
    const r = Math.max(0, radiusPx || 0);
    return Math.PI * r * r;
  }

  function addAreaMeasurement(points) {
    const pixelArea = measurePolygonArea(points);
    const realArea = state.isCalibrated ? pixelArea * Math.pow(state.unitsPerPixel, 2) : null;
    state.areas.push({ points: [...points], pixelArea, realArea });
    updateAreaDisplay();
    drawOverlay();
    updateGuidance();
    updateSaveButtonState();
  }

  function addCircleAreaMeasurement(center, radiusPx) {
    const pixelArea = measureCircleArea(radiusPx);
    const realArea = state.isCalibrated ? pixelArea * Math.pow(state.unitsPerPixel, 2) : null;
    state.areas.push({ type: 'circle', center: { ...center }, radiusPx, pixelArea, realArea });
    updateAreaDisplay();
    drawOverlay();
    updateGuidance();
    updateSaveButtonState();
  }
```

```
2429
2430      function updateAreaDisplay() {
2431        elements.areaMeasurements.innerHTML = state.areas.map((area, i) => `
2432          <div class="measurement-item">
2433            <div>
2434              <span class="font-medium">A${i+1}:</span>
2435              <span class="mono">${formatNumber(area.pixelArea)} px²</span>
2436              ${state.isCalibrated ? `<span class="mono text-green-400 ml-2">${formatNumber(area.realArea)} ${state.unit}²</spa
          : ''}
2437            </div>
2438            <div class="flex items-center gap-2">
2439              <button onclick="toggleAreaVisibility(${i})" class="text-slate-400 hover:text-green-400 text-sm"
          title="${area.visible !== false ? 'Hide' : 'Show'}">
2440                ${area.visible !== false ? '■' : '■'}
2441              </button>
2442              ${area.type === 'circle' ? '' : `<button onclick="editAreaMeasurement(${i})" class="text-slate-400
          hover:text-yellow-400 text-sm" title="Edit">═■</button>`}
2443              <button onclick="deleteAreaMeasurement(${i})" class="text-red-400 hover:text-red-300 text-lg
          leading-none">&times;</button>
2444            </div>
2445          </div>
2446        `).join('');
2447        elements.clearAreas.classList.toggle('hidden', state.areas.length === 0);
2448        updateStatsButtonsVisibility();
2449      }
2450
2451      window.deleteAreaMeasurement = function(index) {
2452        state.areas.splice(index, 1);
2453        updateAreaDisplay();
2454        drawOverlay();
2455        updateSaveButtonState();
2456      };
2457
2458      window.toggleAreaVisibility = function(index) {
2459        state.areas[index].visible = state.areas[index].visible === false ? true : false;
2460        updateAreaDisplay();
2461        drawOverlay();
2462      };
2463
2464      window.editAreaMeasurement = function(index) {
2465        const a = state.areas[index];
2466        if (!a || a.type === 'circle') return;
2467
2468        state.editingAreaIndex = index;
2469        state.currentTool = 'area';
2470
2471        // Switch to area tab UI
2472        elements.tabBtns.forEach(b => b.classList.remove('active'));
2473        document.querySelector('[data-tab="area"]').classList.add('active');
2474        document.querySelectorAll('.tab-content').forEach(content => content.classList.add('hidden'));
2475        document.getElementById('areaTab').classList.remove('hidden');
2476
2477        // Show edit mode controls
2478        elements.areaInstructions.classList.add('hidden');
2479        elements.editModeControls.classList.remove('hidden');
2480        elements.editingAreaLabel.textContent = `A${index + 1}`;
2481
2482        drawOverlay();
2483      };
2484
2485      function saveAreaEdit() {
2486        if (state.editingAreaIndex !== null) {
2487          const area = state.areas[state.editingAreaIndex];
2488          if (!area || !Array.isArray(area.points)) {
2489            exitAreaEditMode();
2490            updateAreaDisplay();
2491            return;
2492          }
2493          area.pixelArea = measurePolygonArea(area.points);
2494          area.realArea = state.isCalibrated ? area.pixelArea * Math.pow(state.unitsPerPixel, 2) : null;
2495
2496          exitAreaEditMode();
2497          updateAreaDisplay();
2498        }
2499      }
2500
2501      function cancelAreaEdit() {
2502        exitAreaEditMode();
2503      }
2504
2505      function exitAreaEditMode() {
```

```
      state.editingAreaIndex = null;
      state.draggedPointIndex = null;
      elements.areaInstructions.classList.remove('hidden');
      elements.editModeControls.classList.add('hidden');
      drawOverlay();
    }

    function clearAllAreas() {
      exitAreaEditMode();
      state.areas = [];
      state.areaPoints = [];
      state.isDrawingCircle = false;
      state.circleDraft = null;
      elements.closePolygonBtn.classList.add('hidden');
      updateAreaDisplay();
      drawOverlay();
      updateSaveButtonState();
    }

    function setAreaMode(mode) {
      if (state.editingAreaIndex !== null) return;
      if (mode !== 'polygon' && mode !== 'circle') return;

      state.areaMode = mode;
      state.areaPoints = [];
      state.isDrawingCircle = false;
      state.circleDraft = null;

      elements.closePolygonBtn.classList.add('hidden');

      if (mode === 'polygon') {
        if (elements.areaInstructions) elements.areaInstructions.textContent = 'Polygon: click points to create polygon.';
        if (elements.areaModePolygonBtn) {
          elements.areaModePolygonBtn.classList.add('btn-primary');
          elements.areaModePolygonBtn.classList.remove('btn-secondary');
        }
        if (elements.areaModeCircleBtn) {
          elements.areaModeCircleBtn.classList.add('btn-secondary');
          elements.areaModeCircleBtn.classList.remove('btn-primary');
        }
      } else {
        if (elements.areaInstructions) elements.areaInstructions.textContent = 'Circle: click and drag to draw a circle.';
        if (elements.areaModePolygonBtn) {
          elements.areaModePolygonBtn.classList.add('btn-secondary');
          elements.areaModePolygonBtn.classList.remove('btn-primary');
        }
        if (elements.areaModeCircleBtn) {
          elements.areaModeCircleBtn.classList.add('btn-primary');
          elements.areaModeCircleBtn.classList.remove('btn-secondary');
        }
      }

      drawOverlay();
    }

    // ========== PARTICLE/HOLE ANALYSIS ==========
    function getImageData() {
      if (!state.imageData) {
        const canvas = document.createElement('canvas');
        canvas.width = state.image.width;
        canvas.height = state.image.height;
        const ctx = canvas.getContext('2d');
        ctx.drawImage(state.image, 0, 0);
        state.imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);
      }
      return state.imageData;
    }

    function toGrayscale(imageData) {
      const data = imageData.data;
      const gray = new Uint8Array(imageData.width * imageData.height);
      for (let i = 0; i < gray.length; i++) {
        const idx = i * 4;
        gray[i] = Math.round(0.299 * data[idx] + 0.587 * data[idx + 1] + 0.114 * data[idx + 2]);
      }
      return gray;
    }

    function blur3x3(gray, width, height) {
      const result = new Uint8Array(gray.length);
      const kernel = [1, 2, 1, 2, 4, 2, 1, 2, 1];
```

```
    const kernelSum = 16;

    for (let y = 1; y < height - 1; y++) {
      for (let x = 1; x < width - 1; x++) {
        let sum = 0;
        let ki = 0;
        for (let ky = -1; ky <= 1; ky++) {
          for (let kx = -1; kx <= 1; kx++) {
            sum += gray[(y + ky) * width + (x + kx)] * kernel[ki++];
          }
        }
        result[y * width + x] = Math.round(sum / kernelSum);
      }
    }
    return result;
  }

  function medianFilter(gray, width, height, kernelSize) {
    const result = new Uint8Array(gray.length);
    const half = Math.floor(kernelSize / 2);
    const values = [];

    for (let y = 0; y < height; y++) {
      for (let x = 0; x < width; x++) {
        values.length = 0;

        for (let ky = -half; ky <= half; ky++) {
          for (let kx = -half; kx <= half; kx++) {
            const ny = y + ky;
            const nx = x + kx;
            if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
              values.push(gray[ny * width + nx]);
            }
          }
        }

        values.sort((a, b) => a - b);
        result[y * width + x] = values[Math.floor(values.length / 2)];
      }
    }

    return result;
  }

  function backgroundSubtractRollingBall(gray, width, height, radius) {
    // Gaussian blur approximation of rolling ball
    const background = new Uint8Array(gray.length);
    const result = new Uint8Array(gray.length);

    // Create large Gaussian kernel
    const kernelSize = radius * 2 + 1;
    const half = radius;
    const sigma = radius / 3;
    const kernel = [];
    let kernelSum = 0;

    for (let y = -half; y <= half; y++) {
      for (let x = -half; x <= half; x++) {
        const weight = Math.exp(-(x * x + y * y) / (2 * sigma * sigma));
        kernel.push(weight);
        kernelSum += weight;
      }
    }

    // Normalize kernel
    for (let i = 0; i < kernel.length; i++) {
      kernel[i] /= kernelSum;
    }

    // Apply large blur to estimate background
    for (let y = 0; y < height; y++) {
      for (let x = 0; x < width; x++) {
        let sum = 0;
        let ki = 0;

        for (let ky = -half; ky <= half; ky++) {
          for (let kx = -half; kx <= half; kx++) {
            const ny = y + ky;
            const nx = x + kx;
            if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
              sum += gray[ny * width + nx] * kernel[ki];
```

```
              }
              ki++;
            }
          }

          background[y * width + x] = Math.round(sum);
        }
      }

      // Subtract background
      for (let i = 0; i < gray.length; i++) {
        const val = gray[i] - background[i] + 128;
        result[i] = Math.max(0, Math.min(255, val));
      }

      return result;
    }

    function otsuThreshold(gray) {
      // Compute histogram
      const histogram = new Array(256).fill(0);
      for (let i = 0; i < gray.length; i++) {
        histogram[gray[i]]++;
      }

      const total = gray.length;
      let sum = 0;
      for (let i = 0; i < 256; i++) {
        sum += i * histogram[i];
      }

      let sumB = 0;
      let wB = 0;
      let wF = 0;
      let maxVariance = 0;
      let threshold = 0;

      for (let t = 0; t < 256; t++) {
        wB += histogram[t];
        if (wB === 0) continue;

        wF = total - wB;
        if (wF === 0) break;

        sumB += t * histogram[t];

        const mB = sumB / wB;
        const mF = (sum - sumB) / wF;

        const variance = wB * wF * (mB - mF) * (mB - mF);

        if (variance > maxVariance) {
          maxVariance = variance;
          threshold = t;
        }
      }

      return threshold;
    }

    function adaptiveThreshold(gray, width, height, blockSize, brightIsObject) {
      const binary = new Uint8Array(gray.length);
      const half = Math.floor(blockSize / 2);

      for (let y = 0; y < height; y++) {
        for (let x = 0; x < width; x++) {
          // Calculate local mean
          let sum = 0;
          let count = 0;

          for (let ky = -half; ky <= half; ky++) {
            for (let kx = -half; kx <= half; kx++) {
              const ny = y + ky;
              const nx = x + kx;
              if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
                sum += gray[ny * width + nx];
                count++;
              }
            }
          }
```

```
            const localMean = sum / count;
            const threshold = localMean - 5; // Small constant

            const i = y * width + x;
            if (brightIsObject) {
              binary[i] = gray[i] >= threshold ? 1 : 0;
            } else {
              binary[i] = gray[i] < threshold ? 1 : 0;
            }
          }
        }

        return binary;
      }

      function morphologicalErode(binary, width, height) {
        const result = new Uint8Array(binary.length);

        for (let y = 0; y < height; y++) {
          for (let x = 0; x < width; x++) {
            const i = y * width + x;

            if (binary[i] === 0) {
              result[i] = 0;
              continue;
            }

            // Check 8-connected neighbors
            let isEdge = false;
            for (let ky = -1; ky <= 1; ky++) {
              for (let kx = -1; kx <= 1; kx++) {
                const ny = y + ky;
                const nx = x + kx;
                if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
                  if (binary[ny * width + nx] === 0) {
                    isEdge = true;
                    break;
                  }
                }
              }
              if (isEdge) break;
            }

            result[i] = isEdge ? 0 : 1;
          }
        }

        return result;
      }

      function morphologicalDilate(binary, width, height) {
        const result = new Uint8Array(binary.length);

        for (let y = 0; y < height; y++) {
          for (let x = 0; x < width; x++) {
            const i = y * width + x;

            if (binary[i] === 1) {
              result[i] = 1;
              continue;
            }

            // Check 8-connected neighbors
            let hasNeighbor = false;
            for (let ky = -1; ky <= 1; ky++) {
              for (let kx = -1; kx <= 1; kx++) {
                const ny = y + ky;
                const nx = x + kx;
                if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
                  if (binary[ny * width + nx] === 1) {
                    hasNeighbor = true;
                    break;
                  }
                }
              }
              if (hasNeighbor) break;
            }

            result[i] = hasNeighbor ? 1 : 0;
          }
        }
```

```
2830
2831        return result;
2832      }
2833
2834      function morphologicalOpening(binary, width, height) {
2835        const eroded = morphologicalErode(binary, width, height);
2836        return morphologicalDilate(eroded, width, height);
2837      }
2838
2839      function thresholdAndLabelComponents(gray, width, height, threshold, brightIsObject, minArea, useAdaptive, blockSize, roi
2840        // Default ROI is the entire image
2841        if (!roi) {
2842          roi = { x: 0, y: 0, width: width, height: height };
2843        }
2844
2845        // Ensure ROI is within bounds
2846        const roiX = Math.max(0, Math.floor(roi.x));
2847        const roiY = Math.max(0, Math.floor(roi.y));
2848        const roiWidth = Math.min(width - roiX, Math.ceil(roi.width));
2849        const roiHeight = Math.min(height - roiY, Math.ceil(roi.height));
2850
2851        let binary;
2852
2853        if (useAdaptive) {
2854          binary = adaptiveThreshold(gray, width, height, blockSize, brightIsObject);
2855        } else {
2856          // Standard global threshold
2857          binary = new Uint8Array(gray.length);
2858          for (let i = 0; i < gray.length; i++) {
2859            if (brightIsObject) {
2860              binary[i] = gray[i] >= threshold ? 1 : 0;
2861            } else {
2862              binary[i] = gray[i] < threshold ? 1 : 0;
2863            }
2864          }
2865        }
2866
2867        // Apply morphological opening if requested
2868        if (elements.morphologicalOpening && elements.morphologicalOpening.checked) {
2869          binary = morphologicalOpening(binary, width, height);
2870        }
2871
2872        // Connected component labeling (8-connected) - only within ROI
2873        const labels = new Int32Array(gray.length);
2874        let currentLabel = 0;
2875        const equivalences = new Map();
2876
2877        function find(label) {
2878          if (!equivalences.has(label)) return label;
2879          let root = label;
2880          while (equivalences.has(root)) root = equivalences.get(root);
2881          // Path compression
2882          let l = label;
2883          while (equivalences.has(l)) {
2884            const next = equivalences.get(l);
2885            equivalences.set(l, root);
2886            l = next;
2887          }
2888          return root;
2889        }
2890
2891        function union(a, b) {
2892          const rootA = find(a);
2893          const rootB = find(b);
2894          if (rootA !== rootB) {
2895            equivalences.set(Math.max(rootA, rootB), Math.min(rootA, rootB));
2896          }
2897        }
2898
2899        // First pass - only within ROI
2900        for (let y = roiY; y < roiY + roiHeight; y++) {
2901          for (let x = roiX; x < roiX + roiWidth; x++) {
2902            const i = y * width + x;
2903            if (binary[i] === 0) continue;
2904
2905            const neighbors = [];
2906            if (y > roiY && x > roiX && labels[(y-1) * width + (x-1)]) neighbors.push(labels[(y-1) * width + (x-1)]);
2907            if (y > roiY && labels[(y-1) * width + x]) neighbors.push(labels[(y-1) * width + x]);
2908            if (y > roiY && x < roiX + roiWidth - 1 && labels[(y-1) * width + (x+1)]) neighbors.push(labels[(y-1) * width +
      (x+1)]);
2909            if (x > roiX && labels[y * width + (x-1)]) neighbors.push(labels[y * width + (x-1)]);
```

```
2910
2911            if (neighbors.length === 0) {
2912              labels[i] = ++currentLabel;
2913            } else {
2914              const minLabel = Math.min(...neighbors);
2915              labels[i] = minLabel;
2916              neighbors.forEach(n => union(minLabel, n));
2917            }
2918          }
2919        }
2920
2921        // Second pass - resolve equivalences and compute properties
2922        const components = new Map();
2923
2924        for (let y = roiY; y < roiY + roiHeight; y++) {
2925          for (let x = roiX; x < roiX + roiWidth; x++) {
2926            const i = y * width + x;
2927            if (labels[i] === 0) continue;
2928
2929            const root = find(labels[i]);
2930            labels[i] = root;
2931
2932            if (!components.has(root)) {
2933              components.set(root, {
2934                area: 0,
2935                minX: width, maxX: 0, minY: height, maxY: 0,
2936                pixels: []
2937              });
2938            }
2939
2940            const comp = components.get(root);
2941            comp.area++;
2942            comp.minX = Math.min(comp.minX, x);
2943            comp.maxX = Math.max(comp.maxX, x);
2944            comp.minY = Math.min(comp.minY, y);
2945            comp.maxY = Math.max(comp.maxY, y);
2946          }
2947        }
2948
2949        // Filter by min area and compute final properties
2950        const result = [];
2951        components.forEach((comp, label) => {
2952          if (comp.area >= minArea) {
2953            const ecd = 2 * Math.sqrt(comp.area / Math.PI);
2954            result.push({
2955              id: result.length + 1,
2956              areaPx: comp.area,
2957              areaReal: state.isCalibrated ? comp.area * Math.pow(state.unitsPerPixel, 2) : null,
2958              ecdPx: ecd,
2959              ecdReal: state.isCalibrated ? ecd * state.unitsPerPixel : null,
2960              bbox: {
2961                x: comp.minX,
2962                y: comp.minY,
2963                w: comp.maxX - comp.minX + 1,
2964                h: comp.maxY - comp.minY + 1
2965              }
2966            });
2967          }
2968        });
2969
2970        return result;
2971      }
2972
2973      function computeStats(components, key) {
2974        if (components.length === 0) return { count: 0, mean: 0, std: 0 };
2975
2976        const values = components.map(c => c[key]);
2977        const mean = values.reduce((a, b) => a + b, 0) / values.length;
2978        const variance = values.reduce((a, b) => a + Math.pow(b - mean, 2), 0) / values.length;
2979
2980        return {
2981          count: components.length,
2982          mean,
2983          std: Math.sqrt(variance)
2984        };
2985      }
2986
2987      // ========== DESCRIPTIVE STATISTICS (PUBLISH MODAL) ==========
2988      let currentStatsType = 'length';
2989      let lastStatsTSV = '';
2990      let statsCopyIdleText = null;
```

```
    let statsCopyTimer = null;

    function quantile(sortedValues, p) {
      const n = sortedValues.length;
      if (n === 0) return NaN;
      if (n === 1) return sortedValues[0];
      const idx = (n - 1) * p;
      const lo = Math.floor(idx);
      const hi = Math.ceil(idx);
      const h = idx - lo;
      return sortedValues[lo] + (sortedValues[hi] - sortedValues[lo]) * h;
    }

    function computeDescriptiveStats(values) {
      const clean = values.filter(v => Number.isFinite(v));
      const n = clean.length;
      if (!n) return null;
      const sorted = [...clean].sort((a, b) => a - b);
      const sum = clean.reduce((acc, v) => acc + v, 0);
      const mean = sum / n;
      const median = (n % 2 === 1) ? sorted[(n - 1) / 2] : (sorted[n / 2 - 1] + sorted[n / 2]) / 2;
      const min = sorted[0];
      const max = sorted[n - 1];
      const q1 = quantile(sorted, 0.25);
      const q3 = quantile(sorted, 0.75);
      const iqr = q3 - q1;

      let std = 0;
      if (n > 1) {
        const ss = clean.reduce((acc, v) => acc + Math.pow(v - mean, 2), 0);
        std = Math.sqrt(ss / (n - 1)); // sample std dev
      }
      const sem = std / Math.sqrt(n);
      const cv = mean !== 0 ? (std / mean) * 100 : 0;

      return { n, sum, mean, median, std, sem, min, max, q1, q3, iqr, cv };
    }

    function statsUnitLabel(kind) {
      if (kind === 'length') return state.isCalibrated ? state.unit : 'px';
      if (kind === 'angle') return 'deg';
      if (kind === 'area') return state.isCalibrated ? `${state.unit}²` : 'px²';
      if (kind === 'ecd') return state.isCalibrated ? state.unit : 'px';
      if (kind === 'componentArea') return state.isCalibrated ? `${state.unit}²` : 'px²';
      return '';
    }

    function getStatsSectionsForType(type) {
      if (type === 'length') {
        return [{
          label: 'Length',
          unit: statsUnitLabel('length'),
          values: state.lengths.map(l => state.isCalibrated ? l.realLength : l.pixelLength)
        }];
      }
      if (type === 'angle') {
        return [{
          label: 'Angle',
          unit: statsUnitLabel('angle'),
          values: state.angles.map(a => a.angleDeg)
        }];
      }
      if (type === 'area') {
        return [{
          label: 'Area',
          unit: statsUnitLabel('area'),
          values: state.areas.map(a => state.isCalibrated ? a.realArea : a.pixelArea)
        }];
      }
      if (type === 'particles') {
        return [
          {
            label: 'Particle Area',
            unit: statsUnitLabel('componentArea'),
            values: state.particles.map(p => state.isCalibrated ? p.areaReal : p.areaPx)
          },
          {
            label: 'Particle ECD',
            unit: statsUnitLabel('ecd'),
            values: state.particles.map(p => state.isCalibrated ? p.ecdReal : p.ecdPx)
          }
```

```
        ];
      }
      if (type === 'holes') {
        return [
          {
            label: 'Hole Area',
            unit: statsUnitLabel('componentArea'),
            values: state.holes.map(h => state.isCalibrated ? h.areaReal : h.areaPx)
          },
          {
            label: 'Hole ECD',
            unit: statsUnitLabel('ecd'),
            values: state.holes.map(h => state.isCalibrated ? h.ecdReal : h.ecdPx)
          }
        ];
      }
      return [];
    }

    function renderStatsTable(stats, unit) {
      const fmt = (x) => Number.isFinite(x) ? formatNumber(x) : '-';
      const row = (label, value) => `
        <tr class="border-b border-slate-200">
          <th class="text-left font-medium py-2 pr-3 align-top text-slate-700 w-[55%]">${label}</th>
          <td class="py-2 mono text-slate-900">${value}</td>
        </tr>
      `;

      return `
        <div class="overflow-x-auto">
          <table class="w-full text-xs">
            <tbody>
              ${row('Count (n)', stats.n)}
              ${row('Mean', `${fmt(stats.mean)} ${unit}`)}
              ${row('Median', `${fmt(stats.median)} ${unit}`)}
              ${row('Std Dev (sample)', `${fmt(stats.std)} ${unit}`)}
              ${row('Std Error (SEM)', `${fmt(stats.sem)} ${unit}`)}
              ${row('Min', `${fmt(stats.min)} ${unit}`)}
              ${row('Q1 (25%)', `${fmt(stats.q1)} ${unit}`)}
              ${row('Q3 (75%)', `${fmt(stats.q3)} ${unit}`)}
              ${row('Max', `${fmt(stats.max)} ${unit}`)}
              ${row('IQR', `${fmt(stats.iqr)} ${unit}`)}
              ${row('Sum', `${fmt(stats.sum)} ${unit}`)}
              ${row('CV (%)', `${fmt(stats.cv)} %`)}
            </tbody>
          </table>
        </div>
      `;
    }

    function buildStatsTSV(type, sections) {
      const lines = [];
      lines.push(`Type\tMetric\tUnit\tn\tMean\tMedian\tStdDev(sample)\tSEM\tMin\tQ1\tQ3\tMax\tIQR\tSum\tCV(%)`);
      sections.forEach(sec => {
        const stats = computeDescriptiveStats(sec.values);
        if (!stats) return;
        const vals = [
          type,
          sec.label,
          sec.unit,
          stats.n,
          stats.mean,
          stats.median,
          stats.std,
          stats.sem,
          stats.min,
          stats.q1,
          stats.q3,
          stats.max,
          stats.iqr,
          stats.sum,
          stats.cv
        ];
        lines.push(vals.join('\t'));
      });
      return lines.join('\n');
    }

    function setStatsType(type) {
      currentStatsType = type;
      if (elements.statsTypeBtns) {
```

```
        elements.statsTypeBtns.forEach(btn => {
          const active = btn.dataset.statsType === type;
          btn.classList.toggle('bg-blue-600', active);
          btn.classList.toggle('text-white', active);
          btn.classList.toggle('border-blue-600', active);
          btn.classList.toggle('bg-slate-50', !active);
          btn.classList.toggle('text-slate-800', !active);
          btn.classList.toggle('border-slate-200', !active);
        });
      }
      renderStatsModal();
    }

    function renderStatsModal() {
      const type = currentStatsType;
      const titleMap = {
        length: 'Length — Descriptive Statistics',
        angle: 'Angle — Descriptive Statistics',
        area: 'Area — Descriptive Statistics',
        particles: 'Particles — Descriptive Statistics',
        holes: 'Holes — Descriptive Statistics'
      };
      if (elements.statsModalTitle) elements.statsModalTitle.textContent = titleMap[type] || 'Descriptive Statistics';

      const sections = getStatsSectionsForType(type);
      const totalN = sections.length ? (sections[0].values.filter(v => Number.isFinite(v)).length) : 0;
      const unitNote = (type === 'angle')
        ? 'Degrees (calibration not required)'
        : (state.isCalibrated ? `Real units (${state.unit})` : 'Pixels (set scale for real units)');
      if (elements.statsModalSubtitle) {
        elements.statsModalSubtitle.textContent = `Based on ${totalN} values. ${unitNote}. (Shown only when n ≥ 4)`;
      }

      // Button availability: only when at least one section has n>=4
      const hasEnough = sections.some(sec => sec.values.filter(v => Number.isFinite(v)).length >= 4);
      if (elements.copyStatsTSVBtn) elements.copyStatsTSVBtn.disabled = !hasEnough;

      if (!elements.statsModalContent) return;

      if (!hasEnough) {
        elements.statsModalContent.innerHTML = `
          <div class="rounded-xl border border-slate-200 bg-slate-50 p-4">
            <p class="text-sm font-medium text-slate-800">Not enough data yet</p>
            <p class="text-xs text-slate-600 mt-1">Add at least 4 measurements/objects in this category to show descriptive
        statistics.</p>
          </div>
        `;
        lastStatsTSV = '';
        return;
      }

      elements.statsModalContent.innerHTML = sections.map(sec => {
        const n = sec.values.filter(v => Number.isFinite(v)).length;
        if (n < 4) {
          return `
            <div class="rounded-xl border border-slate-200 bg-slate-50 p-4">
              <div class="flex items-baseline justify-between gap-2">
                <h4 class="text-sm font-semibold text-slate-900">${sec.label}</h4>
                <span class="text-xs text-slate-500 mono">n=${n}</span>
              </div>
              <p class="text-xs text-slate-600 mt-1">Need at least 4 values for this metric.</p>
            </div>
          `;
        }
        const stats = computeDescriptiveStats(sec.values);
        return `
          <div class="rounded-xl border border-slate-200 bg-white p-4">
            <div class="flex items-baseline justify-between gap-2 mb-3">
              <h4 class="text-sm font-semibold text-slate-900">${sec.label}</h4>
              <span class="text-xs text-slate-500 mono">n=${stats.n} • ${sec.unit}</span>
            </div>
            ${renderStatsTable(stats, sec.unit)}
          </div>
        `;
      }).join('');

      lastStatsTSV = buildStatsTSV(type, sections);
    }

    function openStatsModal(type) {
      if (!elements.descriptiveStatsModal) return;
```

```
    elements.descriptiveStatsModal.classList.remove('hidden');
    setStatsType(type);
  }

  function closeStatsModal() {
    if (!elements.descriptiveStatsModal) return;
    elements.descriptiveStatsModal.classList.add('hidden');
  }

  function setStatsCopyFeedback(text) {
    const btn = elements.copyStatsTSVBtn;
    if (!btn) return;

    if (statsCopyIdleText === null) statsCopyIdleText = btn.textContent;
    btn.textContent = text;

    // Keep the button from being spam-clicked; restore soon.
    btn.disabled = true;
    if (statsCopyTimer) clearTimeout(statsCopyTimer);
    statsCopyTimer = setTimeout(() => {
      btn.textContent = statsCopyIdleText;
      // Restore correct disabled state (depends on current dataset)
      renderStatsModal();
    }, 1200);
  }

  function fallbackCopyText(text) {
    const textarea = document.createElement('textarea');
    textarea.value = text;
    textarea.setAttribute('readonly', '');
    textarea.style.position = 'fixed';
    textarea.style.left = '-9999px';
    textarea.style.top = '0';
    document.body.appendChild(textarea);
    textarea.select();
    textarea.setSelectionRange(0, textarea.value.length);
    const ok = document.execCommand('copy');
    document.body.removeChild(textarea);
    return ok;
  }

  async function copyStatsTSV() {
    if (!lastStatsTSV) {
      setStatsCopyFeedback('No data');
      return;
    }

    try {
      if (navigator.clipboard && navigator.clipboard.writeText) {
        await navigator.clipboard.writeText(lastStatsTSV);
        setStatsCopyFeedback('Copied');
        return;
      }
    } catch (e) {
      // Fall back below
    }

    const ok = fallbackCopyText(lastStatsTSV);
    setStatsCopyFeedback(ok ? 'Copied' : 'Copy failed');
  }

  // ========== HISTOGRAM MODAL (publish-ready, light) ==========
  let currentHistType = 'length';

  function histogramUnitLabel(type, metric) {
    if (type === 'length') return statsUnitLabel('length');
    if (type === 'angle') return statsUnitLabel('angle');
    if (type === 'area') return statsUnitLabel('area');
    if (type === 'particles' || type === 'holes') {
      if (metric === 'area') return statsUnitLabel('componentArea');
      return statsUnitLabel('ecd');
    }
    return '';
  }

  function getHistogramSeriesForType(type) {
    const metricPref = elements.histMetricSelect ? elements.histMetricSelect.value : 'auto';
    const metric = (type === 'particles' || type === 'holes')
      ? (metricPref === 'auto' ? 'ecd' : metricPref)
      : 'value';

```

```
    let values = [];
    let label = '';
    let unit = '';

    if (type === 'length') {
      values = state.lengths.map(l => state.isCalibrated ? l.realLength : l.pixelLength);
      unit = histogramUnitLabel('length');
      label = `Length (${unit})`;
    } else if (type === 'angle') {
      values = state.angles.map(a => a.angleDeg);
      unit = histogramUnitLabel('angle');
      label = `Angle (${unit})`;
    } else if (type === 'area') {
      values = state.areas.map(a => state.isCalibrated ? a.realArea : a.pixelArea);
      unit = histogramUnitLabel('area');
      label = `Area (${unit})`;
    } else if (type === 'particles') {
      if (metric === 'area') {
        values = state.particles.map(p => state.isCalibrated ? p.areaReal : p.areaPx);
        unit = histogramUnitLabel('particles', 'area');
        label = `Particle Area (${unit})`;
      } else {
        values = state.particles.map(p => state.isCalibrated ? p.ecdReal : p.ecdPx);
        unit = histogramUnitLabel('particles', 'ecd');
        label = `Particle ECD (${unit})`;
      }
    } else if (type === 'holes') {
      if (metric === 'area') {
        values = state.holes.map(h => state.isCalibrated ? h.areaReal : h.areaPx);
        unit = histogramUnitLabel('holes', 'area');
        label = `Hole Area (${unit})`;
      } else {
        values = state.holes.map(h => state.isCalibrated ? h.ecdReal : h.ecdPx);
        unit = histogramUnitLabel('holes', 'ecd');
        label = `Hole ECD (${unit})`;
      }
    }

    const clean = values.filter(v => Number.isFinite(v));
    return { values: clean, label, unit, metric };
  }

  function computeHistogram(values, binCount) {
    if (!values.length) return { bins: [], min: NaN, max: NaN };
    const min = Math.min(...values);
    const max = Math.max(...values);
    const range = (max - min) || 1;
    const k = Math.max(3, Math.min(80, Math.round(binCount)));
    const step = range / k;

    const bins = new Array(k).fill(0);
    for (const v of values) {
      const idx = Math.min(k - 1, Math.max(0, Math.floor((v - min) / step)));
      bins[idx] += 1;
    }
    return { bins, min, max };
  }

  function escapeXml(s) {
    return String(s)
      .replaceAll('&', '&amp;')
      .replaceAll('<', '&lt;')
      .replaceAll('>', '&gt;')
      .replaceAll('"', '&quot;')
      .replaceAll("'", '&apos;');
  }

  function niceTicks(min, max, tickCount = 5) {
    if (!Number.isFinite(min) || !Number.isFinite(max)) return [];
    if (min === max) return [min];
    const span = max - min;
    const rawStep = span / Math.max(1, tickCount);
    const mag = Math.pow(10, Math.floor(Math.log10(rawStep)));
    const norm = rawStep / mag;
    const niceNorm = norm <= 1 ? 1 : (norm <= 2 ? 2 : (norm <= 5 ? 5 : 10));
    const step = niceNorm * mag;
    const start = Math.floor(min / step) * step;
    const end = Math.ceil(max / step) * step;
    const out = [];
    for (let v = start; v <= end + step / 2; v += step) out.push(v);
    return out;
```

```
  }

  function renderHistogramSvg() {
    const svg = elements.histSvg;
    if (!svg) return;

    const binsInput = elements.histBinsNumber ? parseInt(elements.histBinsNumber.value || '15', 10) : 15;
    const binCount = Number.isFinite(binsInput) ? binsInput : 15;

    const xTicksInput = elements.histXTicksNumber ? parseInt(elements.histXTicksNumber.value || '6', 10) : 6;
    const xTickCount = Number.isFinite(xTicksInput) ? xTicksInput : 6;

    const yTicksInput = elements.histYTicksNumber ? parseInt(elements.histYTicksNumber.value || '6', 10) : 6;
    const yTickCount = Number.isFinite(yTicksInput) ? yTicksInput : 6;

    const { values, label } = getHistogramSeriesForType(currentHistType);
    const n = values.length;

    if (elements.histN) elements.histN.textContent = n ? String(n) : '0';

    if (!n) {
      svg.innerHTML = '';
      if (elements.histRange) elements.histRange.textContent = '-';
      if (elements.histEmptyNote) elements.histEmptyNote.classList.remove('hidden');
      if (elements.downloadHistSvgBtn) elements.downloadHistSvgBtn.disabled = true;
      return;
    }

    if (elements.histEmptyNote) elements.histEmptyNote.classList.add('hidden');
    if (elements.downloadHistSvgBtn) elements.downloadHistSvgBtn.disabled = false;

    const { bins, min, max } = computeHistogram(values, binCount);
    if (elements.histRange) elements.histRange.textContent = `${formatNumber(min)} to ${formatNumber(max)}`;

    const W = 900, H = 520;
    const margin = { l: 76, r: 22, t: 52, b: 74 };
    const iw = W - margin.l - margin.r;
    const ih = H - margin.t - margin.b;
    const x0 = margin.l;
    const y0 = margin.t + ih;

    const maxCount = Math.max(...bins, 1);
    const barW = iw / bins.length;
    const yScale = (c) => (margin.t + ih) - (c / maxCount) * ih;

    const xTicks = niceTicks(min, max, Math.max(2, Math.min(12, xTickCount)));
    const yTicks = niceTicks(0, maxCount, Math.max(2, Math.min(12, yTickCount)));

    const gridY = yTicks.map(v => {
      const y = yScale(v);
      return `<line x1="${x0}" y1="${y}" x2="${x0 + iw}" y2="${y}" stroke="#e5e7eb" stroke-width="1" />`;
    }).join('\n');

    const gridX = xTicks.map(v => {
      const t = (v - min) / ((max - min) || 1);
      if (t <= 0 || t >= 1) return '';
      const x = x0 + t * iw;
      return `<line x1="${x}" y1="${margin.t}" x2="${x}" y2="${y0}" stroke="#e5e7eb" stroke-width="1" />`;
    }).filter(Boolean).join('\n');

    const bars = bins.map((c, i) => {
      const x = x0 + i * barW;
      const y = yScale(c);
      const h = y0 - y;
      return `<rect x="${x + 1}" y="${y}" width="${Math.max(0, barW - 2)}" height="${h}" style="fill: #000000; fill-opacity
    1; stroke: #000000; stroke-opacity: 1;" />`;
    }).join('\n');

    const axis = `
      <line x1="${x0}" y1="${y0}" x2="${x0 + iw}" y2="${y0}" stroke="#0f172a" stroke-width="1.5" />
      <line x1="${x0}" y1="${margin.t}" x2="${x0}" y2="${y0}" stroke="#0f172a" stroke-width="1.5" />
    `;

    const yLabels = yTicks.map(v => {
      const y = yScale(v);
      return `
        <line x1="${x0 - 6}" y1="${y}" x2="${x0}" y2="${y}" stroke="#0f172a" stroke-width="1.25" />
        <text x="${x0 - 10}" y="${y}" text-anchor="end" dominant-baseline="middle" font-family="Inter, sans-serif"
    font-size="12" fill="#0f172a">${escapeXml(String(Math.round(v)))}</text>
      `;
    }).join('\n');
```

```
3474
3475        const xLabels = xTicks.map(v => {
3476          const t = (v - min) / ((max - min) || 1);
3477          const x = x0 + t * iw;
3478          return `
3479            <line x1="${x}" y1="${y0}" x2="${x}" y2="${y0 + 6}" stroke="#0f172a" stroke-width="1.25" />
3480            <text x="${x}" y="${y0 + 22}" text-anchor="middle" dominant-baseline="hanging" font-family="Inter, sans-serif"
          font-size="12" fill="#0f172a">${escapeXml(formatNumber(v))}</text>
3481          `;
3482        }).join('\n');
3483
3484        const titleMap = {
3485          length: 'Length Histogram',
3486          angle: 'Angle Histogram',
3487          area: 'Area Histogram',
3488          particles: 'Particles Histogram',
3489          holes: 'Holes Histogram'
3490        };
3491        const title = titleMap[currentHistType] || 'Histogram';
3492
3493        svg.innerHTML = `
3494          <rect x="0" y="0" width="${W}" height="${H}" fill="#ffffff" />
3495          <text x="${margin.l}" y="28" font-family="Inter, sans-serif" font-size="18" font-weight="600"
          fill="#0f172a">${escapeXml(title)}</text>
3496          <text x="${margin.l}" y="48" font-family="Inter, sans-serif" font-size="12" fill="#64748b">${escapeXml(label)} •
          bins=${bins.length}</text>
3497          ${gridY}
3498          ${gridX}
3499          ${axis}
3500          ${bars}
3501          ${yLabels}
3502          ${xLabels}
3503          <text x="${margin.l + iw / 2}" y="${H - 26}" text-anchor="middle" font-family="Inter, sans-serif" font-size="13"
          fill="#0f172a">${escapeXml(label)}</text>
3504          <text x="18" y="${margin.t + ih / 2}" text-anchor="middle" transform="rotate(-90 18 ${margin.t + ih / 2})"
          font-family="Inter, sans-serif" font-size="13" fill="#0f172a">Count</text>
3505        `;
3506      }
3507
3508      function setHistType(type) {
3509        currentHistType = type;
3510        if (elements.histTypeBtns) {
3511          elements.histTypeBtns.forEach(btn => {
3512            const active = btn.dataset.histType === type;
3513            btn.classList.toggle('bg-blue-600', active);
3514            btn.classList.toggle('text-white', active);
3515            btn.classList.toggle('border-blue-600', active);
3516            btn.classList.toggle('bg-slate-50', !active);
3517            btn.classList.toggle('text-slate-800', !active);
3518            btn.classList.toggle('border-slate-200', !active);
3519          });
3520        }
3521        renderHistogramModal();
3522      }
3523
3524      function renderHistogramModal() {
3525        const titleMap = {
3526          length: 'Length — Histogram',
3527          angle: 'Angle — Histogram',
3528          area: 'Area — Histogram',
3529          particles: 'Particles — Histogram',
3530          holes: 'Holes — Histogram'
3531        };
3532        if (elements.histModalTitle) elements.histModalTitle.textContent = titleMap[currentHistType] || 'Histogram';
3533
3534        const unitNote = (currentHistType === 'angle')
3535          ? 'Degrees (calibration not required)'
3536          : (state.isCalibrated ? `Real units (${state.unit})` : 'Pixels (set scale for real units)');
3537        if (elements.histModalSubtitle) {
3538          elements.histModalSubtitle.textContent = `${unitNote}. Adjust bins to control resolution.`;
3539        }
3540
3541        const showMetric = (currentHistType === 'particles' || currentHistType === 'holes');
3542        if (elements.histMetricSelect) {
3543          elements.histMetricSelect.disabled = !showMetric;
3544          if (elements.histMetricSelect.parentElement) {
3545            elements.histMetricSelect.parentElement.classList.toggle('opacity-60', !showMetric);
3546          }
3547        }
3548
3549        renderHistogramSvg();
```

```
  }

  function openHistogramModal(type) {
    if (!elements.histogramModal) return;
    elements.histogramModal.classList.remove('hidden');
    setHistType(type);
  }

  function closeHistogramModal() {
    if (!elements.histogramModal) return;
    elements.histogramModal.classList.add('hidden');
  }

  function downloadCurrentHistogramSvg() {
    const svg = elements.histSvg;
    if (!svg || !svg.innerHTML.trim()) return;

    const serializer = new XMLSerializer();
    const svgText = serializer.serializeToString(svg);
    const blob = new Blob([svgText], { type: 'image/svg+xml;charset=utf-8' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `micromeasure_histogram_${currentHistType}.svg`;
    a.click();
    URL.revokeObjectURL(url);
  }

  function updateStatsButtonsVisibility() {
    const showLen = state.lengths.length >= 4;
    const showAngle = state.angles.length >= 4;
    const showArea = state.areas.length >= 4;
    const showParticles = state.particles.length >= 4;
    const showHoles = state.holes.length >= 4;

    if (elements.lengthStatsBtn) elements.lengthStatsBtn.classList.toggle('hidden', !showLen);
    if (elements.lengthHistBtn) elements.lengthHistBtn.classList.toggle('hidden', !showLen);
    if (elements.angleStatsBtn) elements.angleStatsBtn.classList.toggle('hidden', !showAngle);
    if (elements.angleHistBtn) elements.angleHistBtn.classList.toggle('hidden', !showAngle);
    if (elements.areaStatsBtn) elements.areaStatsBtn.classList.toggle('hidden', !showArea);
    if (elements.areaHistBtn) elements.areaHistBtn.classList.toggle('hidden', !showArea);
    if (elements.particleStatsBtn) elements.particleStatsBtn.classList.toggle('hidden', !showParticles);
    if (elements.particleHistBtn) elements.particleHistBtn.classList.toggle('hidden', !showParticles);
    if (elements.holeStatsBtn) elements.holeStatsBtn.classList.toggle('hidden', !showHoles);
    if (elements.holeHistBtn) elements.holeHistBtn.classList.toggle('hidden', !showHoles);

    // Per-feature actions (export + clear)
    updateFeatureActionButtons();

    if (elements.histogramModal && !elements.histogramModal.classList.contains('hidden')) {
      renderHistogramModal();
    }
  }

  function updateFeatureActionButtons() {
    const hasLen = state.lengths.length > 0;
    const hasAngle = state.angles.length > 0;
    const hasArea = state.areas.length > 0;
    const hasParticles = state.particles.length > 0;
    const hasHoles = state.holes.length > 0;

    if (elements.lengthExportRow) elements.lengthExportRow.classList.toggle('hidden', !hasLen);
    if (elements.angleExportRow) elements.angleExportRow.classList.toggle('hidden', !hasAngle);
    if (elements.areaExportRow) elements.areaExportRow.classList.toggle('hidden', !hasArea);
    if (elements.particlesExportRow) elements.particlesExportRow.classList.toggle('hidden', !hasParticles);
    if (elements.holesExportRow) elements.holesExportRow.classList.toggle('hidden', !hasHoles);

    if (elements.copyLengthTSVBtn) elements.copyLengthTSVBtn.disabled = !hasLen;
    if (elements.downloadLengthTSVBtn) elements.downloadLengthTSVBtn.disabled = !hasLen;
    if (elements.copyAngleTSVBtn) elements.copyAngleTSVBtn.disabled = !hasAngle;
    if (elements.downloadAngleTSVBtn) elements.downloadAngleTSVBtn.disabled = !hasAngle;
    if (elements.copyAreaTSVBtn) elements.copyAreaTSVBtn.disabled = !hasArea;
    if (elements.downloadAreaTSVBtn) elements.downloadAreaTSVBtn.disabled = !hasArea;
    if (elements.copyParticlesTSVBtn) elements.copyParticlesTSVBtn.disabled = !hasParticles;
    if (elements.downloadParticlesTSVBtn) elements.downloadParticlesTSVBtn.disabled = !hasParticles;
    if (elements.copyHolesTSVBtn) elements.copyHolesTSVBtn.disabled = !hasHoles;
    if (elements.downloadHolesTSVBtn) elements.downloadHolesTSVBtn.disabled = !hasHoles;

    if (elements.clearParticles) elements.clearParticles.classList.toggle('hidden', !hasParticles);
    if (elements.clearHoles) elements.clearHoles.classList.toggle('hidden', !hasHoles);
  }
```

```
3631
3632      function drawHistogram(canvas, data, color) {
3633        const ctx = canvas.getContext('2d');
3634        const w = canvas.width = canvas.clientWidth * 2;
3635        const h = canvas.height = canvas.clientHeight * 2;
3636        ctx.scale(2, 2);
3637
3638        const width = w / 2;
3639        const height = h / 2;
3640
3641        ctx.clearRect(0, 0, width, height);
3642
3643        if (data.length === 0) {
3644          ctx.fillStyle = '#64748b';
3645          ctx.font = '11px Inter';
3646          ctx.textAlign = 'center';
3647          ctx.fillText('No data', width / 2, height / 2);
3648          return;
3649        }
3650
3651        // Create histogram bins
3652        const numBins = Math.min(20, Math.max(5, Math.ceil(Math.sqrt(data.length))));
3653        const min = Math.min(...data);
3654        const max = Math.max(...data);
3655        const range = max - min || 1;
3656        const binWidth = range / numBins;
3657
3658        const bins = new Array(numBins).fill(0);
3659        data.forEach(v => {
3660          const bin = Math.min(numBins - 1, Math.floor((v - min) / binWidth));
3661          bins[bin]++;
3662        });
3663
3664        const maxCount = Math.max(...bins);
3665        const barWidth = (width - 40) / numBins;
3666        const chartHeight = height - 20;
3667
3668        // Draw bars
3669        ctx.fillStyle = color;
3670        bins.forEach((count, i) => {
3671          const barHeight = (count / maxCount) * (chartHeight - 10);
3672          ctx.fillRect(30 + i * barWidth + 1, chartHeight - barHeight, barWidth - 2, barHeight);
3673        });
3674
3675        // Draw axis labels
3676        ctx.fillStyle = '#94a3b8';
3677        ctx.font = '9px JetBrains Mono';
3678        ctx.textAlign = 'center';
3679        ctx.fillText(min.toFixed(1), 30, height - 2);
3680        ctx.fillText(max.toFixed(1), width - 10, height - 2);
3681
3682        ctx.textAlign = 'right';
3683        ctx.fillText(maxCount.toString(), 25, 12);
3684        ctx.fillText('0', 25, chartHeight);
3685      }
3686
3687      // ========== ROI SELECTION ==========
3688      function startROISelection(mode) {
3689        state.roiMode = mode;
3690        state.isDrawingROI = true;
3691        state.roi = null;
3692
3693        // Update UI
3694        if (mode === 'particles') {
3695          elements.selectParticleROI.textContent = '═■ Drawing ROI... (Click & Drag)';
3696          elements.selectParticleROI.classList.remove('btn-secondary');
3697          elements.selectParticleROI.classList.add('btn-primary');
3698        } else {
3699          elements.selectHoleROI.textContent = '═■ Drawing ROI... (Click & Drag)';
3700          elements.selectHoleROI.classList.remove('btn-secondary');
3701          elements.selectHoleROI.classList.add('btn-primary');
3702        }
3703
3704        elements.overlayCanvas.style.cursor = 'crosshair';
3705        drawOverlay();
3706      }
3707
3708      function finishROISelection() {
3709        state.isDrawingROI = false;
3710        state.roiStart = null;
3711
```

```
3712        if (!state.roi) {
3713          // User cancelled, reset buttons
3714          if (state.roiMode === 'particles') {
3715            elements.selectParticleROI.textContent = '■ Select ROI (Draw Rectangle)';
3716            elements.selectParticleROI.classList.add('btn-secondary');
3717            elements.selectParticleROI.classList.remove('btn-primary');
3718          } else {
3719            elements.selectHoleROI.textContent = '■ Select ROI (Draw Rectangle)';
3720            elements.selectHoleROI.classList.add('btn-secondary');
3721            elements.selectHoleROI.classList.remove('btn-primary');
3722          }
3723          state.roiMode = null;
3724          return;
3725        }
3726
3727        // Update UI to show ROI is active
3728        const roiText = `${Math.round(state.roi.width)}×${Math.round(state.roi.height)} px at (${Math.round(state.roi.x)},
           ${Math.round(state.roi.y)})`;
3729
3730        if (state.roiMode === 'particles') {
3731          elements.particleROIDimensions.textContent = roiText;
3732          elements.particleROIInfo.classList.remove('hidden');
3733          elements.selectParticleROI.classList.add('hidden');
3734        } else {
3735          elements.holeROIDimensions.textContent = roiText;
3736          elements.holeROIInfo.classList.remove('hidden');
3737          elements.selectHoleROI.classList.add('hidden');
3738        }
3739
3740        state.roiMode = null;
3741        elements.overlayCanvas.style.cursor = 'crosshair';
3742      }
3743
3744      function clearROI(mode) {
3745        state.roi = null;
3746
3747        if (mode === 'particles') {
3748          elements.particleROIInfo.classList.add('hidden');
3749          elements.selectParticleROI.classList.remove('hidden');
3750          elements.selectParticleROI.textContent = '■ Select ROI (Draw Rectangle)';
3751          elements.selectParticleROI.classList.add('btn-secondary');
3752          elements.selectParticleROI.classList.remove('btn-primary');
3753        } else {
3754          elements.holeROIInfo.classList.add('hidden');
3755          elements.selectHoleROI.classList.remove('hidden');
3756          elements.selectHoleROI.textContent = '■ Select ROI (Draw Rectangle)';
3757          elements.selectHoleROI.classList.add('btn-secondary');
3758          elements.selectHoleROI.classList.remove('btn-primary');
3759        }
3760
3761        drawOverlay();
3762      }
3763
3764      // ========== PROGRESS MODAL ==========
3765      function showProcessingModal(title = 'Processing Image') {
3766        elements.processingTitle.textContent = title;
3767        elements.processingStep.textContent = 'Initializing...';
3768        elements.processingTime.textContent = 'Estimating time...';
3769        elements.progressBar.style.width = '0%';
3770        elements.processingModal.classList.remove('hidden');
3771      }
3772
3773      function updateProgress(step, progress, timeLeft = null) {
3774        elements.processingStep.textContent = step;
3775        elements.progressBar.style.width = progress + '%';
3776
3777        if (timeLeft !== null) {
3778          if (timeLeft < 1) {
3779            elements.processingTime.textContent = 'Less than 1 second remaining';
3780          } else if (timeLeft === 1) {
3781            elements.processingTime.textContent = '1 second remaining';
3782          } else {
3783            elements.processingTime.textContent = `${Math.ceil(timeLeft)} seconds remaining`;
3784          }
3785        }
3786      }
3787
3788      function hideProcessingModal() {
3789        elements.processingModal.classList.add('hidden');
3790      }
3791
```

```
    // Async wrapper to allow UI updates during processing
    function sleep(ms) {
      return new Promise(resolve => setTimeout(resolve, ms));
    }

    async function analyzeParticles() {
      const startTime = Date.now();
      showProcessingModal('Analyzing Particles');

      await sleep(50); // Let modal render

      const imageData = getImageData();

      // Determine analysis region
      const roi = state.roi || { x: 0, y: 0, width: imageData.width, height: imageData.height };

      const totalSteps = 4;
      let currentStep = 0;

      // Step 1: Convert to grayscale
      updateProgress('Converting to grayscale...', (++currentStep / totalSteps) * 100, null);
      await sleep(10);
      let gray = toGrayscale(imageData);

      // Step 2: Background subtraction
      if (elements.backgroundSubtraction.checked) {
        const elapsed = (Date.now() - startTime) / 1000;
        const estimated = elapsed * (totalSteps / currentStep) - elapsed;
        updateProgress('Subtracting background...', (++currentStep / totalSteps) * 100, estimated);
        await sleep(10);
        const radius = parseInt(elements.rollingBallRadius.value);
        gray = backgroundSubtractRollingBall(gray, imageData.width, imageData.height, radius);
      } else {
        currentStep++;
      }

      // Step 3: Smoothing
      const smoothMethod = elements.smoothingMethod.value;
      if (smoothMethod !== 'none') {
        const elapsed = (Date.now() - startTime) / 1000;
        const estimated = elapsed * (totalSteps / currentStep) - elapsed;
        updateProgress('Applying smoothing filter...', (++currentStep / totalSteps) * 100, estimated);
        await sleep(10);

        if (smoothMethod === 'gaussian3') {
          gray = blur3x3(gray, imageData.width, imageData.height);
        } else if (smoothMethod === 'median3') {
          gray = medianFilter(gray, imageData.width, imageData.height, 3);
        } else if (smoothMethod === 'median5') {
          gray = medianFilter(gray, imageData.width, imageData.height, 5);
        }
      } else {
        currentStep++;
      }

      // Step 4: Thresholding and labeling
      const elapsed = (Date.now() - startTime) / 1000;
      const estimated = elapsed * (totalSteps / currentStep) - elapsed;
      updateProgress('Detecting particles...', (++currentStep / totalSteps) * 100, estimated);
      await sleep(10);

      const threshold = parseInt(elements.particleThreshold.value);
      const minAreaInput = parseFloat(elements.minParticleArea.value) || 0.001;
      // If calibrated, convert from real units back to pixels
      const minArea = state.isCalibrated ? Math.round(minAreaInput / Math.pow(state.unitsPerPixel, 2)) :
      Math.round(minAreaInput);
      const brightIsObject = document.querySelector('input[name="particlePolarity"]:checked').value === 'bright';
      const useAdaptive = elements.adaptiveThreshold.checked;
      const blockSize = parseInt(elements.adaptiveBlockSize.value);

      state.particles = thresholdAndLabelComponents(
        gray,
        imageData.width,
        imageData.height,
        threshold,
        brightIsObject,
        minArea,
        useAdaptive,
        blockSize,
        roi
      );
```

```
3872
3873       // Finalize
3874       updateProgress('Finalizing results...', 100, 0);
3875       await sleep(10);
3876
3877       updateParticleResults();
3878       drawOverlay();
3879       updateGuidance();
3880       updateSaveButtonState();
3881
3882       // Show completion briefly before hiding
3883       await sleep(300);
3884       hideProcessingModal();
3885     }
3886
3887     function updateParticleResults() {
3888       const stats = computeStats(state.particles, 'areaPx');
3889       const ecdStats = computeStats(state.particles, 'ecdPx');
3890
3891       elements.particleCount.textContent = stats.count;
3892       elements.particleMeanArea.textContent = formatNumber(stats.mean) + ' px²' +
3893         (state.isCalibrated ? ` (${formatNumber(stats.mean * Math.pow(state.unitsPerPixel, 2))} ${state.unit}²)` : '');
3894       elements.particleMeanECD.textContent = formatNumber(ecdStats.mean) + ' px' +
3895         (state.isCalibrated ? ` (${formatNumber(ecdStats.mean * state.unitsPerPixel)} ${state.unit})` : '');
3896       elements.particleStdDev.textContent = formatNumber(ecdStats.std) + ' px';
3897
3898       elements.particleResults.classList.remove('hidden');
3899       drawHistogram(elements.particleHistogram, state.particles.map(p => p.ecdPx), '#f97316');
3900       updateStatsButtonsVisibility();
3901     }
3902
3903     async function analyzeHoles() {
3904       const startTime = Date.now();
3905       showProcessingModal('Analyzing Holes');
3906
3907       await sleep(50); // Let modal render
3908
3909       const imageData = getImageData();
3910
3911       // Determine analysis region
3912       const roi = state.roi || { x: 0, y: 0, width: imageData.width, height: imageData.height };
3913
3914       const totalSteps = 4;
3915       let currentStep = 0;
3916
3917       // Step 1: Convert to grayscale
3918       updateProgress('Converting to grayscale...', (++currentStep / totalSteps) * 100, null);
3919       await sleep(10);
3920       let gray = toGrayscale(imageData);
3921
3922       // Step 2: Background subtraction
3923       if (document.getElementById('holeBackgroundSubtraction').checked) {
3924         const elapsed = (Date.now() - startTime) / 1000;
3925         const estimated = elapsed * (totalSteps / currentStep) - elapsed;
3926         updateProgress('Subtracting background...', (++currentStep / totalSteps) * 100, estimated);
3927         await sleep(10);
3928         const radius = parseInt(document.getElementById('holeRollingBallRadius').value);
3929         gray = backgroundSubtractRollingBall(gray, imageData.width, imageData.height, radius);
3930       } else {
3931         currentStep++;
3932       }
3933
3934       // Step 3: Smoothing
3935       const smoothMethod = document.getElementById('holeSmoothingMethod').value;
3936       if (smoothMethod !== 'none') {
3937         const elapsed = (Date.now() - startTime) / 1000;
3938         const estimated = elapsed * (totalSteps / currentStep) - elapsed;
3939         updateProgress('Applying smoothing filter...', (++currentStep / totalSteps) * 100, estimated);
3940         await sleep(10);
3941
3942         if (smoothMethod === 'gaussian3') {
3943           gray = blur3x3(gray, imageData.width, imageData.height);
3944         } else if (smoothMethod === 'median3') {
3945           gray = medianFilter(gray, imageData.width, imageData.height, 3);
3946         } else if (smoothMethod === 'median5') {
3947           gray = medianFilter(gray, imageData.width, imageData.height, 5);
3948         }
3949       } else {
3950         currentStep++;
3951       }
3952
```

```
3953        // Step 4: Thresholding and labeling
3954        const elapsed = (Date.now() - startTime) / 1000;
3955        const estimated = elapsed * (totalSteps / currentStep) - elapsed;
3956        updateProgress('Detecting holes...', (++currentStep / totalSteps) * 100, estimated);
3957        await sleep(10);
3958
3959        const threshold = parseInt(elements.holeThreshold.value);
3960        const minAreaInput = parseFloat(elements.minHoleArea.value) || 0.001;
3961        // If calibrated, convert from real units back to pixels
3962        const minArea = state.isCalibrated ? Math.round(minAreaInput / Math.pow(state.unitsPerPixel, 2)) :
            Math.round(minAreaInput);
3963        const darkIsObject = document.querySelector('input[name="holePolarity"]:checked').value === 'dark';
3964        const useAdaptive = document.getElementById('adaptiveThresholdHole').checked;
3965        const blockSize = parseInt(document.getElementById('holeAdaptiveBlockSize').value);
3966        const useMorphOpening = document.getElementById('holeMorphologicalOpening').checked;
3967
3968        // Store original morphological setting and temporarily set it for this analysis
3969        const originalMorphSetting = elements.morphologicalOpening ? elements.morphologicalOpening.checked : false;
3970        if (elements.morphologicalOpening) {
3971          elements.morphologicalOpening.checked = useMorphOpening;
3972        }
3973
3974        state.holes = thresholdAndLabelComponents(
3975          gray,
3976          imageData.width,
3977          imageData.height,
3978          threshold,
3979          !darkIsObject,
3980          minArea,
3981          useAdaptive,
3982          blockSize,
3983          roi
3984        );
3985
3986        // Restore original morphological setting
3987        if (elements.morphologicalOpening) {
3988          elements.morphologicalOpening.checked = originalMorphSetting;
3989        }
3990
3991        // Finalize
3992        updateProgress('Finalizing results...', 100, 0);
3993        await sleep(10);
3994
3995        updateHoleResults();
3996        drawOverlay();
3997        updateGuidance();
3998        updateSaveButtonState();
3999
4000        // Show completion briefly before hiding
4001        await sleep(300);
4002        hideProcessingModal();
4003      }
4004
4005      function updateHoleResults() {
4006        const stats = computeStats(state.holes, 'areaPx');
4007        const ecdStats = computeStats(state.holes, 'ecdPx');
4008
4009        elements.holeCount.textContent = stats.count;
4010        elements.holeMeanArea.textContent = formatNumber(stats.mean) + ' px²' +
4011          (state.isCalibrated ? ` (${formatNumber(stats.mean * Math.pow(state.unitsPerPixel, 2))} ${state.unit}²)` : '');
4012        elements.holeMeanECD.textContent = formatNumber(ecdStats.mean) + ' px' +
4013          (state.isCalibrated ? ` (${formatNumber(ecdStats.mean * state.unitsPerPixel)} ${state.unit})` : '');
4014        elements.holeStdDev.textContent = formatNumber(ecdStats.std) + ' px';
4015
4016        elements.holeResults.classList.remove('hidden');
4017        drawHistogram(elements.holeHistogram, state.holes.map(h => h.ecdPx), '#a855f7');
4018        updateStatsButtonsVisibility();
4019      }
4020
4021      function updateMeasurementDisplays() {
4022        // Recalculate real values with new scale
4023        state.lengths.forEach(len => {
4024          len.realLength = state.isCalibrated ? len.pixelLength * state.unitsPerPixel : null;
4025        });
4026        state.areas.forEach(area => {
4027          area.realArea = state.isCalibrated ? area.pixelArea * Math.pow(state.unitsPerPixel, 2) : null;
4028        });
4029        state.particles.forEach(p => {
4030          p.areaReal = state.isCalibrated ? p.areaPx * Math.pow(state.unitsPerPixel, 2) : null;
4031          p.ecdReal = state.isCalibrated ? p.ecdPx * state.unitsPerPixel : null;
4032        });
```

```
    state.holes.forEach(h => {
      h.areaReal = state.isCalibrated ? h.areaPx * Math.pow(state.unitsPerPixel, 2) : null;
      h.ecdReal = state.isCalibrated ? h.ecdPx * state.unitsPerPixel : null;
    });

    updateLengthDisplay();
    updateAngleDisplay();
    updateAreaDisplay();
    if (state.particles.length) updateParticleResults();
    if (state.holes.length) updateHoleResults();
    updateStatsButtonsVisibility();
  }

  // ========== PER-FEATURE EXPORT ==========
  function buildFeatureTSV(type) {
    let tsv = 'Type\tID\tValue_px\tValue_real\tUnit\n';

    if (type === 'length') {
      state.lengths.forEach((len, i) => {
        tsv += `Length\tL${i + 1}\t${len.pixelLength.toFixed(4)}\t${state.isCalibrated ? len.realLength.toFixed(4) :
    ''}\t${state.isCalibrated ? state.unit : 'px'}\n`;
      });
      return tsv;
    }

    if (type === 'angle') {
      state.angles.forEach((ang, i) => {
        const v = Number.isFinite(ang.angleDeg) ? ang.angleDeg : 0;
        tsv += `Angle\tAng${i + 1}\t${v.toFixed(4)}\t${v.toFixed(4)}\tdeg\n`;
      });
      return tsv;
    }

    if (type === 'area') {
      state.areas.forEach((area, i) => {
        tsv += `Area\tA${i + 1}\t${area.pixelArea.toFixed(4)}\t${state.isCalibrated ? area.realArea.toFixed(4) :
    ''}\t${state.isCalibrated ? state.unit + '²' : 'px²'}\n`;
      });
      return tsv;
    }

    if (type === 'particles') {
      state.particles.forEach((p) => {
        tsv += `Particle_Area\tP${p.id}\t${p.areaPx}\t${state.isCalibrated ? p.areaReal.toFixed(4) : ''}\t${state.isCalibra
    ? state.unit + '²' : 'px²'}\n`;
        tsv += `Particle_ECD\tP${p.id}\t${p.ecdPx.toFixed(4)}\t${state.isCalibrated ? p.ecdReal.toFixed(4) :
    ''}\t${state.isCalibrated ? state.unit : 'px'}\n`;
      });
      return tsv;
    }

    if (type === 'holes') {
      state.holes.forEach((h) => {
        tsv += `Hole_Area\tH${h.id}\t${h.areaPx}\t${state.isCalibrated ? h.areaReal.toFixed(4) : ''}\t${state.isCalibrated
    state.unit + '²' : 'px²'}\n`;
        tsv += `Hole_ECD\tH${h.id}\t${h.ecdPx.toFixed(4)}\t${state.isCalibrated ? h.ecdReal.toFixed(4) :
    ''}\t${state.isCalibrated ? state.unit : 'px'}\n`;
      });
      return tsv;
    }

    return tsv;
  }

  function escapeCsvValue(value) {
    const text = String(value ?? '');
    if (!/[",\n\r]/.test(text)) return text;
    return `"${text.replace(/"/g, '""')}"`;
  }

  function buildFeatureCSV(type) {
    const tsv = buildFeatureTSV(type);
    const rows = tsv
      .trim()
      .split('\n')
      .map((line) => line.split('\t').map(escapeCsvValue).join(','));
    return rows.length ? `${rows.join('\n')}\n` : '';
  }

  async function copyFeatureTSV(type) {
    const tsv = buildFeatureTSV(type);
```

```
        const hasRows = tsv.trim().split('\n').length > 1;
        if (!hasRows) return;

        try {
          if (navigator.clipboard && navigator.clipboard.writeText) {
            await navigator.clipboard.writeText(tsv);
            return;
          }
        } catch (e) {
          // Fall back below
        }

        fallbackCopyText(tsv);
      }

      function downloadFeatureCSV(type) {
        const csv = buildFeatureCSV(type);
        const hasRows = csv.trim().split('\n').length > 1;
        if (!hasRows) return;

        const blob = new Blob([csv], { type: 'text/csv;charset=utf-8' });
        const url = URL.createObjectURL(blob);
        const a = document.createElement('a');
        a.href = url;
        a.download = `micromeasure_${type}.csv`;
        a.click();
        URL.revokeObjectURL(url);
      }

      function saveScreenshot() {
        if (!state.image) return;

        // Create a temporary canvas to combine main image and overlay
        const exportCanvas = document.createElement('canvas');
        exportCanvas.width = state.image.width;
        exportCanvas.height = state.image.height;
        const exportCtx = exportCanvas.getContext('2d');

        // Draw the main image
        exportCtx.drawImage(state.image, 0, 0);

        // Draw the overlay on top
        exportCtx.drawImage(elements.overlayCanvas, 0, 0);

        // Convert to blob and download
        exportCanvas.toBlob((blob) => {
          const url = URL.createObjectURL(blob);
          const a = document.createElement('a');
          a.href = url;
          a.download = 'micromeasure_screenshot.png';
          a.click();
          URL.revokeObjectURL(url);
        }, 'image/png');
      }

      function clearAllParticles() {
        state.particles = [];
        if (elements.particleResults) elements.particleResults.classList.add('hidden');
        updateStatsButtonsVisibility();
        drawOverlay();
        updateGuidance();
        updateSaveButtonState();
      }

      function clearAllHoles() {
        state.holes = [];
        if (elements.holeResults) elements.holeResults.classList.add('hidden');
        updateStatsButtonsVisibility();
        drawOverlay();
        updateGuidance();
        updateSaveButtonState();
      }

      function resetProject() {
        state.isCalibrated = false;
        state.calibrationLine = null;
        state.lengths = [];
        state.lengthPoints = [];
        state.anglePoints = [];
        state.angles = [];
        state.editingAngleIndex = null;
```

```
        state.draggedAnglePointIndex = null;
        state.angleEditBackup = null;
        state.areaMode = 'polygon';
        state.areas = [];
        state.areaPoints = [];
        state.editingAreaIndex = null;
        state.draggedPointIndex = null;
        state.isDrawingCircle = false;
        state.circleDraft = null;
        state.particles = [];
        state.holes = [];
        state.roi = null;
        state.isDrawingROI = false;
        state.roiStart = null;
        state.roiMode = null;

        elements.scaleInfo.classList.add('hidden');
        elements.particleResults.classList.add('hidden');
        elements.holeResults.classList.add('hidden');

        // Reset ROI UI
        elements.particleROIInfo.classList.add('hidden');
        elements.selectParticleROI.classList.remove('hidden');
        elements.selectParticleROI.textContent = '■ Select ROI (Draw Rectangle)';
        elements.selectParticleROI.classList.add('btn-secondary');
        elements.selectParticleROI.classList.remove('btn-primary');

        elements.holeROIInfo.classList.add('hidden');
        elements.selectHoleROI.classList.remove('hidden');
        elements.selectHoleROI.textContent = '■ Select ROI (Draw Rectangle)';
        elements.selectHoleROI.classList.add('btn-secondary');
        elements.selectHoleROI.classList.remove('btn-primary');

        if (elements.editAngleModeControls) elements.editAngleModeControls.classList.add('hidden');

        // Reset Area mode UI
        if (elements.closePolygonBtn) elements.closePolygonBtn.classList.add('hidden');
        if (elements.editModeControls) elements.editModeControls.classList.add('hidden');
        if (elements.areaInstructions) {
          elements.areaInstructions.classList.remove('hidden');
          elements.areaInstructions.textContent = 'Polygon: click points to create polygon.';
        }
        if (elements.areaModePolygonBtn) {
          elements.areaModePolygonBtn.classList.add('btn-primary');
          elements.areaModePolygonBtn.classList.remove('btn-secondary');
        }
        if (elements.areaModeCircleBtn) {
          elements.areaModeCircleBtn.classList.add('btn-secondary');
          elements.areaModeCircleBtn.classList.remove('btn-primary');
        }

        updateLengthDisplay();
        updateAngleDisplay();
        updateAreaDisplay();
        drawOverlay();
        updateGuidance();
        updateSaveButtonState();
      }

      function resetToStart() {
        // Clear all measurements/results/state using existing logic
        resetProject();

        // Return to the initial "no image" state
        state.image = null;
        state.imageData = null;
        state.zoom = 100;
        state.panX = 0;
        state.panY = 0;
        state.isPanning = false;

        // Keep onboarding overlay disabled
        state.quickStartDismissed = true;

        // UI reset
        elements.zoomSlider.value = 100;
        elements.zoomValue.textContent = '100%';
        elements.imageDimensions.classList.add('hidden');
        elements.fitToView.classList.add('hidden');
        const panControls = document.getElementById('panControls');
        if (panControls) panControls.classList.add('hidden');
```

```
4270
4271        elements.noImagePlaceholder.classList.remove('hidden');
4272        elements.mainCanvas.classList.add('hidden');
4273        elements.overlayCanvas.classList.add('hidden');
4274
4275        elements.setScaleBtn.disabled = true;
4276        elements.analyzeParticles.disabled = true;
4277        elements.analyzeHoles.disabled = true;
4278        elements.selectParticleROI.disabled = true;
4279        elements.selectHoleROI.disabled = true;
4280
4281        // Hide calibration input panel if open
4282        elements.calibrationInputs.classList.add('hidden');
4283        state.isSettingScale = false;
4284        state.scalePoints = [];
4285
4286        // Clear canvases
4287        mainCtx.clearRect(0, 0, elements.mainCanvas.width, elements.mainCanvas.height);
4288        overlayCtx.clearRect(0, 0, elements.overlayCanvas.width, elements.overlayCanvas.height);
4289
4290        updateGuidance();
4291        updateSaveButtonState();
4292      }
4293
4294      function openResetConfirm() {
4295        if (!elements.resetConfirmModal) return;
4296        elements.resetConfirmModal.classList.remove('hidden');
4297      }
4298
4299      function closeResetConfirm() {
4300        if (!elements.resetConfirmModal) return;
4301        elements.resetConfirmModal.classList.add('hidden');
4302      }
4303
4304      function openSaveImageConfirm() {
4305        if (!elements.saveImageConfirmModal || elements.saveScreenshot.disabled) return;
4306        elements.saveImageConfirmModal.classList.remove('hidden');
4307      }
4308
4309      function closeSaveImageConfirm() {
4310        if (!elements.saveImageConfirmModal) return;
4311        elements.saveImageConfirmModal.classList.add('hidden');
4312      }
4313
4314      function openHomeConfirm() {
4315        if (!elements.homeConfirmModal) return;
4316        elements.homeConfirmModal.classList.remove('hidden');
4317      }
4318
4319      function closeHomeConfirm() {
4320        if (!elements.homeConfirmModal) return;
4321        elements.homeConfirmModal.classList.add('hidden');
4322      }
4323
4324      function openDemoImageConfirm() {
4325        if (!elements.demoImageConfirmModal) return;
4326        elements.demoImageConfirmModal.classList.remove('hidden');
4327      }
4328
4329      function closeDemoImageConfirm() {
4330        if (!elements.demoImageConfirmModal) return;
4331        elements.demoImageConfirmModal.classList.add('hidden');
4332      }
4333
4334      function goToFacilityHomePage() {
4335        window.location.href = 'https://www.inanolab.com/#extras';
4336      }
4337
4338      // ========== EVENT LISTENERS ==========
4339      elements.imageInput.addEventListener('change', (e) => {
4340        if (e.target.files[0]) loadImage(e.target.files[0]);
4341        e.target.value = '';
4342      });
4343
4344      elements.zoomSlider.addEventListener('input', (e) => {
4345        state.zoom = parseInt(e.target.value);
4346        elements.zoomValue.textContent = state.zoom + '%';
4347        if (state.image) updateCanvasTransform();
4348      });
4349
4350      elements.fitToView.addEventListener('click', fitToView);
```

```
4351
4352      // Pan buttons
4353      elements.panLeft.addEventListener('click', () => {
4354        state.panX += 50;
4355        updateCanvasTransform();
4356      });
4357
4358      elements.panRight.addEventListener('click', () => {
4359        state.panX -= 50;
4360        updateCanvasTransform();
4361      });
4362
4363      elements.panUp.addEventListener('click', () => {
4364        state.panY += 50;
4365        updateCanvasTransform();
4366      });
4367
4368      elements.panDown.addEventListener('click', () => {
4369        state.panY -= 50;
4370        updateCanvasTransform();
4371      });
4372
4373      elements.resetPan.addEventListener('click', () => {
4374        state.panX = 0;
4375        state.panY = 0;
4376        updateCanvasTransform();
4377      });
4378
4379      elements.setScaleBtn.addEventListener('click', setScale);
4380      elements.saveScale.addEventListener('click', saveScale);
4381      elements.cancelScale.addEventListener('click', cancelScale);
4382      elements.resetScaleBtn.addEventListener('click', resetScale);
4383      elements.clearLengths.addEventListener('click', clearAllLengths);
4384      if (elements.lengthStatsBtn) elements.lengthStatsBtn.addEventListener('click', () => openStatsModal('length'));
4385      if (elements.lengthHistBtn) elements.lengthHistBtn.addEventListener('click', () => openHistogramModal('length'));
4386      if (elements.clearAngles) elements.clearAngles.addEventListener('click', clearAllAngles);
4387      if (elements.angleStatsBtn) elements.angleStatsBtn.addEventListener('click', () => openStatsModal('angle'));
4388      if (elements.angleHistBtn) elements.angleHistBtn.addEventListener('click', () => openHistogramModal('angle'));
4389      if (elements.saveAngleEdit) elements.saveAngleEdit.addEventListener('click', saveAngleEdit);
4390      if (elements.cancelAngleEdit) elements.cancelAngleEdit.addEventListener('click', cancelAngleEdit);
4391      elements.saveLengthEdit.addEventListener('click', saveLengthEdit);
4392      elements.cancelLengthEdit.addEventListener('click', cancelLengthEdit);
4393      elements.clearAreas.addEventListener('click', clearAllAreas);
4394      if (elements.areaStatsBtn) elements.areaStatsBtn.addEventListener('click', () => openStatsModal('area'));
4395      if (elements.areaHistBtn) elements.areaHistBtn.addEventListener('click', () => openHistogramModal('area'));
4396      if (elements.areaModePolygonBtn) elements.areaModePolygonBtn.addEventListener('click', () => setAreaMode('polygon'));
4397      if (elements.areaModeCircleBtn) elements.areaModeCircleBtn.addEventListener('click', () => setAreaMode('circle'));
4398      elements.closePolygonBtn.addEventListener('click', () => {
4399        if (state.areaMode === 'polygon' && state.areaPoints.length >= 3) {
4400          addAreaMeasurement(state.areaPoints);
4401          state.areaPoints = [];
4402          elements.closePolygonBtn.classList.add('hidden');
4403          drawOverlay();
4404        }
4405      });
4406      elements.saveAreaEdit.addEventListener('click', saveAreaEdit);
4407      elements.cancelAreaEdit.addEventListener('click', cancelAreaEdit);
4408      elements.analyzeParticles.addEventListener('click', analyzeParticles);
4409      elements.analyzeHoles.addEventListener('click', analyzeHoles);
4410      if (elements.particleStatsBtn) elements.particleStatsBtn.addEventListener('click', () => openStatsModal('particles'));
4411      if (elements.holeStatsBtn) elements.holeStatsBtn.addEventListener('click', () => openStatsModal('holes'));
4412      if (elements.particleHistBtn) elements.particleHistBtn.addEventListener('click', () => openHistogramModal('particles'));
4413      if (elements.holeHistBtn) elements.holeHistBtn.addEventListener('click', () => openHistogramModal('holes'));
4414      elements.selectParticleROI.addEventListener('click', () => startROISelection('particles'));
4415      elements.clearParticleROI.addEventListener('click', () => clearROI('particles'));
4416      elements.selectHoleROI.addEventListener('click', () => startROISelection('holes'));
4417      elements.clearHoleROI.addEventListener('click', () => clearROI('holes'));
4418      elements.saveScreenshot.addEventListener('click', openSaveImageConfirm);
4419      if (elements.homeBtn) elements.homeBtn.addEventListener('click', openHomeConfirm);
4420      if (elements.loadDemoBtn) elements.loadDemoBtn.addEventListener('click', openDemoImageConfirm);
4421
4422      // Per-feature export buttons
4423      if (elements.copyLengthTSVBtn) elements.copyLengthTSVBtn.addEventListener('click', () => copyFeatureTSV('length'));
4424      if (elements.downloadLengthTSVBtn) elements.downloadLengthTSVBtn.addEventListener('click', () =>
           downloadFeatureCSV('length'));
4425      if (elements.copyAngleTSVBtn) elements.copyAngleTSVBtn.addEventListener('click', () => copyFeatureTSV('angle'));
4426      if (elements.downloadAngleTSVBtn) elements.downloadAngleTSVBtn.addEventListener('click', () => downloadFeatureCSV('angle'
4427      if (elements.copyAreaTSVBtn) elements.copyAreaTSVBtn.addEventListener('click', () => copyFeatureTSV('area'));
4428      if (elements.downloadAreaTSVBtn) elements.downloadAreaTSVBtn.addEventListener('click', () => downloadFeatureCSV('area'));
4429      if (elements.copyParticlesTSVBtn) elements.copyParticlesTSVBtn.addEventListener('click', () => copyFeatureTSV('particles'
4430      if (elements.downloadParticlesTSVBtn) elements.downloadParticlesTSVBtn.addEventListener('click', () =>
```

```
  downloadFeatureCSV('particles'));
4431      if (elements.copyHolesTSVBtn) elements.copyHolesTSVBtn.addEventListener('click', () => copyFeatureTSV('holes'));
4432      if (elements.downloadHolesTSVBtn) elements.downloadHolesTSVBtn.addEventListener('click', () => downloadFeatureCSV('holes'
4433
4434      if (elements.clearParticles) elements.clearParticles.addEventListener('click', clearAllParticles);
4435      if (elements.clearHoles) elements.clearHoles.addEventListener('click', clearAllHoles);
4436
4437      // Header reset with confirmation
4438      if (elements.headerResetBtn) elements.headerResetBtn.addEventListener('click', openResetConfirm);
4439      if (elements.confirmResetNo) elements.confirmResetNo.addEventListener('click', closeResetConfirm);
4440      if (elements.confirmResetYes) elements.confirmResetYes.addEventListener('click', () => {
4441        closeResetConfirm();
4442        resetToStart();
4443      });
4444      if (elements.resetConfirmModal) {
4445        elements.resetConfirmModal.addEventListener('click', (e) => {
4446          if (e.target === elements.resetConfirmModal) closeResetConfirm();
4447        });
4448      }
4449      if (elements.confirmSaveImageNo) elements.confirmSaveImageNo.addEventListener('click', closeSaveImageConfirm);
4450      if (elements.confirmSaveImageYes) elements.confirmSaveImageYes.addEventListener('click', () => {
4451        closeSaveImageConfirm();
4452        saveScreenshot();
4453      });
4454      if (elements.saveImageConfirmModal) {
4455        elements.saveImageConfirmModal.addEventListener('click', (e) => {
4456          if (e.target === elements.saveImageConfirmModal) closeSaveImageConfirm();
4457        });
4458      }
4459      if (elements.confirmHomeNo) elements.confirmHomeNo.addEventListener('click', closeHomeConfirm);
4460      if (elements.confirmHomeYes) elements.confirmHomeYes.addEventListener('click', () => {
4461        closeHomeConfirm();
4462        goToFacilityHomePage();
4463      });
4464      if (elements.homeConfirmModal) {
4465        elements.homeConfirmModal.addEventListener('click', (e) => {
4466          if (e.target === elements.homeConfirmModal) closeHomeConfirm();
4467        });
4468      }
4469      if (elements.confirmDemoImageNo) elements.confirmDemoImageNo.addEventListener('click', closeDemoImageConfirm);
4470      if (elements.confirmDemoImageYes) elements.confirmDemoImageYes.addEventListener('click', () => {
4471        closeDemoImageConfirm();
4472        loadDemoImage();
4473      });
4474      if (elements.demoImageConfirmModal) {
4475        elements.demoImageConfirmModal.addEventListener('click', (e) => {
4476          if (e.target === elements.demoImageConfirmModal) closeDemoImageConfirm();
4477        });
4478      }
4479
4480      // Documentation modal
4481      document.getElementById('helpBtn').addEventListener('click', () => {
4482        document.getElementById('docModal').classList.remove('hidden');
4483        try { document.getElementById('closeDocBtn')?.focus(); } catch {}
4484      });
4485
4486      document.getElementById('closeDocBtn').addEventListener('click', () => {
4487        document.getElementById('docModal').classList.add('hidden');
4488      });
4489
4490      // Close documentation modal when clicking outside
4491      document.getElementById('docModal').addEventListener('click', (e) => {
4492        if (e.target.id === 'docModal') {
4493          document.getElementById('docModal').classList.add('hidden');
4494        }
4495      });
4496
4497      // Close documentation modal with Escape (match Roughness Visualizer)
4498      document.addEventListener('keydown', (e) => {
4499        if (e.key !== 'Escape') return;
4500        const docModal = document.getElementById('docModal');
4501        if (docModal && !docModal.classList.contains('hidden')) docModal.classList.add('hidden');
4502      });
4503
4504      // Quick Start overlay
4505      if (elements.quickStartOk) {
4506        elements.quickStartOk.addEventListener('click', () => {
4507          state.quickStartDismissed = true;
4508          updateGuidance();
4509        });
4510      }
```

```
4511
4512      // Particle preprocessing UI controls
4513      elements.backgroundSubtraction.addEventListener('change', (e) => {
4514        elements.backgroundOptions.classList.toggle('hidden', !e.target.checked);
4515      });
4516
4517      elements.adaptiveThreshold.addEventListener('change', (e) => {
4518        elements.manualThresholdControls.classList.toggle('hidden', e.target.checked);
4519        elements.adaptiveThresholdControls.classList.toggle('hidden', !e.target.checked);
4520      });
4521
4522      // Hole preprocessing UI controls
4523      document.getElementById('holeBackgroundSubtraction').addEventListener('change', (e) => {
4524        document.getElementById('holeBackgroundOptions').classList.toggle('hidden', !e.target.checked);
4525      });
4526
4527      document.getElementById('adaptiveThresholdHole').addEventListener('change', (e) => {
4528        document.getElementById('manualThresholdHoleControls').classList.toggle('hidden', e.target.checked);
4529        document.getElementById('adaptiveThresholdHoleControls').classList.toggle('hidden', !e.target.checked);
4530      });
4531
4532      document.getElementById('autoThresholdHoleBtn').addEventListener('click', () => {
4533        if (!state.image) return;
4534
4535        const imageData = getImageData();
4536        let gray = toGrayscale(imageData);
4537
4538        // Apply same preprocessing as hole analysis
4539        if (document.getElementById('holeBackgroundSubtraction').checked) {
4540          const radius = parseInt(document.getElementById('holeRollingBallRadius').value);
4541          gray = backgroundSubtractRollingBall(gray, imageData.width, imageData.height, radius);
4542        }
4543
4544        const smoothMethod = document.getElementById('holeSmoothingMethod').value;
4545        if (smoothMethod === 'gaussian3') {
4546          gray = blur3x3(gray, imageData.width, imageData.height);
4547        } else if (smoothMethod === 'median3') {
4548          gray = medianFilter(gray, imageData.width, imageData.height, 3);
4549        } else if (smoothMethod === 'median5') {
4550          gray = medianFilter(gray, imageData.width, imageData.height, 5);
4551        }
4552
4553        const threshold = otsuThreshold(gray);
4554        elements.holeThreshold.value = threshold;
4555        elements.holeThresholdValue.textContent = threshold;
4556      });
4557
4558      // Descriptive stats modal controls
4559      if (elements.closeStatsModalBtn) elements.closeStatsModalBtn.addEventListener('click', closeStatsModal);
4560      if (elements.closeStatsModalBtnBottom) elements.closeStatsModalBtnBottom.addEventListener('click', closeStatsModal);
4561      if (elements.copyStatsTSVBtn) elements.copyStatsTSVBtn.addEventListener('click', copyStatsTSV);
4562      if (elements.statsTypeBtns) {
4563        elements.statsTypeBtns.forEach(btn => {
4564          btn.addEventListener('click', () => setStatsType(btn.dataset.statsType));
4565        });
4566      }
4567      if (elements.descriptiveStatsModal) {
4568        elements.descriptiveStatsModal.addEventListener('click', (e) => {
4569          if (e.target === elements.descriptiveStatsModal) closeStatsModal();
4570        });
4571      }
4572
4573      // Histogram modal controls
4574      if (elements.closeHistModalBtn) elements.closeHistModalBtn.addEventListener('click', closeHistogramModal);
4575      if (elements.closeHistModalBtnBottom) elements.closeHistModalBtnBottom.addEventListener('click', closeHistogramModal);
4576      if (elements.downloadHistSvgBtn) elements.downloadHistSvgBtn.addEventListener('click', downloadCurrentHistogramSvg);
4577      if (elements.histBinsNumber) {
4578        elements.histBinsNumber.addEventListener('input', () => {
4579          const v = Math.max(3, Math.min(80, parseInt(elements.histBinsNumber.value || '15', 10)));
4580          elements.histBinsNumber.value = String(v);
4581          renderHistogramModal();
4582        });
4583      }
4584      if (elements.histXTicksNumber) {
4585        elements.histXTicksNumber.addEventListener('input', () => {
4586          const v = Math.max(2, Math.min(12, parseInt(elements.histXTicksNumber.value || '6', 10)));
4587          elements.histXTicksNumber.value = String(v);
4588          renderHistogramModal();
4589        });
4590      }
4591      if (elements.histYTicksNumber) {
```

```
    elements.histYTicksNumber.addEventListener('input', () => {
      const v = Math.max(2, Math.min(12, parseInt(elements.histYTicksNumber.value || '6', 10)));
      elements.histYTicksNumber.value = String(v);
      renderHistogramModal();
    });
  }
  if (elements.histMetricSelect) elements.histMetricSelect.addEventListener('change', renderHistogramModal);
  if (elements.histTypeBtns) {
    elements.histTypeBtns.forEach(btn => {
      btn.addEventListener('click', () => setHistType(btn.dataset.histType));
    });
  }
  if (elements.histogramModal) {
    elements.histogramModal.addEventListener('click', (e) => {
      if (e.target === elements.histogramModal) closeHistogramModal();
    });
  }

  elements.autoThresholdBtn.addEventListener('click', () => {
    if (!state.image) return;

    const imageData = getImageData();
    let gray = toGrayscale(imageData);

    // Apply same preprocessing as particle analysis
    if (elements.backgroundSubtraction.checked) {
      const radius = parseInt(elements.rollingBallRadius.value);
      gray = backgroundSubtractRollingBall(gray, imageData.width, imageData.height, radius);
    }

    const smoothMethod = elements.smoothingMethod.value;
    if (smoothMethod === 'gaussian3') {
      gray = blur3x3(gray, imageData.width, imageData.height);
    } else if (smoothMethod === 'median3') {
      gray = medianFilter(gray, imageData.width, imageData.height, 3);
    } else if (smoothMethod === 'median5') {
      gray = medianFilter(gray, imageData.width, imageData.height, 5);
    }

    const threshold = otsuThreshold(gray);
    elements.particleThreshold.value = threshold;
    elements.thresholdValue.textContent = threshold;
  });

  elements.particleThreshold.addEventListener('input', (e) => {
    elements.thresholdValue.textContent = e.target.value;
  });

  elements.holeThreshold.addEventListener('input', (e) => {
    elements.holeThresholdValue.textContent = e.target.value;
  });

  // Tab switching
  elements.tabBtns.forEach(btn => {
    btn.addEventListener('click', () => {
      elements.tabBtns.forEach(b => b.classList.remove('active'));
      btn.classList.add('active');

      const tab = btn.dataset.tab;
      state.currentTool = tab;

      // Clear in-progress area when switching away from area tool
      if (tab !== 'area') {
        let needsRedraw = false;

        if (state.areaPoints.length > 0) {
          state.areaPoints = [];
          elements.closePolygonBtn.classList.add('hidden');
          needsRedraw = true;
        }

        if (state.isDrawingCircle || state.circleDraft) {
          state.isDrawingCircle = false;
          state.circleDraft = null;
          _circlePointerId = null;
          needsRedraw = true;
        }

        if (needsRedraw) drawOverlay();
      }
```

```
        // Clear in-progress angle when switching away from angle tool
        if (tab !== 'angle' && state.anglePoints.length > 0) {
          state.anglePoints = [];
          drawOverlay();
        }

        document.querySelectorAll('.tab-content').forEach(content => content.classList.add('hidden'));
        document.getElementById(tab + 'Tab').classList.remove('hidden');
      });
    });

    // Canvas interactions (Pointer Events)
    // Unifies mouse + touch + pen, and makes edit-handle dragging work on phones.
    const _activePointers = new Map(); // pointerId -> { x, y, startX, startY, startTs, pointerType }
    let _panPointerType = null;
    let _panPrimaryPointerId = null;
    let _roiPointerId = null;
    let _dragPointerId = null;
    let _circlePointerId = null;

    const _lastTap = {
      mouse: { ts: 0, x: 0, y: 0 },
      touch: { ts: 0, x: 0, y: 0 },
      pen: { ts: 0, x: 0, y: 0 },
      unknown: { ts: 0, x: 0, y: 0 }
    };

    function pointerCanvasCoords(e) {
      const rect = elements.overlayCanvas.getBoundingClientRect();
      const scale = state.zoom / 100;
      return {
        x: (e.clientX - rect.left) / scale,
        y: (e.clientY - rect.top) / scale
      };
    }

    function canvasHitRadius() {
      // Aim for ~18 screen px hit radius regardless of zoom.
      const scale = state.zoom / 100;
      const r = 18 / (scale || 1);
      return Math.max(6, Math.min(30, r));
    }

    function beginEditHandleDragAt(coords) {
      const hit = canvasHitRadius();

      if (state.editingLengthIndex !== null) {
        const len = state.lengths[state.editingLengthIndex];
        const dist1 = Math.hypot(coords.x - len.p1.x, coords.y - len.p1.y);
        const dist2 = Math.hypot(coords.x - len.p2.x, coords.y - len.p2.y);
        if (dist1 <= hit) {
          state.draggedLengthPoint = 'p1';
          return true;
        }
        if (dist2 <= hit) {
          state.draggedLengthPoint = 'p2';
          return true;
        }
      }

      if (state.editingAreaIndex !== null) {
        const area = state.areas[state.editingAreaIndex];
        for (let i = 0; i < area.points.length; i++) {
          const p = area.points[i];
          const dist = Math.hypot(coords.x - p.x, coords.y - p.y);
          if (dist <= hit) {
            state.draggedPointIndex = i;
            return true;
          }
        }
      }

      if (state.editingAngleIndex !== null) {
        const ang = state.angles[state.editingAngleIndex];
        const pts = [ang.p1, ang.p2, ang.p3];
        for (let i = 0; i < pts.length; i++) {
          const p = pts[i];
          const dist = Math.hypot(coords.x - p.x, coords.y - p.y);
          if (dist <= hit) {
            state.draggedAnglePointIndex = i;
            return true;
```

```
          }
        }
      }

      return false;
    }

    function handleCanvasTap(coords) {
      if (!state.image) return;

      // Don't allow other interactions while drawing ROI
      if (state.isDrawingROI) return;

      // Setting scale mode (exactly 2 points)
      if (state.isSettingScale) {
        if (state.scalePoints.length >= 2) return;
        state.scalePoints.push(coords);
        if (state.scalePoints.length === 2) {
          elements.setScaleBtn.textContent = 'Set Scale';
        }
        drawOverlay();
        return;
      }

      // Length tool
      if (state.currentTool === 'length') {
        state.lengthPoints.push(coords);
        if (state.lengthPoints.length === 2) {
          const len = measureLength(state.lengthPoints[0], state.lengthPoints[1]);
          addLengthMeasurement(len);
          state.lengthPoints = [];
        }
        drawOverlay();
        return;
      }

      // Angle tool (3 clicks; 2nd click is vertex)
      if (state.currentTool === 'angle') {
        // In edit mode, don't add new points with regular click
        if (state.editingAngleIndex !== null) return;

        state.anglePoints.push(coords);
        if (state.anglePoints.length === 3) {
          addAngleMeasurement(state.anglePoints[0], state.anglePoints[1], state.anglePoints[2]);
          state.anglePoints = [];
        }
        drawOverlay();
        return;
      }

      // Area tool
      if (state.currentTool === 'area') {
        // In edit mode, don't add new points with regular click
        if (state.editingAreaIndex !== null) return;

        // Circle mode uses drag-to-draw; ignore tap-to-add behavior.
        if (state.areaMode !== 'polygon') return;

        state.areaPoints.push(coords);

        // Show close button when 3+ points
        if (state.areaPoints.length >= 3) {
          elements.closePolygonBtn.classList.remove('hidden');
          elements.closePolygonBtn.textContent = `Close Polygon (${state.areaPoints.length} points)`;
        }

        drawOverlay();
        return;
      }
    }

    function isTap(pointer) {
      const dt = Date.now() - pointer.startTs;
      if (dt > 300) return false;
      const dx = pointer.x - pointer.startX;
      const dy = pointer.y - pointer.startY;
      const dist = Math.hypot(dx, dy);
      // Use a screen-space threshold (~8px)
      return dist <= 8;
    }

```

```
    function isDoubleTapForType(pointerType, x, y) {
      const t = _lastTap[pointerType] || _lastTap.unknown;
      const now = Date.now();
      const dt = now - t.ts;
      const dist = Math.hypot(x - t.x, y - t.y);
      return dt > 0 && dt < 300 && dist < 20;
    }

    function recordTap(pointerType, x, y) {
      const slot = _lastTap[pointerType] || _lastTap.unknown;
      slot.ts = Date.now();
      slot.x = x;
      slot.y = y;
    }

    function clearTap(pointerType) {
      const slot = _lastTap[pointerType] || _lastTap.unknown;
      slot.ts = 0;
      slot.x = 0;
      slot.y = 0;
    }

    function getPanMidpoint() {
      let sumX = 0;
      let sumY = 0;
      let n = 0;
      for (const p of _activePointers.values()) {
        if (_panPointerType && p.pointerType !== _panPointerType) continue;
        sumX += p.x;
        sumY += p.y;
        n++;
      }
      if (!n) return null;
      return { x: sumX / n, y: sumY / n };
    }

    function endPointerInteraction(pointerId) {
      if (_roiPointerId === pointerId) {
        _roiPointerId = null;
        // Finish ROI drawing
        if (state.isDrawingROI && state.roiStart) {
          finishROISelection();
        }
      }

      if (_circlePointerId === pointerId) {
        _circlePointerId = null;
        state.isDrawingCircle = false;
        state.circleDraft = null;
      }

      if (_dragPointerId === pointerId) {
        _dragPointerId = null;
        state.draggedPointIndex = null;
        state.draggedLengthPoint = null;
        state.draggedAnglePointIndex = null;
      }

      if (_panPrimaryPointerId === pointerId) {
        _panPrimaryPointerId = null;
      }

      // If we were touch/pen panning and fewer than 2 pointers remain, stop panning.
      if (state.isPanning && (_panPointerType === 'touch' || _panPointerType === 'pen')) {
        let remaining = 0;
        for (const p of _activePointers.values()) {
          if (p.pointerType === _panPointerType) remaining++;
        }
        if (remaining < 2) {
          state.isPanning = false;
          _panPointerType = null;
        }
      }

      // If mouse panning and primary pointer released, stop.
      if (state.isPanning && _panPointerType === 'mouse' && !_panPrimaryPointerId) {
        state.isPanning = false;
        _panPointerType = null;
      }

      const cursor = (state.editingAreaIndex !== null || state.editingLengthIndex !== null || state.editingAngleIndex !== nul
```

```
        ? 'pointer'
        : 'crosshair';
      elements.overlayCanvas.style.cursor = cursor;
    }

    elements.overlayCanvas.addEventListener('pointerdown', (e) => {
      if (!state.image) return;

      // Track pointers for multi-touch panning.
      _activePointers.set(e.pointerId, {
        x: e.clientX,
        y: e.clientY,
        startX: e.clientX,
        startY: e.clientY,
        startTs: Date.now(),
        pointerType: e.pointerType || 'unknown'
      });

      // Capture so we keep receiving move/up even if finger leaves canvas.
      if (elements.overlayCanvas.setPointerCapture) {
        elements.overlayCanvas.setPointerCapture(e.pointerId);
      }

      // Mouse pan: middle button OR Alt+Left
      if ((e.pointerType === 'mouse') && (e.button === 1 || (e.button === 0 && e.altKey))) {
        state.isPanning = true;
        _panPointerType = 'mouse';
        _panPrimaryPointerId = e.pointerId;
        state.lastPanX = e.clientX;
        state.lastPanY = e.clientY;
        elements.overlayCanvas.style.cursor = 'grabbing';
        e.preventDefault();
        return;
      }

      // Touch/pen pan begins when 2 pointers are active.
      if ((e.pointerType === 'touch' || e.pointerType === 'pen')) {
        let count = 0;
        for (const p of _activePointers.values()) {
          if (p.pointerType === e.pointerType) count++;
        }
        if (count >= 2) {
          if (state.isDrawingCircle || state.circleDraft) {
            state.isDrawingCircle = false;
            state.circleDraft = null;
            _circlePointerId = null;
            drawOverlay();
          }
          state.isPanning = true;
          _panPointerType = e.pointerType;
          const mid = getPanMidpoint();
          if (mid) {
            state.lastPanX = mid.x;
            state.lastPanY = mid.y;
          }
          elements.overlayCanvas.style.cursor = 'grabbing';
          e.preventDefault();
          return;
        }
      }

      // ROI drawing mode (single pointer)
      if (state.isDrawingROI && (e.button === 0 || e.pointerType !== 'mouse')) {
        const coords = pointerCanvasCoords(e);
        state.roiStart = coords;
        _roiPointerId = e.pointerId;
        e.preventDefault();
        return;
      }

      // Edit handle dragging (single pointer)
      if ((e.button === 0 || e.pointerType !== 'mouse') && (state.editingLengthIndex !== null || state.editingAreaIndex !== n
      || state.editingAngleIndex !== null)) {
        const coords = pointerCanvasCoords(e);
        if (beginEditHandleDragAt(coords)) {
          _dragPointerId = e.pointerId;
          elements.overlayCanvas.style.cursor = 'move';
          e.preventDefault();
          return;
        }
      }
```

```
4996
4997        // Area: Circle drawing mode (single pointer drag)
4998        if ((e.button === 0 || e.pointerType !== 'mouse') && state.currentTool === 'area' && state.areaMode === 'circle' &&
            state.editingAreaIndex === null && _circlePointerId === null) {
4999          const coords = pointerCanvasCoords(e);
5000          state.isDrawingCircle = true;
5001          state.circleDraft = { center: coords, radiusPx: 0 };
5002          _circlePointerId = e.pointerId;
5003          drawOverlay();
5004          e.preventDefault();
5005          return;
5006        }
5007      }, { passive: false });
5008
5009      elements.overlayCanvas.addEventListener('pointermove', (e) => {
5010        const p = _activePointers.get(e.pointerId);
5011        if (p) {
5012          p.x = e.clientX;
5013          p.y = e.clientY;
5014        }
5015
5016        // Panning
5017        if (state.isPanning) {
5018          if (_panPointerType === 'mouse') {
5019            if (e.pointerId !== _panPrimaryPointerId) return;
5020            state.panX += e.clientX - state.lastPanX;
5021            state.panY += e.clientY - state.lastPanY;
5022            state.lastPanX = e.clientX;
5023            state.lastPanY = e.clientY;
5024            updateCanvasTransform();
5025            e.preventDefault();
5026            return;
5027          }
5028
5029          // Touch/pen panning uses midpoint
5030          const mid = getPanMidpoint();
5031          if (mid) {
5032            state.panX += mid.x - state.lastPanX;
5033            state.panY += mid.y - state.lastPanY;
5034            state.lastPanX = mid.x;
5035            state.lastPanY = mid.y;
5036            updateCanvasTransform();
5037            e.preventDefault();
5038            return;
5039          }
5040        }
5041
5042        // ROI drawing
5043        if (state.isDrawingROI && state.roiStart && _roiPointerId === e.pointerId) {
5044          const coords = pointerCanvasCoords(e);
5045          const x = Math.min(state.roiStart.x, coords.x);
5046          const y = Math.min(state.roiStart.y, coords.y);
5047          const width = Math.abs(coords.x - state.roiStart.x);
5048          const height = Math.abs(coords.y - state.roiStart.y);
5049          state.roi = { x, y, width, height };
5050          drawOverlay();
5051          e.preventDefault();
5052          return;
5053        }
5054
5055        // Area: circle draft update
5056        if (state.isDrawingCircle && _circlePointerId === e.pointerId && state.circleDraft) {
5057          const coords = pointerCanvasCoords(e);
5058          const c = state.circleDraft.center;
5059          state.circleDraft.radiusPx = Math.hypot(coords.x - c.x, coords.y - c.y);
5060          drawOverlay();
5061          e.preventDefault();
5062          return;
5063        }
5064
5065        // Drag length endpoint
5066        if (_dragPointerId === e.pointerId && state.draggedLengthPoint !== null && state.editingLengthIndex !== null) {
5067          const coords = pointerCanvasCoords(e);
5068          const len = state.lengths[state.editingLengthIndex];
5069          len[state.draggedLengthPoint] = coords;
5070          drawOverlay();
5071          e.preventDefault();
5072          return;
5073        }
5074
5075        // Drag area vertex
```

```
      if (_dragPointerId === e.pointerId && state.draggedPointIndex !== null && state.editingAreaIndex !== null) {
        const coords = pointerCanvasCoords(e);
        const area = state.areas[state.editingAreaIndex];
        area.points[state.draggedPointIndex] = coords;
        drawOverlay();
        e.preventDefault();
        return;
      }

      // Drag angle handle
      if (_dragPointerId === e.pointerId && state.draggedAnglePointIndex !== null && state.editingAngleIndex !== null) {
        const coords = pointerCanvasCoords(e);
        const ang = state.angles[state.editingAngleIndex];
        if (state.draggedAnglePointIndex === 0) ang.p1 = coords;
        if (state.draggedAnglePointIndex === 1) ang.p2 = coords;
        if (state.draggedAnglePointIndex === 2) ang.p3 = coords;
        ang.angleDeg = computeSelectedAngleDeg(ang.p1, ang.p2, ang.p3, !!ang.reflex);
        drawOverlay();
        e.preventDefault();
        return;
      }
    }, { passive: false });

    function handlePointerUpOrCancel(e) {
      const p = _activePointers.get(e.pointerId);
      if (!p) {
        endPointerInteraction(e.pointerId);
        return;
      }

      const pointerType = p.pointerType || 'unknown';
      const wasDragging = (_dragPointerId === e.pointerId);
      const wasROI = (_roiPointerId === e.pointerId);
      const wasCircle = (_circlePointerId === e.pointerId);

      // Update pointer end position
      p.x = e.clientX;
      p.y = e.clientY;

      // Remove pointer now; this affects multi-touch panning end logic.
      _activePointers.delete(e.pointerId);

      // Circle mode: finalize or cancel circle drawing on pointer up/cancel.
      if (wasCircle && state.circleDraft) {
        const endCoords = pointerCanvasCoords(e);
        const center = state.circleDraft.center;
        const radiusPx = Math.hypot(endCoords.x - center.x, endCoords.y - center.y);

        state.isDrawingCircle = false;
        state.circleDraft = null;
        _circlePointerId = null;

        // Only commit if we are still in circle mode and this is a normal pointer-up.
        if (e.type === 'pointerup' && state.currentTool === 'area' && state.areaMode === 'circle' && radiusPx >= 2) {
          addCircleAreaMeasurement(center, radiusPx);
        } else {
          drawOverlay();
        }

        clearTap(pointerType);
        endPointerInteraction(e.pointerId);
        e.preventDefault();
        return;
      }

      // Consider tap ONLY if it wasn't a drag/ROI/pan
      const tappable = !state.isPanning && !wasDragging && !wasROI;
      if (tappable && isTap(p)) {
        const nowX = e.clientX;
        const nowY = e.clientY;
        const doubleTap = isDoubleTapForType(pointerType, nowX, nowY);

        if (pointerType === 'touch' || pointerType === 'pen') {
          // Touch/pen: on double-tap, close polygon without adding another point.
          if (doubleTap) {
            if (state.currentTool === 'area' && state.areaMode === 'polygon' && state.areaPoints.length >= 3) {
              addAreaMeasurement(state.areaPoints);
              state.areaPoints = [];
            }
            clearTap(pointerType);
          } else {
```

```
            recordTap(pointerType, nowX, nowY);
            handleCanvasTap(pointerCanvasCoords(e));
          }
        } else {
          // Mouse: preserve old behavior (second click adds point; then we close if it was a double-click).
          const coords = pointerCanvasCoords(e);
          handleCanvasTap(coords);
          if (doubleTap) {
            if (state.currentTool === 'area' && state.areaMode === 'polygon' && state.areaPoints.length >= 3) {
              addAreaMeasurement(state.areaPoints);
              state.areaPoints = [];
            }
            clearTap(pointerType);
          } else {
            recordTap(pointerType, nowX, nowY);
          }
        }
      }

      endPointerInteraction(e.pointerId);
    }

    elements.overlayCanvas.addEventListener('pointerup', handlePointerUpOrCancel, { passive: false });
    elements.overlayCanvas.addEventListener('pointercancel', handlePointerUpOrCancel, { passive: false });

    // Set crosshair cursor when image loaded
    elements.overlayCanvas.style.cursor = 'crosshair';
    // Prevent browser scrolling/zoom gestures from stealing pointer events on mobile.
    elements.overlayCanvas.style.touchAction = 'none';
    // Hint browser that these elements will transform frequently
    elements.mainCanvas.style.willChange = 'transform';
    elements.overlayCanvas.style.willChange = 'transform';

    // Mouse wheel zoom
    elements.canvasWrapper.addEventListener('wheel', (e) => {
      if (!state.image) return;

      e.preventDefault();

      const zoomStep = 10;
      const delta = e.deltaY < 0 ? zoomStep : -zoomStep;
      const newZoom = Math.max(50, Math.min(500, state.zoom + delta));

      if (newZoom !== state.zoom) {
        state.zoom = newZoom;
        elements.zoomSlider.value = state.zoom;
        elements.zoomValue.textContent = state.zoom + '%';
        updateCanvasTransform();
      }
    }, { passive: false });

    // Keyboard controls for panning
    document.addEventListener('keydown', (e) => {
      if (!state.image) return;

      const panStep = 50;
      let handled = false;

      switch(e.key) {
        case 'ArrowLeft':
          state.panX += panStep;
          handled = true;
          break;
        case 'ArrowRight':
          state.panX -= panStep;
          handled = true;
          break;
        case 'ArrowUp':
          state.panY += panStep;
          handled = true;
          break;
        case 'ArrowDown':
          state.panY -= panStep;
          handled = true;
          break;
      }

      if (handled) {
        e.preventDefault();
        updateCanvasTransform();
      }
```

```
    });

    // Recenter on viewport changes (mobile orientation/resize)
    let _resizeTimer = null;
    function _scheduleTransformUpdate() {
      if (!state.image) return;
      if (_resizeTimer) clearTimeout(_resizeTimer);
      _resizeTimer = setTimeout(() => {
        // Recompute fit on viewport changes for mobile usability
        fitToView();
      }, 150);
    }
    window.addEventListener('resize', _scheduleTransformUpdate);
    window.addEventListener('orientationchange', _scheduleTransformUpdate);
    if (window.visualViewport) {
      window.visualViewport.addEventListener('resize', _scheduleTransformUpdate);
    }

    // Drag and Drop support
    elements.canvasWrapper.addEventListener('dragover', (e) => {
      e.preventDefault();
      e.stopPropagation();
      elements.canvasWrapper.classList.add('drag-over');
    });

    elements.canvasWrapper.addEventListener('dragleave', (e) => {
      e.preventDefault();
      e.stopPropagation();
      // Only remove class if leaving the wrapper itself, not child elements
      if (e.target === elements.canvasWrapper) {
        elements.canvasWrapper.classList.remove('drag-over');
      }
    });

    elements.canvasWrapper.addEventListener('drop', (e) => {
      e.preventDefault();
      e.stopPropagation();
      elements.canvasWrapper.classList.remove('drag-over');

      const files = e.dataTransfer.files;
      if (files.length > 0) {
        loadImage(files[0]);
      }
    });

    // Mobile-friendly tap to load (fallback when drag-drop isn't available)
    elements.canvasWrapper.addEventListener('click', () => {
      if (!state.image && elements.imageInput) {
        elements.imageInput.click();
      }
    });
    elements.canvasWrapper.addEventListener('touchend', () => {
      if (!state.image && elements.imageInput) {
        elements.imageInput.click();
      }
    });

    // Initial button state
    updateStatsButtonsVisibility();
    updateGuidance();
  </script>
 </body>
</html>
```